



IT DEPARTMENT

End of Studies Project Report

Presented to Obtain:

National Diploma of Computer Engineering

SPECIALTY: SOFTWARE ENGINEERING-SOFTWARE ARCHITECTURE

**Agile Deep Assistant for Automatic sprint tasks estimation
generation and team-evaluation**

Hosting Company



Elaborated by:

Yasmine Ben Abdeljelil

Presented on the:

09/07/2024 in front of the jury:

President:

Ms. KANZARI Dalel

ISSAT of sousse

Examiner:

Mr. BEN AHMED Sofiene

ISSAT of sousse

Academic supervisor:

Ms. BELGACEM Selma

ISSAT of sousse

Company supervisor:

Mr. GUERSENE Oussama

ENVAST

Academic Year: 2023 / 2024

Subject Code: FI-GL24-103

Abstract

This report was prepared as part of the end-of-study project to obtain the *National Diploma of Engineering in Computer science* from *the Higher Institute of Applied Sciences and Technology of Sousse*. This project was carried out during a five-months internship within the Tunisian startup *Envast*.

During this period, we aimed to develop an *Agile Deep Assistant* designed to optimize the sprint processes for *SCRUM teams* with a focus on enhancing sprint planning and retrospective meetings. This tool suggests developer assignments and offers recommendations for team improvements. It uses a *Retrieval-Augmented Generation (RAG)* setup, leveraging the capabilities of a large language model to enhance decision-making processes.

Keywords: SCRUM, agile, recommendation, Retrieval-Augmented Generation (RAG), JIRA Integration, Real-Time

Résumé

Ce rapport a été élaboré dans le cadre du projet de fin d'études pour l'obtention du *Diplôme National d'Ingénieur en Informatique* de l'*Institut Supérieur des Sciences Appliquées et de Technologie de Sousse*. Ce projet a été réalisé au cours d'un stage de cinq mois au sein de la startup tunisienne *Envast*.

Durant cette période, notre objectif était de développer *un Assistant Agile Avancé*, conçu pour optimiser les processus de sprint pour *les équipes SCRUM* en mettant l'accent sur l'amélioration de la planification des sprints et des réunions rétrospectives. Cet outil propose des affectations de développeurs et offre des recommandations pour l'amélioration de l'équipe. Il utilise la *RAG (Génération Augmentée par Récupération)*, exploitant les capacités d'un grand modèle de langage pour améliorer les processus de prise de décision.

Mots-clés : SCRUM, agile, recommandation, Génération Augmentée par Récupération (RAG), intégration JIRA, temps réel

Dedications

To my beloved mother and father,
whose love and guidance have always been my shining
light.

To my siblings,
for their endless support and camaraderie through every
step of my journey.

To all my friends,
for their unwavering friendship and encouragement, which
have been my strength and joy.

Yasmine Ben Abdeljelil

ACKNOWLEDGMENTS

I would like to take this moment to express my gratitude to everyone who contributed, both directly and indirectly, to the development of this project.

I extend my heartfelt thanks and appreciation to my academic supervisor, **Ms. Selma BELGACEM**, for her continuous guidance and support. Her expertise, patience, and encouragement were crucial to the success of this project. Her insightful critiques shaped the structure of the work and enhanced the quality of each section. I am deeply grateful for her valuable expertise.

Additionally, I owe deep gratitude to my company supervisor, **Mr. Oussema Fethi Guersene**, for his support, valuable advice, and for sharing his expertise and knowledge. I also extend special thanks to the entire **ENVAST family** for welcoming us not just as an intern, but as a potential colleague and part of the family.

Moreover, my deepest respect and gratitude are due to every professor at the Higher Institute of Applied Sciences and Technology of Sousse, who contributed to my academic development, enabling me to undertake this project. I also extend special thanks and respect to the jury members who honored me by examining and evaluating this work.

Contents

General Introduction	6
1 General Presentation of the Project	8
1.1 Presentation of the Host Organization	8
1.2 Presentation of the Project	8
1.2.1 Context and Concepts	8
1.2.2 Problematic	13
1.2.3 Proposed Solution	13
1.2.4 Objectives	13
1.3 Study of the existing	14
1.3.1 Existing solutions presentation	14
1.3.2 Existing solutions review	15
1.3.3 Critics	16
1.4 Adopted Methodology: SCRUM Application	16
1.4.1 SCRUM Team	17
1.4.2 Product Backlog	17
2 General Requirement and Architecture Specification and Development Environnement	20
2.1 Actors	20
2.2 Functional Requirements	20
2.2.1 Sprint Management	20
2.2.2 Sprint Planning Management	21
2.2.3 User Stories Assignment Management	21
2.2.4 Retrospectives Management	21
2.2.5 Developer Engagement in Estimation Processes	22
2.2.6 Developer Engagement in Retrospective Processes	22
2.2.7 Calendar Consultation	22
2.3 Non Functional Requirements	22
2.4 Use Case Diagram	24
2.5 Project Modules Overview	25
2.5.1 Domain-Driven Design (DDD)	26
2.5.2 Application of the DDD Approach	27
2.6 Module Architecture	28
2.6.1 Module Flow and Dependencies	29
2.7 Development environment	29
2.7.1 Hardware environment	29
2.7.2 Software environment	30
2.7.3 Deployment Diagram	32

3	Implementation of Sprint 1	34
3.1	Sprint backlog	34
3.2	Functional specifications	35
3.2.1	Sprint 1 Use Case Diagram	35
3.2.2	Refinement of first sprint use cases	35
3.3	System Design	38
3.3.1	Class Diagrams	38
3.3.2	Sequence diagrams	42
3.4	Implementation	43
4	Implementation of Sprint 2	49
4.1	Sprint backlog	49
4.2	Functional specifications	50
4.2.1	Sprint 2 Use Case Diagram	50
4.2.2	Refinement of second sprint use cases	51
4.3	System Design	55
4.3.1	Class Diagrams	55
4.3.2	Sequence diagrams	58
4.3.3	Activity diagrams	62
4.4	Implementation	62
5	Implementation of Sprint 3	72
5.1	Sprint backlog	72
5.2	Functional specifications	73
5.2.1	Sprint 3 Use Case Diagram	73
5.2.2	Refinement of third sprint use cases	74
5.3	System Design	79
5.3.1	Class Diagrams	79
5.3.2	Sequence diagrams	82
5.3.3	Activity diagrams	83
5.4	Implementation	83
	General Conclusion and Perspectives	93
	References	96
	Annexe	97
A.1	Sprint Management System	97
A.2	Sprint Planning Management System	98
A.3	Retrospective Management System	100

List of Figures

1.1	ENVAST logo	8
1.2	Big Project composition	9
1.3	Scrum Framework [25]	10
1.4	Retrieval Augmented Generation System Layers	12
2.1	General Use Case Diagram	24
2.2	Project Modules Overview	25
2.3	Generalized Module Architecture Package Diagram	28
2.4	Deployment diagram of the Agile Deep Assistant 2	32
3.1	Use Case Diagram for sprin1	35
3.2	Use Case Diagram for sprint management	35
3.3	Use Case Diagram for sprint backlog management	37
3.4	Sprint 1 Models package class diagram	38
3.5	Sprint 1 Controllers - Services interaction class diagram	40
3.6	Sprint 1 Facades-Services interaction class diagram	41
3.7	Repositories-Services interaction Class Diagram	41
3.8	Create Sprint Sequence Diagram	42
3.9	Start Sprint Sequence Diagram	43
3.10	Create New Sprint Interface	43
3.11	Initialize Sprint Backlog Interface	44
3.12	Sprint Details Interface	44
3.13	Edit Sprint Interface	45
3.14	Upload Jira Sprint's Statistic Interface	45
3.15	Upload Success confirmation	46
3.16	Sprint Details Insights	47
3.17	Sprints List : Cards View Interface	48
3.18	Sprints List : List View Interface	48
4.1	Sprint 2 general Use Case Diagram	50
4.2	Manage Sprint Planning Room Use Case Diagram	51
4.3	Manage Developers Assignment Use Case Diagram	52
4.4	Manage User Stories Estimation Use Case Diagram	54
4.5	Sprint2 Models Class Diagram	55
4.6	Sprint2 Controllers-Services-Repositories interaction Class Diagram	56
4.7	Sprint2 Events-Services-Notifcations Interaction Class Diagram	57
4.8	RAG System 2 class diagram	58
4.9	Uploading Jira Statistic File Sequence Diagram	59
4.10	Developer Assignment Sequence Diagram	60
4.11	Sprint Planning flow activity Diagram	62
4.12	Create New Sprint Planning Interface	63
4.13	Invite developers Interface	63

4.14	New Sprint Planning Created Interface	64
4.15	Estimation Room Interface	64
4.16	Room Projection Interface	65
4.17	User Story Details Consultation View	65
4.18	Presnece Developers View	66
4.19	Assign Developer Interface	66
4.20	Estimation Session Completion Confirmation Interface	67
4.21	Sprint Planning Estimation History Interface	67
4.22	Developer Authentification Interface	68
4.23	Developer Profile Interface	68
4.24	Sprint Planning Details Interface	69
4.25	Estimation Room Interface	70
4.26	User Story Details Interface	70
4.27	User Story Assignment complete Interface	71
4.28	Estimation Room Closed Interface	71
5.1	Use Case Diagram for sprin3	73
5.2	Use Case Diagram for Manage Retrospective Room	74
5.3	Use Case Diagram for Manage Ideas	75
5.4	Use Case Diagram for Manage Topics	76
5.5	Use Case Diagram for Manage Actions	77
5.6	Sprint 3 Models Packages Interaction Class Diagram	79
5.7	Sprint 3 :Controllers-Services-Repositories Interaction Class Diagram	80
5.8	Retrospective RAG system Class Diagram	81
5.9	Generate actions suggestions Sequence Diagram	82
5.10	Retrospective flow activity Diagram	83
5.11	Create New Retrospective Interface	84
5.12	Retrospective Room Step1 : Barinstorm Interface	84
5.13	Retrospective Room Step2 : Group Interface	85
5.14	Retrospective Room Step3 : Vote Interface	85
5.15	Retrospective Room Step4 : Action Interface	86
5.16	Retrospective Closure and Summary Interface	87
5.17	End Retrospective Meeting Confirmation Interface	87
5.18	Developer Todays Meeting Interface	88
5.19	Developer Calandar Interface	88
5.20	Retrospective Room Step1 : Brainstorm Developer Interfaces	89
5.21	Retrospective Room Step3 : Vote Developer Interface	90
5.22	Retrospective Summary Developer Interface	91
5.23	Notification Interface	92
5.24	Notifcation Consulation on Device	92
A.1	Create and Store Poker Planning Sequence Diagram	98
A.2	Developer User Story Estimation Sequence Diagram	99
A.3	Create and Store Retrospective Sequence Diagram	100
A.4	Create an idea Sequence Diagram	101
A.5	Vote on a Topic Sequence Diagram	102

List of Tables

1.1	Comparative description of Estimation and retrospective Solutions	15
1.2	Comparative description of Project Management Solutions	16
1.3	Product Backlog	17
3.1	Sprint Backlog 1	34
3.2	Use Case Specification for "Create a Sprint"	36
3.3	Use Case Specification for "Upload JIRA Statistic File in the Sprint"	36
3.4	Use Case Specification for "Add a New User Story in a Sprint"	37
4.1	Sprint Backlog 2	49
4.2	Use Case Specification for "Create New Sprint Planning"	52
4.3	Use Case Specification for "Assign a Developer to a User Story"	53
4.4	Use Case Specification for "Developer Story Estimation Submission"	55
5.1	Sprint Backlog 3	73
5.2	Use Case Specification for "Create New Retrospective room"	75
5.3	Use Case Specification for "Create Idea"	76
5.4	Use Case Specification for "Vote on Topic"	77
5.5	Use Case Specification for "Generate actions suggestions in Retrospective"	78
A.1	Use Case Specification for "Start a Sprint"	97

General Introduction

In the rapidly evolving world of artificial intelligence, the adoption of large language models (LLMs) such as GPT-3 and its successors has become crucial. These models excel at processing and interpreting large amounts of unstructured data in ways traditional methods cannot. They have shown remarkable abilities in generating human-like text, understanding context, and delivering insights that go beyond mere keyword matching. This evolution signifies a shift from conventional data processing to more sophisticated, context-aware algorithms that adapt to the specific nuances of different datasets.

For businesses, this means the opportunity to employ a more tailored AI, enhancing customer interactions with personalized responses and improving decision-making with deeper, more relevant insights.

Simultaneously, the roles of those leading and working within development teams have become more crucial, particularly for those adopting Agile methodologies. Agile, especially the Scrum framework, focuses on iterative development through sprints. This process involves key practices like sprint planning and retrospective meetings (See Section SCRUM Events). These practices ensure that Agile methodologies effectively manage the complexities of modern software development, aligning project goals with stakeholder expectations and maintaining team flexibility and productivity.

This project serves as the culmination of our training in software engineering at the Higher Institute of Applied Sciences and Technology of Sousse, marking a significant milestone in our end-of-studies project. During our internship, our goal is to design and implement an Agile Deep Assistant, a tool aimed at automating sprint task management and team evaluation.

This innovative tool is designed to leverage the capabilities of LLMs within the Agile framework to automate sprint task management and team evaluation. By doing so, it aims to enhance the efficiency of sprint planning and retrospective meetings, ensuring more effective communication and precise project execution. Thus, our project not only supports key areas of Agile development but also incorporates advanced AI to provide teams with a more dynamic and insightful toolset.

This report will present the different stages of carrying out this project and will be spread over five chapters:

- The first chapter entitled “General presentation of the project” is an introductory chapter in which we provide a brief description of the hosting company, the treated issue, the proposed solution, the project objectives, a study of the existing solutions, and the adopted software development life cycle.
- The second chapter entitled “General Requirement and Architecture Specification and Development Environnement”, where we identify functional and non-functional needs. Then, we deduce the global use case diagram and finally describe the general architecture as well as the environment hardware and software.
- The following 3 chapters are dedicated to the three sprints of our application, each being divided into four modules: the sprint backlog, analysis, design and implementation.

Finally we close this report with a general conclusion in which we summarize our work and we propose the possible perspectives.

Chapter 1

General Presentation of the Project

Introduction

In this chapter, we introduce the hosting company ENVAST, the existing issue and the proposed solution . Additionally, we describe the adopted software development methodology.

1.1 Presentation of the Host Organization



Figure 1.1: ENVAST logo
[6]

ENVAST, a Tunisian company established in 2016, is a hub for seasoned talents specialized in crafting innovative digital products. It consists of different developing units :

- Video games
- Mobile applications
- Web development
- Virtual reality
- Augmented reality

1.2 Presentation of the Project

1.2.1 Context and Concepts

Big project

This project is part of a larger initiative aimed at refining various aspects of Agile project management, integrating advanced algorithms (RAG setup) to enhance decision-making and efficiency.

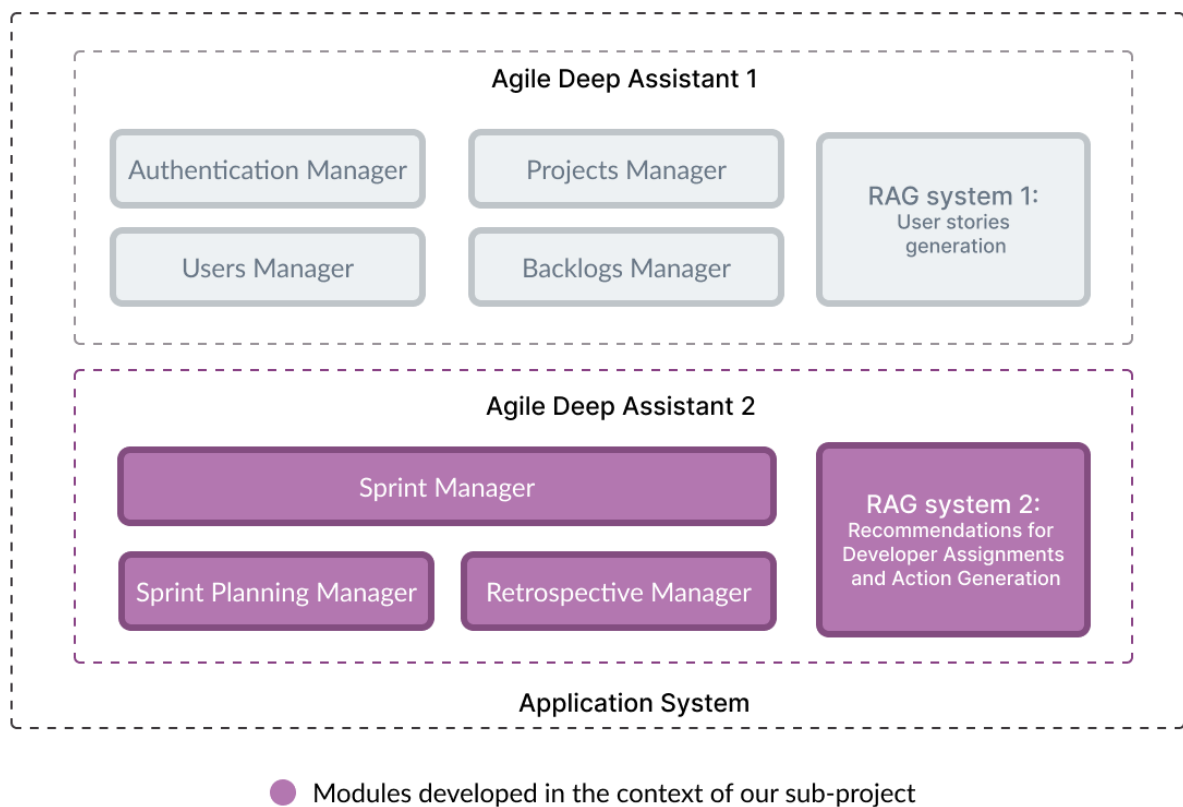


Figure 1.2: Big Project composition

The figure 1.2 illustrates the structure of a comprehensive Agile project management system, which includes various management components and a RAG system.

Components like the Users Manager, Authentication Manager, Backlogs Manager, Projects Manager, and a RAG system 1 for user stories generation are developed by other team members. These elements handle user credentials and roles, project backlogs, and generate user stories based on historical data.

For the subproject we are developing, the focus is on the purple highlighted components: Sprint Manager, Sprint Planning Manager, Retrospective Manager, and RAG system 2. These modules are specifically designed to streamline sprint management for Scrum teams, enhance sprint planning and retrospective meetings by offering recommendations for developer assignments, facilitating the regroupment of ideas, and generating actionable insights for team evaluation.

By integrating these functionalities, the project intends to provide a more dynamic and effective toolkit for Agile teams, helping them make better decisions and improve overall team performance.

SCRUM definition

Our solution is tailored for Agile teams that utilize Scrum, a project management methodology implemented throughout our development process. This Agile framework as illustrated in figure 1.3, promotes flexibility, teamwork, and iterative progress through periods known as sprints toward a well-defined goal. Scrum helps teams quickly adapt to changing project needs, ensuring continuous enhancement and efficiency.

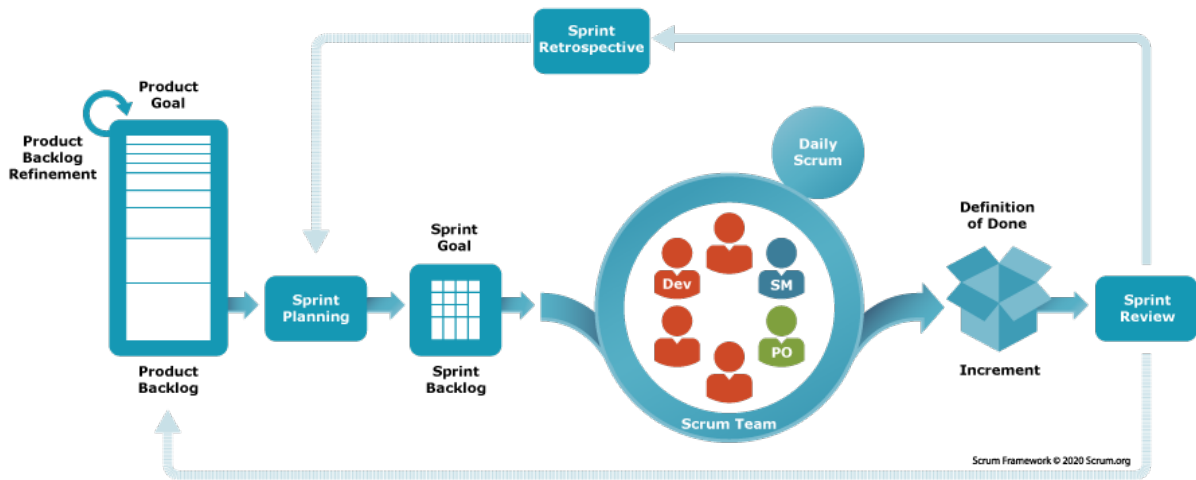


Figure 1.3: Scrum Framework [25]

The key components of this framework include:

1. **Roles:** Scrum defines three fundamental roles [31] to ensure a clear distribution of responsibilities:

- **Product Owner:** Acts as the client's representative, defining the requirements and priorities of the project. The Product Owner is responsible for validating the delivered increments through acceptance testing
- **Scrum Master** A specialist in the Scrum methodology who facilitates the process, helps resolve obstacles and ensures that Scrum practices are followed.
- **Development Team:** A Cross-functional members consisting of 5 to 9 members who do the actual work (analyze, design, develop, test,...). This team is self-organized, self-managed, engages in collective work, and remains stable throughout the work iteration (Sprint), with no changes in team members during this period.

2. **Artifacts:**

- **Product Backlog** An ordered list of user-stories expressed by the product owner.
- **Sprint Backlog** A list of tasks identified by the scrum team to be completed during the current sprint.
Per the standard SCRUM definition, these tasks are technical operations designed to fulfill expressed user-stories. Nevertheless, in our project's context, user stories themselves are treated as tasks to be completed and directly included in the sprint backlog. In other words, the sprint backlog is a segment of the product backlog.
- **Increment:** The sum of all the product backlog items completed during a sprint, resulting in a potentially shippable product state at the end of a sprint.

3. **Events:**

- **Sprint:** A fixed timeframe that is repeatedly used to deliver selected features from the backlog. The objective is to produce valuable product increments from each sprint.

- **Sprint Planning Meeting** A meeting at the beginning of the sprint where the team elaborates the sprint backlog by setting to do tasks according to expressed user-stories. During this meeting, **estimation** is performed to evaluate the amount of effort required for each user story in the sprint backlog. A common method used for this estimation is **poker planning**, a technique that involves team members making estimates by playing numbered cards face down, then revealing them simultaneously to foster discussion and arrive at a collective understanding of the user story complexity.
- **Daily Scrum Meeting** A short (typically 15 minutes) meeting each day to discuss progress, plan for the next day, and identify any obstacles.
- **Sprint Review Meeting** Held at the end of each sprint to present the work done to stakeholders and receive feedback.
- **Sprint Retrospective Meeting** A meeting where the team auto-evaluate each member productivity and professional behavior and identifies strategies to improve self-skills and the global team productivity.

Generative methods : RAG system

Our project uses a Retrieval-Augmented Generation (RAG) setup and advanced language models to improve decision-making and automate essential sprint management tasks. This includes generating personalized responses and precise user stories estimations, which are vital for effective sprint planning and thorough retrospective evaluations. The goal is to not only replicate human-like interactions but also to offer deeper contextual insights and predictive capabilities that go beyond basic keyword searches. Typically, models are restricted to the data they are trained on, which may not include a company's custom development data. Such capability is crucial in scenarios where the model needs to be aware of the project context before generating a response, ensuring relevance and accuracy.

This section will delve into fundamental concepts that underpin the technology of our project, particularly focusing on the Retrieval Augmented Generation (RAG) system.

1. **Retrieval Augmented Generation (RAG)** [1] Retrieval Augmented Generation is a method that combines the powers of large pre-trained language models with external retrieval or search mechanisms. This approach allows generative models to access vast amounts of data and custom documents outside the base model, enabling model to generate more real, more individual, and more reliable outputs.

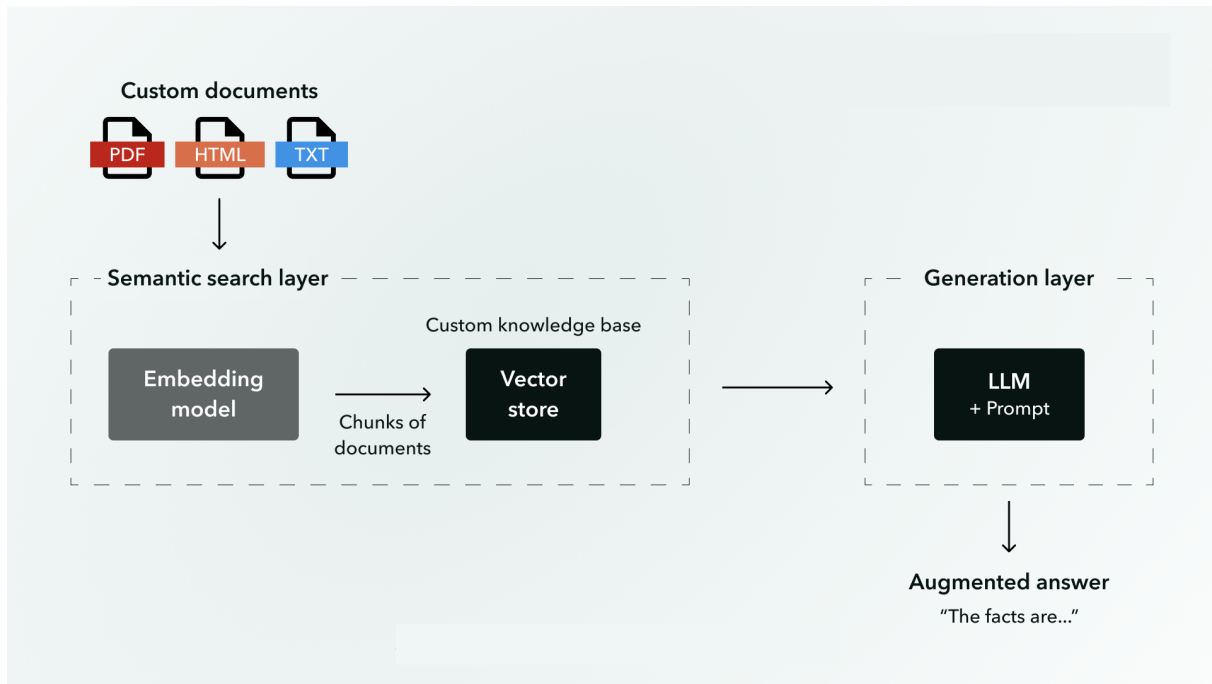


Figure 1.4: Retrieval Augmented Generation System Layers
[22]

- **Large Language Models (LLM):** A type of machine learning model designed to understand and generate human-like text based on the massive training data it has processed.
- **Prompt:** A text input given to an AI model to guide him generate a specific response or output based on the input context.
- **Vector store:** A type of database designed to efficiently handle and retrieve high-dimensional feature vectors, using specialized indexing structures optimized for quick access to these data representations.
- **Feature Vector:** It's a way to represent data as a list of numbers, where each number reflects a specific characteristic important to analyze that data.
- **Embedding [24]:** Also known as a characterization system its's a method that transforms inputs such as words or phrases into vectors of numerical values (features), allowing complex data to be represented in a simpler, uniform format.
- **Embedding Model [5]:** Also known as a characterization model that is responsible for encoding text into numerical vectors. For example, the sentence "Hi, how are you?" could be represented as an embedding vector [0.12, 0.2, 2.85, 1.33, 0.01, ..., -0.42] with N dimensions, where each dimension captures a specific feature or aspect of the input. By converting text into embedding vectors, generative models can process data more efficiently, revealing patterns that are not easily detectable in the original text format.

The figure 1.4 illustrates the architecture of a RAG system layers.

- Semantic Search Layer:** Utilizes embedding vectors to semantically search through custom documents to find the most relevant content related to the input prompt.
- Generation Layer:** A generative model then will takes the retrieved data and the initial prompt to generate coherent and contextually appropriate responses.

2. Benefits of Using RAG

Using a RAG system offers significant advantages, including:

- **Preventing AI hallucinations:** The more accurate and relevant in-context information LLMs have, the less likely they'll invent facts or respond out of context.

- **Providing LLMs with up-to-date information:** Sometimes LLM training data can be incomplete or outdated over time. RAG allows adding new and/or updated documents without retraining the LLM from scratch and maintain an up to date knowledge base
- **Flexibility:** RAG model can be adapted for specific domains by changing or expanding the external knowledge source without retraining the underlying generative model.

1.2.2 Problematic

The effectiveness of project planning within the Agile framework, encompassing both sprint planning meetings at the beginning of each sprint and sprint retrospective meetings at its conclusion, is influenced by multiple challenges, which touch on several key issues faced by many software development teams today. These include:

- Traditional task estimation methods can be inaccurate, leading to either too much or too little work being planned . This misestimation directly impacts project timelines
- Incorrectly assigning tasks to team members who may not have the optimal skill set or bandwidth which can lead to production delays and reduced quality of work.
- Teams often handle different parts of the project management with several tools. In addition to the main project management tool, they use Excel sheets within the company to see sprint insights, entering this information by hand. This fragmentation can lead to inefficiencies and miscommunication.
- The benefits of retrospectives in identifying areas for improvement and adapting processes are often overlooked, which can hinder team development, missing chances to get better at what they do.

These problems make it hard to stick to the agile goal of fast and flexible development, showing a clear need for a better, all-in-one solution.

1.2.3 Proposed Solution

Addressing these complex challenges necessitates the adoption of an integrated, AI-based project management tool that brings everything together. This tool would use data from past projects to make better guesses about how long tasks will take and suggest the best team members for each task, considering their skills and current workload. It would also automatically gather data and feedback from tools like Jira throughout the project and sprint's lifespan, to continuously and automatically improve the time-to-market and used ressources exploitation. This approach not only makes project management more straightforward and efficient but also helps teams stay true to the agile principles of quick adaptation and ongoing improvement, leading to better project results and happier teams.

1.2.4 Objectives

After identifying key challenges, we set the following goals to improve our project management:

- **Enhance Developer Assignment:** Provide an AI assistant to automatically suggest suitable developers from previous projects during sprint planning meetings, based on their experience with similar user stories.
- **Improve Sprint Reporting:** Enhance the presentation of closed sprint data to clearly showcase team performance and task progression, highlighting successes and areas needing improvement.

- **Automate Team Evaluation:** Employ a pretrained language model during sprint retrospectives to identify performance issues and recommend specific improvements.

1.3 Study of the existing

1.3.1 Existing solutions presentation

In this section, we discuss the currently existing Scrum project management tools. Through the study of the existing solutions, we extract their key differences, advantages, and drawbacks which will be compensated and considered in our project objectives.

Project Management Tools

- **Jira:** A widely used tool for project management that offers robust features for tracking issues and organizing tasks. Jira supports agile projects with its customizable Scrum and Kanban boards, and it integrates seamlessly with a variety of plugins and external tools, making it a versatile choice for agile teams.
- **Monday** A highly visual project management tool that helps teams track projects and workflows with customizable boards and integrations.
- **ClickUp** An all-in-one project management platform that provides solutions for task management, document sharing, and collaboration across all types of industries.

Estimation and Retrospective Tools

- **PlanITPoker:** A web-based tool designed for agile teams to facilitate the estimation of project tasks.
- **Scrumpy Planning Poker:** An estimation tool that integrates with project management software, aiming at enhancing agile planning and estimation.
- **Parabol:** An open-source software focused on streamlining agile retrospectives and sprint planning sessions.
- **GoRetro:** A retrospective tool for assisting agile teams in reflecting on and improving their workflow and processes.

1.3.2 Existing solutions review

Solution	Pros	Cons
PlanITPoker	• Customizable cards • Easy team collaboration • Free to use	• No backlog integrations • Distracting ads
Scrumpy Planning Poker	• Multiple platforms • Async voting • Integrations with Jira, Slack, GitHub, etc. • Colorful cards	• Complicated UI • Hard to set up integrations • Not Open Source
Parabol	• Open source • Built-in communication tools • team evaluation mechanism (Anonymous features, Ice-breaker questions) • Rich library of templates • AI summaries	• No central repository for summaries
GoRetro	• Action item tracking • Data-driven insights • Capacity calculators	• Browser compatibility issues • Some functions hard to find • Can become tedious with many items

Table 1.1: Comparative description of Estimation and retrospective Solutions

Solution	Pros	Cons
Jira	• Comprehensive project management • Extensive integration capabilities • Strong support for agile methodologies	• Complex for new users • Requires significant setup and maintenance
Monday	• Visual project planning • integration with other tools • High customizability	• Can be expensive for larger teams • cluttered interface
ClickUp	• All-in-one workspace • Highly versatile and customizable	• Challenging to learn • Notifications can be overwhelming

Table 1.2: Comparative description of Project Management Solutions

1.3.3 Critics

Tools like Monday, and ClickUp reveals that while they offer robust features, they can be difficult to learn. While Jira is highly configurable and integrates well with other tools, but without proper configuration it can become complex rather than helpful. Tools like PlanITPoker and GoRetro facilitate task estimation and retrospectives but rely heavily on manual input.

Our proposed AI-based solution addresses these gaps by integrating all aspects of the project lifecycle and its sprints into a single platform, using AI to automate and refine processes. It uses historical data to improve user story estimation, automates developer assignments based on experience and workload, and enhances sprint reporting and team evaluation, making project management more efficient and aligned with agile principles.

Our project management solution is not designed to replace tools like Jira but rather to enhance and complement them. It will integrate seamlessly with Jira, leveraging its powerful sprint Kanban for tracking developer performance and task updates. This integration enables our system to incorporate established features from external tools while also providing functionalities that are missing or challenging to configure in our solution, enhancing its overall capability and user experience.

1.4 Adopted Methodology: SCRUM Application

For our project's software development life cycle (SDLC), we adopted the Agile methodology SCRUM, aligning with the practices already in place at the company for which we are developing this solution. This choice ensures consistency between our development approach and the company's existing framework, facilitating seamless integration and communication.

1.4.1 SCRUM Team

In this subsection, we outline the application of Scrum roles within the company as assigned to this project.

- **Product Owner:** *Meryam Boughzala* - Responsible for maximizing the value of the product resulting from the work of the Scrum Team. Maryem defines the project goals, manages the Product Backlog, and ensures the team always have a clear vision.
- **Scrum Master:** *Omar Trabelsi* - Serves as both facilitator and coach for the Scrum Team, ensuring that the project consistently follows Scrum practices.
- **The Scrum Team:**
 - **Scrum Team Leader:** *Bilel Ben Salem* - Leads the team in setting the next sprint goals and establishing the sprint backlog. During development, he helps the team remove any obstacles that may obstruct progress, ensuring smooth project execution.
 - **Tech Lead Backend:** *Oussema Fethi Guersene* - Ensures architectural coherence of backend modules. Leads code review sessions to maintain code quality.
 - **Tech Lead Frontend:** *Louay Atoui* - Leads the frontend development team, focusing on user interface design and user experience optimization.
 - **Developer:** *Yasmine Ben Abdeljelil* - Focuses on the automation of Sprint management and team evaluation processes.
 - **Developer:** *Mohamed Ali Meddeb* - Workes on the automatic generation of the Product Backlog and the crafting of user stories.

1.4.2 Product Backlog

The product backlog outlines the prioritized user-stories expressed by the product owner. Table 1.3 presents the product backlog for our project.

Table 1.3: Product Backlog

ID	As a scrum team leader, I can (User story)	Priority	Title
1	Add a sprint	High	Sprint Management
2	Start a sprint	High	
3	Stop a sprint	Medium	
4	Close a sprint	High	
5	Modify a sprint	Medium	
6	Delete a sprint	Low	
7	View insights of a sprint	High	
8	Consult the list of sprints	Medium	
9	Filter my sprints searches	Medium	
10	Add a user story to sprint backlog	High	Sprint Backlog Management
11	Switch a user story from one sprint backlog to another	Medium	
12	Update a user story from a sprint backlog	Medium	
13	Delete a user story from a sprint backlog	Low	
14	Update user stories in a sprint backlog	Medium	
15	Filter user stories within a sprint backlog	Medium	
16	Create a room for estimation	High	Sprint Planning Management
17	Add user story from available backlogs	Medium	

18	Consult the details of a user story	Low	
19	Invite developer to complete the estimate	High	
20	Start the estimate	High	
21	Moderate the meeting (skip, clear votes, next stories)	High	
22	Consult a developer's estimate	Medium	
23	Consult the history of estimation room	Low	
24	Assign a developer for a user story	High	Developers Assignment Management
25	Review the list of assignment requests from developers	Medium	
26	Generate Recommendations for Suitable Developer	High	
27	Assign a developer for a user story	High	
28	Modify the assignment of a developer for a user story	Medium	
29	Remove the assignment of a developer for a user story	Low	
ID	As a Scrum Master, I can (User story)	Priority	Title
30	Create a retrospective session	High	Retrospective Management
31	Send invitations to all scrum team members for the retrospective	Medium	
32	Input my ideas during the first step of the retrospective	High	
33	Create a topic by organize related ideas into groups using a drag-and-drop during the second step of the retrospective	Medium	
34	Generate grouped topics suggestions	High	
35	Approve or Decline all suggested grouped topics collectively	High	
36	Approve or Decline grouped topic suggestion individually	Medium	
37	Vote to determine the priority of discussion topics during the third step of the retrospective	High	
38	Add actionable items during the fourth step of the retrospective	High	
39	Link specific actions to related discussion topics	Low	
40	Generate actions suggestions for discussed topics	High	
41	Approve or Decline all suggested actions collectively	High	
42	Approve or Decline each suggested action individually	Medium	
43	Review the retrospective report	Medium	
44	Export the retrospective outcomes for record keeping	Low	
ID	As a developer, I can (User story)	Priority	Title
45	Consult the list of user stories planned for an estimate	Medium	User Story Estimation Management
46	Join a sprint planning meet	High	
47	Consult the details of a story	Medium	
48	Estimate a user story	High	
49	Modify the estimate of a user story	Medium	
50	Vote to take charge of a user story	High	Developer Ideas Management in Retrospectives
51	Participate in a retrospective meeting	Medium	
52	Contribute my ideas at the start of the retrospective	High	
53	Group ideas into topics during the second step of the retrospective meeting	Medium	
54	Vote to prioritize topics during the third step of the retrospective meeting	High	
55	Access a summary of the retrospective outcomes	Low	

56	Receive a notification to complete estimates	High	Developer
57	Receive notification of my assignment on a user story	Medium	Notification
58	Receive an invitations notification to participate in retrospectives	High	Consultation
59	Access the calendar to check schedules for upcoming retrospectives and sprint planning meetings	High	Calendar Consultation

Conclusion

This chapter presented the project in its global context. We started by introducing the host company Integration Objects, then we provided a detailed description of the subject, the issue, and the objectives of this work. Finally, we illustrated our software development life cycle. In the next chapter, we detail the user requirements and our platform functionalities

Chapter 2

General Requirement and Architecture Specification and Development Environnement

Introduction

In this section, we will define the interacting actors with our application and we detail the functional and non-functional requirements. Then we specify those requirements through a general use case diagrams. Refined use case diagrams will be presented through sprints chapters. We will also outline the architecture we adopted for developing our project, as well as the hardware, software, and technologies utilized to establish our development environment.

2.1 Actors

Our application interacts with three primary actors, each having distinct responsibilities within the the overall system :

- **Scrum Master:** Facilitates retrospective meetings, ensuring that they are productive and focuses on continuous improvement.
- **Scrum Team Leader:** A unique role in our system to ensure clear accountability on the platform. While part of the Scrum team, this role assumes additional responsibilities, moderating both the sprint and sprint planning sessions, reflecting the scrum team's self-managed nature.
- **Developer:** A core member of the scrum team, engages in both sprint planning and retrospectives.

2.2 Functional Requirements

The process of identifying functional requirements involves translating the project objectives into a set of features. According to defined actors, the functional requirements of our project are :

2.2.1 Sprint Management

This module falls under the responsibility of the Scrum Team Leader, who is tasked with:

- **Creating and Managing Sprints:** Initiating and managing new sprints.

- **Start, Close:** Starting a sprint to commence work, closing a completed sprint to finalize it.
- **Consult Sprints List:** Reviewing a comprehensive list of all sprints within a project, applying filters based on various criteria.
- **Detailed Analytics:** Accessing and consulting detailed analytics for a sprint, visualizing the progress status of user stories and evaluating overall team performance.
- **Sprint Backlog Management:** Managing the sprint backlog by adding, updating, and deleting user stories.

2.2.2 Sprint Planning Management

Managed by the Scrum Team Leader, this module facilitates:

- **Sprint Planning Room Management:** Creating, updating, and deleting rooms for estimating user stories.
- **Team Engagement:** Inviting team members to participate in the sprint planning sessions.
- **Estimation Execution:** Start and manage the session process, allowing for real-time submission of estimates
- **User Story Management:** Reordering, skipping, or clearing votes for user stories as needed during estimation sessions.
- **Real-Time Monitoring and History:** Viewing estimates in real-time and reviewing the history of estimations.

2.2.3 User Stories Assignment Management

Also under the Scrum Team Leader's purview, this section details the processes for managing the assignment of user stories to developers within sprint planning meeting :

- **View Requests:** Monitor incoming requests from developers seeking assignments for specific user story.
- **Assign Developers:** Assign developers to user stories.
- **Generate Developer Recommendations:** Utilize AI-driven methods to suggest the most appropriate developers for a specific user story, based on their previous work on similar stories.

2.2.4 Retrospectives Management

This section outlines the systematic approach the Scrum Master uses to organize and conduct retrospective sessions effectively:

- **Room Management:** Manages the lifecycle of a retrospective session, which includes creating, updating, and deleting sessions.
- **Team Engagement:** Invites all scrum team members to participate in the retrospective.
- **Idea Contribution and Organization:** Add ideas and observations, followed by organizing these ideas to create topics.
- **Voting and Action Taking:** Conducts voting to prioritize topics, and records actions to address these priorities.

- **AI-Suggested Idea Regroupment:** Utilizes AI suggestions to automatically regroup related ideas.
- **AI-Suggested Action Items:** Employs AI-driven recommendations to suggest actionable steps for resolving discussed topics.
- **Consult Summary:** Reviews the outcomes of the retrospective and provides the capability to export the data for record-keeping purposes.

2.2.5 Developer Engagement in Estimation Processes

This section highlights activities a developer is involved in regarding sprint planning and user story management:

- **Meeting Notifications and Participation:** Developers receive notifications to participate in sprint planning meetings.
- **Sprint Planning Review:** Developers consult the list of user stories planned for estimation, sprint goals and meeting participants.
- **Estimation Contributions:** Developers add, update, or remove estimations for user stories.
- **User Story Assignment Notifications:** Developers receive a notification on their assignments to specific user stories.
- **User Story Ownership Voting:** Developers vote to take charge of a user story.

2.2.6 Developer Engagement in Retrospective Processes

This section describes the developer's involvement in retrospective sessions:

- **Retrospective Meeting Participation Notification:** Developers receive invitations to join retrospective meetings.
- **Idea Contribution:** Developers add their ideas and observations.
- **Voting on Key Topics:** Developers vote to prioritize topics or issues for detailed discussion during the retrospective.
- **Retrospective Outcomes Review:** Developers receive summaries that outline key outcomes and action items from the retrospective session.

2.2.7 Calendar Consultation

Developers consult their personal calendars to keep track of upcoming sprint planning and retrospective meetings.

2.3 Non Functional Requirements

- **Satisfaction:** Our application must ensure an engaging and enjoyable experience for users through its functionality, performance, and ease of use.
- **Interoperability:** Our application must offer the ability to interact with other software, such as JIRA, to enhance integration capabilities.
- **Real-time:** The application must ensure responds and updates in real-time, providing immediate feedback and interaction for users.

- **Reusability:** As this project is part of a larger application, with other modules being developed by different team members, our application will be designed to integrate seamlessly across various parts of the same system. This ensures consistency in user interfaces through reusable components and maintains a uniform architecture across modules. To achieve this, we developed a script that generates a unique module folder structure, adhering to a consistent architecture.
- **Modifiability:** Our application must be modular, well-documented, and must adhere to coding standards, making it easy to be modified and maintained in the future.
- **Security:** Our application implements security features using JSON Web Tokens (JWT) for user authentication and authorization, ensuring that only authorized users can access certain parts of the application.

2.4 Use Case Diagram

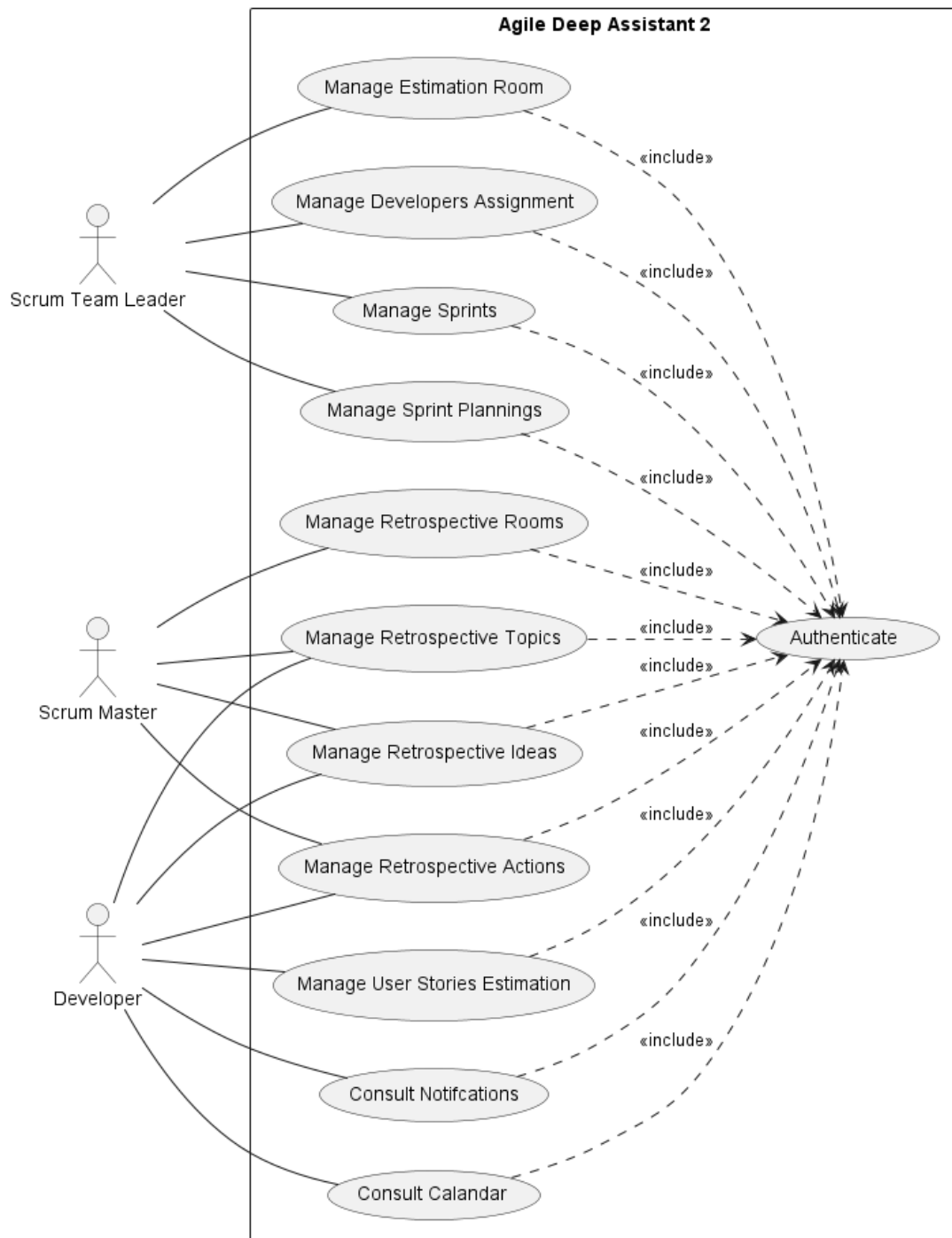


Figure 2.1: General Use Case Diagram

2.5 Project Modules Overview

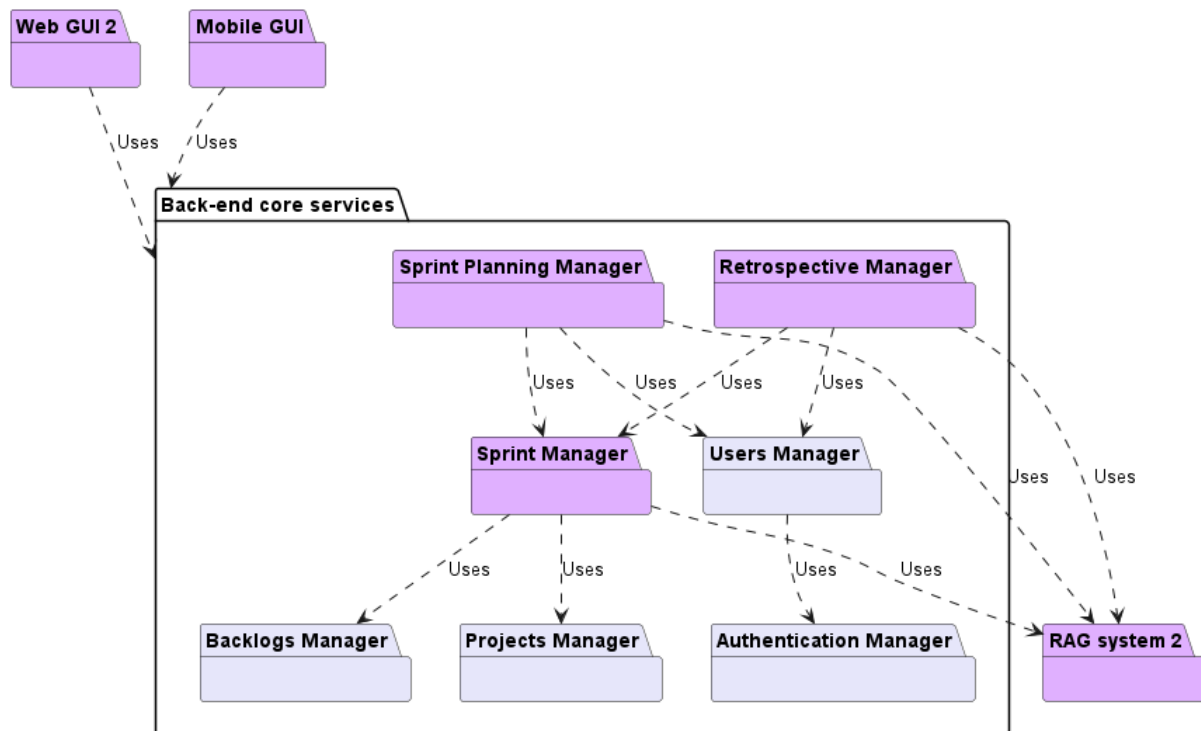


Figure 2.2: Project Modules Overview

This provided package diagram represents the architectural design for our Agile Assistant application. It is structured to showcase how different components of the system are organized and interact across three main areas:

1. Back-end core services

- ***Sprint Planning Manager***: Manages all sprint planning-related functionalities.
- ***Retrospective Manager***: Handles functionalities associated with sprint retrospectives.
- ***Sprint Manager***: Central to managing ongoing sprints and their statuses.
- ***Authentication Manager***: Responsible for user authentication and security.
- ***Projects Manager***: Manages project-related information and operations.
- ***Users Manager***: Oversees user profiles and permissions.
- ***Backlogs Manager***: Manages product backlogs including their user stories.

Our specific mission was to develop the modules for sprint planning, sprint management, and retrospectives, while other modules were handled by different team members

2. User Interface (UI) Packages

Those front-end packages are developed in the context of our sub-project.

- ***Web GUI 2***: Represents the interfaces for both the scrum team leader and the scrum master. It includes distinct packages for each corresponding module in the Back-end core services package.

- **Mobile GUI:** This package is designed for the developer interface and includes specific mobile packages for sprint planning and retrospective activities.
3. **RAG Package** The RAG system 2 will combines the capabilities of information retrieved from a knowledge sources with the advanced generative capabilities of language models to offer detailed and contextually accurate suggestion. These suggestions are focused on enhancing various aspects of our project, including:
- **Developer Assignment Suggestion:** This feature offers AI-generated suggestions for assigning developers to user stories in sprint planning sessions.
 - **Topics Regroupement Suggestion:** It provides AI-driven recommendations for grouping related discussion topics during retrospective meetings.
 - **Actions Suggestion:** This component delivers AI suggestions for actionable items to be addressed in retrospective meetings.

These packages are also developed in the context of our sub-project and will be described in detail in the section 4.8 and 5.8.

2.5.1 Domain-Driven Design (DDD)

Domain-Driven Design (DDD) [4] is an approach to software development that focuses on understanding and modeling the problem domain within which a software system operates. It emphasizes the importance of collaborating closely with domain experts to develop a deep understanding of the domain's complexities.

Key Concepts of DDD

1. **Ubiquitous Language:**
DDD emphasizes the use of a common language that is understood by both developers and domain experts. This ubiquitous language is used consistently throughout the project to minimize misunderstandings and ensure that all stakeholders have a common understanding of the domain.
2. **Bounded Contexts:**
This principle involves dividing the domain into multiple bounded contexts, where each context represents a specific responsibility and evolve independently. Each bounded context has its own model and does not need to be concerned with the details of other contexts.
3. **Entities and Value Objects:**
Entities are distinct and identifiable objects with a unique identity that runs through time and different states of the application. Value objects, on the other hand, are objects that do not have an identity but only defined by their attributes. Value objects are immutable, meaning their state cannot be changed once they're created.
4. **Domain Events:**
Domain events are significant business events that trigger transactions or changes within the domain. Recognizing and handling these events explicitly can help in designing more reactive systems.
5. **Repositories:**
Repositories are used to encapsulate the logic required to access data sources for aggregates, enhancing the abstraction layer between the domain model and data management layers.
6. **Services:**
In DDD, services represent actions that are important within the domain and that involves multiple objects and not a specific one.

2.5.2 Application of the DDD Approach

We applied the DDD approach throughout our system to ensure that our software architecture closely aligns with the complex needs of the business domain, faithfully embodying the foundational principles of DDD.

Our system contains modules such as:

- Sprint Planning Manager - Retrospective Manager - Sprint Manager - RAG system

- Each *module* represent a ***bounded context***, encapsulating specific functionalities and responsibilities that correspond to different aspects of our business processes.
- Each module utilizes ***ubiquitous language***, ensuring that terms like '*sprint*', '*backlog*', and 'Sprint Planning' are used consistently to ensure communication and understanding among both developers and domain experts.
- ***Repositories*** within each manager abstract data handling, separating the infrastructure layer from domain logic. This aligns with DDD's emphasis on decoupling system architecture components to promote flexibility and maintainability.
- ***Services*** within these managers encapsulate complex business rules, further separating domain logic from application logic and ensuring that the system functions smoothly.
- ***Domain events*** are leveraged throughout the system to handle significant business events, reflecting the system's reactivity to changes within the domain.

By applying DDD, we are preparing our system for a future migration to a microservices architecture. This approach organizes our system into distinct modules and bounded contexts, such as Sprint Planning Manager and Retrospective Manager, facilitating their eventual transition into independent, deployable microservices.

This strategic setup enhances modularity and scalability while simplifying future deployments.

2.6 Module Architecture

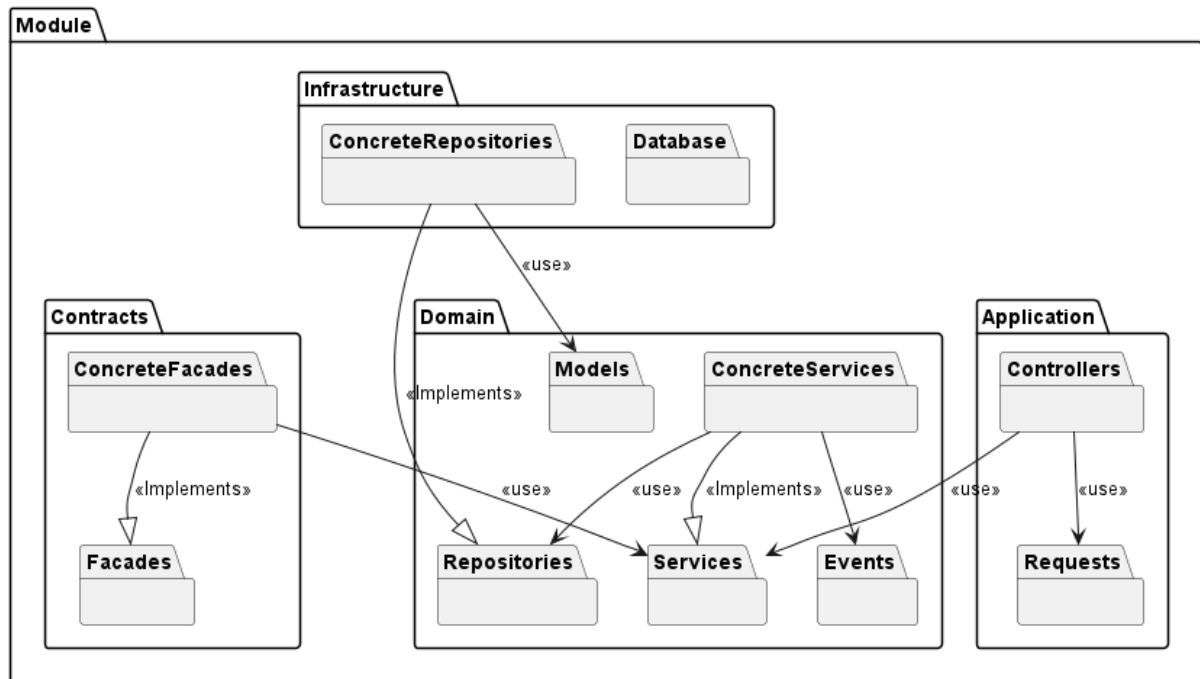


Figure 2.3: Generalized Module Architecture Package Diagram

The Figure 2.3 showcases the organized structure of a single Module within our modular architecture system. This modular architecture is applied for each package developed in the context of this project.

1. **Infrastructure** :Handles data storage and lower-level data access operations.
 - **Database**: Manages data storage and access.
 - **ConcreteRepositories** : Provides specific implementations for data access.
2. **Application** : Manages the input and output that interface with users or external systems
 - **Controllers**: Handle incoming HTTP requests and delegate tasks to appropriate services.
 - **Requests**: Represents incoming data from users through both the web and mobile applications and ensures that requests are correctly formatted and validated before they reach the business logic.
3. **Domain**: Core of the business logic.
 - **Services**: Abstract definitions of the business logic services.
 - **ConcreteServices**: Specific implementations of the services defined in "Services."
 - **Models**: Represents domain entities and their behavior in the business domain.
 - **Events**: Encapsulates events that signify important occurrences in the system and trigger related real time updates elsewhere in the system.
 - **Repositories**: Interfaces for data access, used by services to retrieve and store data.

4. **Contracts:** Defines interfaces and their concrete implementations exposed by the module for other modules.
 - **Facades:** Abstract interfaces that simplify interactions with more complex subsystems
 - **ConcreteFacades:** Implementations of the facades, providing specific functionalities.

2.6.1 Module Flow and Dependencies

- **Controllers** use **Requests** to handle incoming data and utilize **Services** to process this data.
- **ConcreteServices** implement the abstract **Services** and interact directly with **Repositories** to manage data. They also dispatch **Events** as part of the business logic execution.
- **ConcreteRepositories** implement data access mechanisms defined by **Repositories** and interact with **Models** to fetch or store business data.
- **ConcreteFacades** implement **Facades** and utilize **Services** to offer a simplified interface, facilitating easier interaction with other modules.

We designed this architecture with several principles to enhance decoupling of system components:

1. **Separation of concerns** [27] : This principle is applied by clearly defining functional areas within specific packages (such as Infrastructure , Application and Domain), allowing each to focus on distinct responsibilities.
2. **Dependency Inversion Principle** [28] : We ensure that high-level modules, like the Controllers, rely on abstractions (Service Interfaces) rather than concrete implementations.

2.7 Development environment

The hardware environment and the software environment used to attend the completion of this project are described in this section.

2.7.1 Hardware environment

For the development of this application, we used the following equipment:

Personal Computer

- Brand: HP
- Operating System: Windows 10
- Processor: Intel(R) Core(TM) i7-8565U CPU @ 1.80GHz 1.99 GHz
- GPU: NVIDIA GeForce MX130
- RAM: 12 GB
- Storage: 1 TB HDD, 480 GB SSD

Physical Smartphone

- Brand: Oppo
- Operating System: Android 13
- Processor: Qualcomm® Snapdragon™ 662 Octa-core
- RAM: 6 GB
- Storage: 128 GB

2.7.2 Software environment

In this section we define the list of the software environment used throughout the project.

- **Microsoft Visual Studio Code** [23]
A powerful code editor with support for multiple languages and extensions.
- **Postman** [19]
A tool for API testing, response viewing, and debugging to ensure they function as expected and integrate well with other services.
- **Overleaf** [15]
An online LaTeX editor for collaborative writing and formatting of academic documents and reports.
- **Figma** [10]
A cloud-based tool for designing user interfaces and prototyping.
- **GitLab** [11]
A version control system platform for code management, issue tracking, and collaboration on software development projects.
- **PlantUML** [18] A tool for creating UML diagrams from a plain text language.
- **Expo** [7]
A platform for building native apps on Android, iOS, and the web using JavaScript and React.
- **Jupyter Notebook** [12]
A web app for creating and testing script chunks interactively and visualizing data directly within the notebook.

Frameworks and Libraries

- **Laravel** [14]
A PHP framework for building front-end and back-end packages.
- **React Js** [21]
A JavaScript library for creating dynamic web interfaces.
- **React Native** [20]
A framework for developing cross-platform mobile apps with a single codebase.
- **FastAPI** [9]
A modern, fast and high-performance web framework for building APIs with Python

Programming Languages

- **Typescript**

A JavaScript superset that adds static typing, aiding in scalable application development.

- **PHP**

A server scripting language, and a powerful tool for making dynamic and interactive Web pages.

- **Python**

A high-level programming language that emphasizes readability and efficiency with dynamic typing, supporting multiple programming paradigms.

Databases

- **MySQL** [17]

A reliable and flexible open-source relational database system.

AI technologies

- **Llama3** [16]

A Meta-developed language model known for its broad applicative abilities and human-like text generation.

- **LangChain** [13]

A library for integrating AI language models into applications.

- **ChromaDB** [3]

An open-source vector database designed for managing and performing similarity searches on text embeddings in AI and machine learning scenarios like recommendation systems and natural language processing tasks.

2.7.3 Deployment Diagram

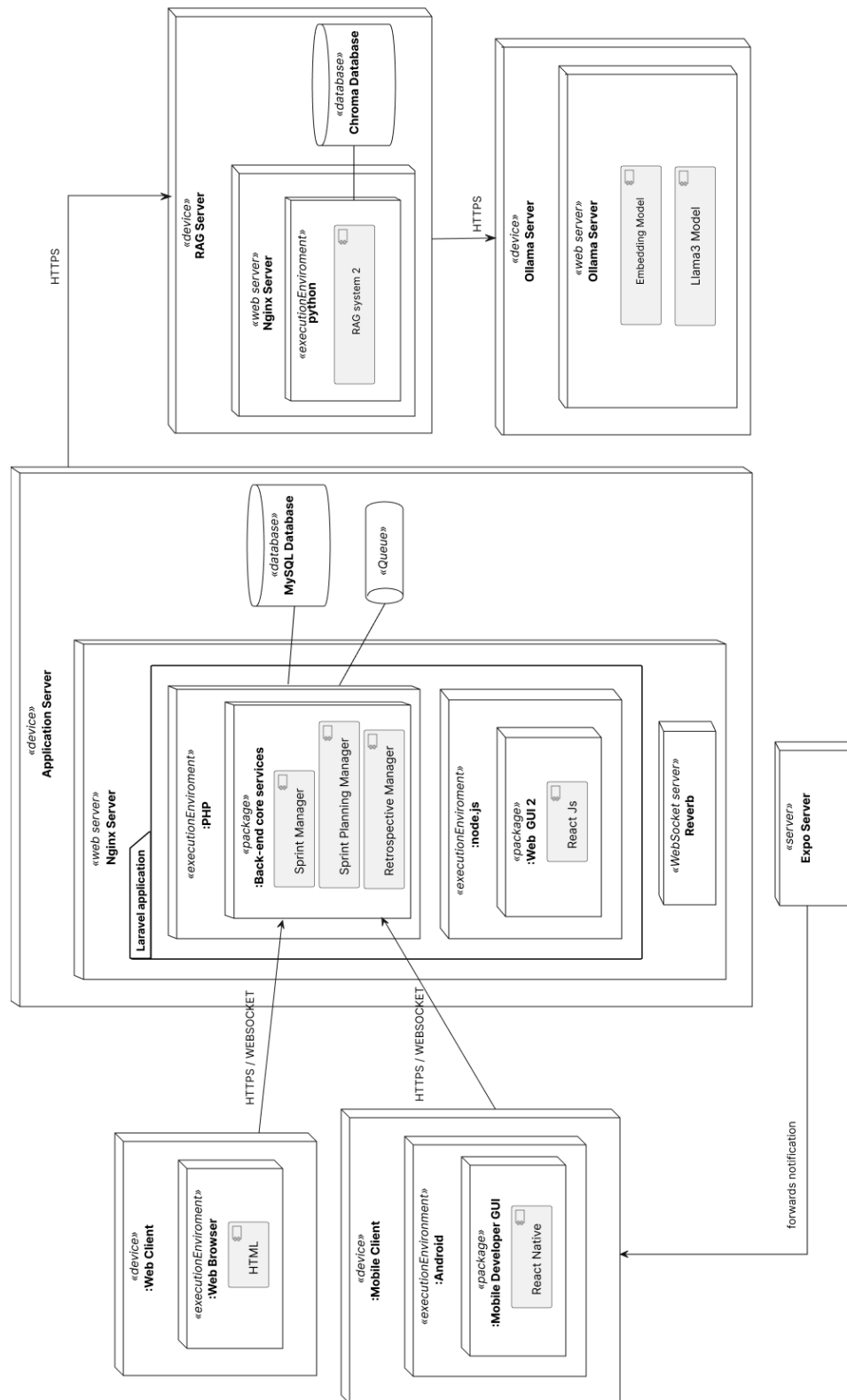


Figure 2.4: Deployment diagram of the Agile Deep Assistant 2

Figure 2.4 illustrates the deployment architecture of our project, showing how various servers and client devices interact.

Conclusion

In this chapter, we identified the functional and non-functional needs of our project then presented the general use case diagram, we detailed the architecture of our project, from a broad overview to a module specifications and the principles applied within this framework. Additionally, we provided an overview of the hardware and software environments utilized in the development and deployment processes. In the next chapter, we begin the development of the first sprint.

Chapter 3

Implementation of Sprint 1

Introduction

This chapter presents the first sprint which covers sprint management. First, we will present the sprint backlog. Then we discuss the analysis, design and implementation phases.

3.1 Sprint backlog

ID	As a scrum team leader, I can (User story)	Priority	Title
1	Add a sprint	High	Sprint Management
2	Start a sprint	High	
3	Stop a sprint	Medium	
4	Close a sprint	High	
5	Modify a sprint	Medium	
6	Delete a sprint	Low	
7	View insights of a sprint	High	
8	Consult the list of sprints	Medium	
9	Filter my sprints searches	Medium	
10	Add a user story to sprint backlog	High	Sprint Backlog Management
11	Switch a user story from one sprint backlog to another	Medium	
12	Update a user story from a sprint backlog	Medium	
13	Delete a user story from a sprint backlog	Low	
14	Update user stories in a sprint backlog	Medium	
15	Filter user stories within a sprint backlog	Medium	

Table 3.1: Sprint Backlog 1

3.2 Functional specifications

3.2.1 Sprint 1 Use Case Diagram

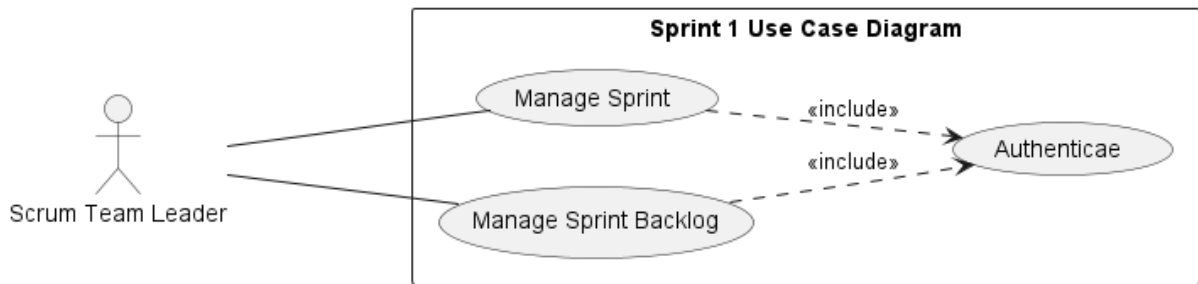


Figure 3.1: Use Case Diagram for sprint1

3.2.2 Refinement of first sprint use cases

During this section, we detail some use cases, describing in each case, the preconditions, the postconditions as well as the main scenario, and possible exceptions linked to this case.

Specification of the “Manage Sprint” use case

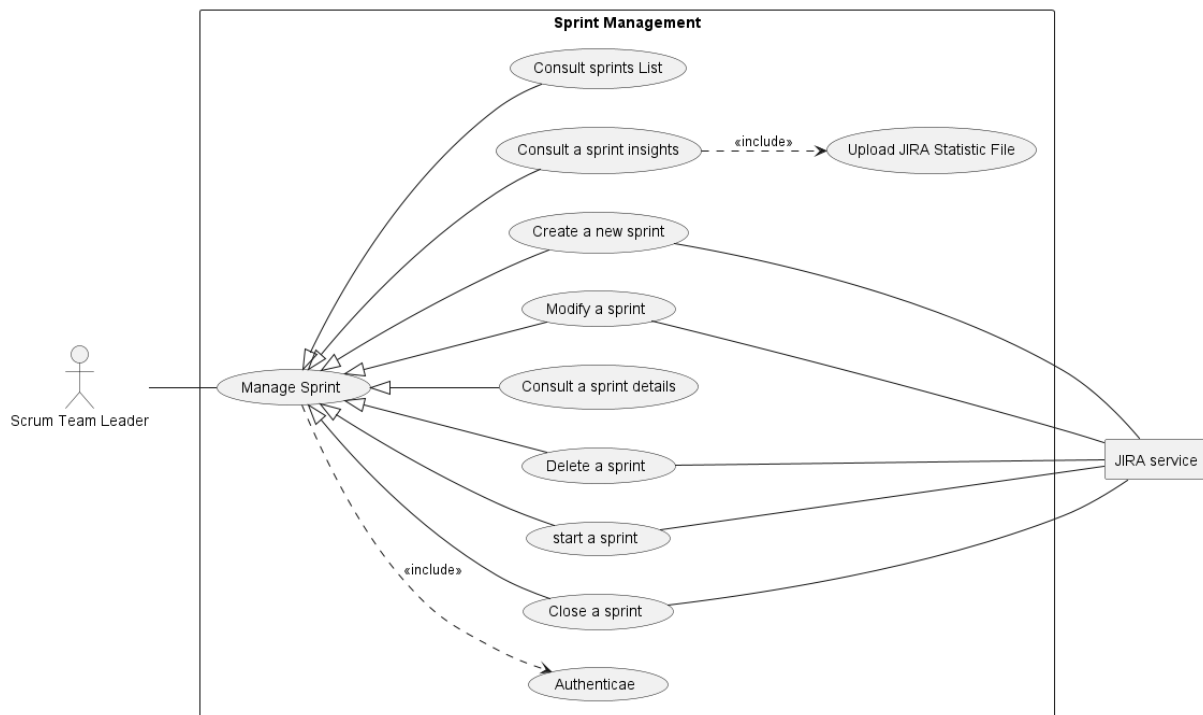


Figure 3.2: Use Case Diagram for sprint management

The figure 3.2 illustrates the Sprint Management System, emphasizing the integration with JIRA. This integration guarantees that any sprint-related operations conducted by the Scrum Team Leader are directly synchronized with the created project in JIRA platform.

Title	Create a Sprint
Actor	Scrum Team Leader(primary actor), JIRA (secondary actor)
Precondition	The Scrum Team Leader is authenticated and has access to the project for which the sprint is being planned.
Postcondition	A new sprint is successfully created and integrated within the system and in JIRA.
Main Scenario	1. Scrum Team Leader selects the option to create a new sprint. 2. Scrum Team Leader inputs the sprint details such as start date, end date, and goals. 3. The system validates the input and displays the available backlogs. 4. Scrum Team Leader drags and drops user stories from the backlog to the sprint backlog. 5. The system updates the user stories dragged to a specific sprint. 6. Scrum Team Leader clicks on the finish button and confirms. 7. The system uses Jira REST API [2] to create the sprint in JIRA, including the updated user stories. 8. The system confirms sprint creation to the Scrum Team Leader.
Exception Scenario	- If the input details are invalid (overlapping dates with existing sprints), the system displays an error message and requests the Scrum Team Leader to correct the details. - If a sprint with the same name already exists within the same project, the system notifies the Scrum Team Leader and prevents the creation of the duplicate sprint.

Table 3.2: Use Case Specification for "Create a Sprint"

Table 3.2 outlines the process for 'Create a Sprint', detailing the steps involved in setting up a new sprint within the system.

Title	Upload JIRA Statistic File in the Sprint
Actor	Scrum Team Leader
Precondition	The Scrum Team Leader is authenticated and has permissions to manage sprint data within the sprint management system.
Postcondition	The JIRA statistic file is uploaded, processed, and the sprint's user stories statuses are updated in the database.
Main Scenario	1. Scrum Team Leader accesses the sprint insights interface in the system. 2. Scrum Team Leader selects the option to upload a JIRA statistic file for the current sprint. 3. Scrum Team Leader uploads the file. 4. The system verifies the file format and integrity. 5. Upon successful validation, the system updates the user stories' statuses in the database based on the new data. 6. The system confirms the successful processing and updating of data to the Scrum Team Leader, and displays the sprint insights.
Exception Scenario	- If the uploaded file format is incorrect, the system displays an error message indicating the issue and prompts the Scrum Team Leader to upload a correctly formatted file.

Table 3.3: Use Case Specification for "Upload JIRA Statistic File in the Sprint"

Table 3.3 presents the use case specification for 'Upload JIRA Statistic File in the Sprint,' detailing the procedure for uploading statistical files at the end of a sprint to display insights.

Specification of the “Manage Sprint Backlog” use case

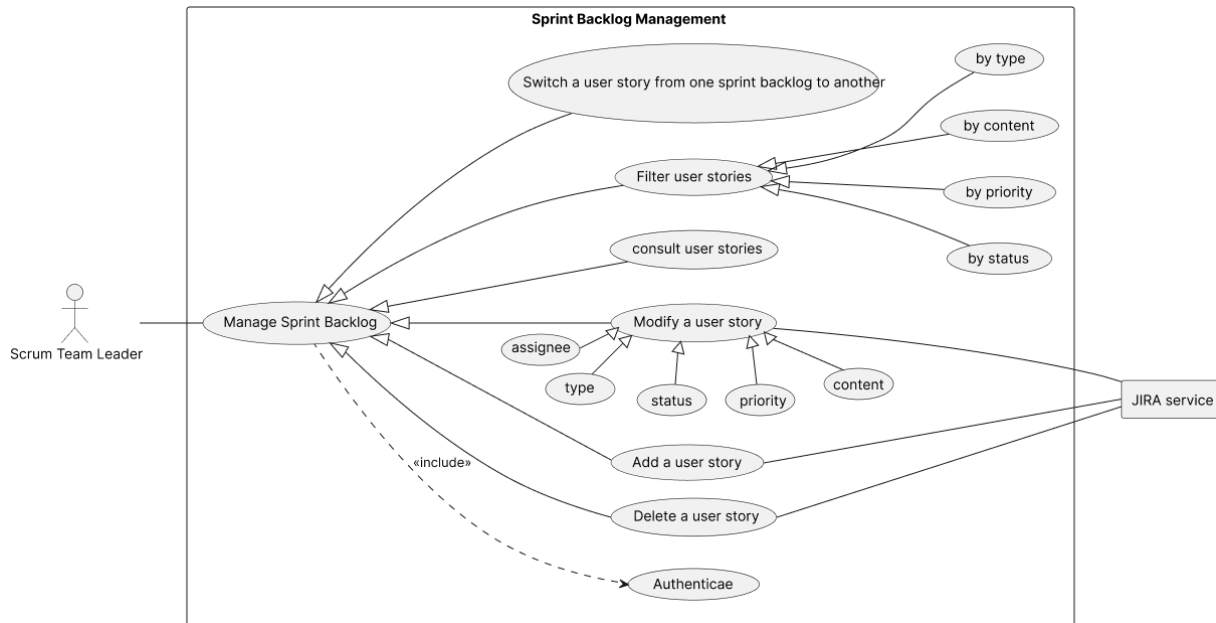


Figure 3.3: Use Case Diagram for sprint backlog management

The diagram 3.3 illustrates the interactions between the “Scrum Team Leader” and the “Sprint Backlog Manager” system.

Title	Add a New User Story in a Sprint
Actor	Scrum Team Leader (primary actor), JIRA (secondary actor)
Precondition	The Scrum Team Leader is authenticated and has access to the specific sprint within the project management system.
Postcondition	A new user story is added to the sprint, both within the internal system and in JIRA.
Main Scenario	1. Scrum Team Leader accesses the sprint screen within the sprint management system. 2. Scrum Team Leader selects the option to add a new user story. 3. Scrum Team Leader inputs the title for the user story. 4. Upon submission, the system validates the details for completeness and consistency. 5. If validated successfully, the system creates the new user story in the system and uses JIRA’s REST API to create the issue in JIRA, adding it to the designated sprint. 6. The system confirms the addition to the Scrum Team Leader, displaying a success message.
Exception Scenario	- If there is a failure in communicating with JIRA’s REST API (network issues, JIRA downtime), the system logs the error and notifies the Scrum Team Leader of the failure to sync with JIRA, advising a retry.

Table 3.4: Use Case Specification for “Add a New User Story in a Sprint”

3.3 System Design

This section details the architecture within our sprint module, illustrating how the various components interact and communicate with each other. We use class diagrams and sequence diagrams to clearly describe these relationships and flows.

3.3.1 Class Diagrams

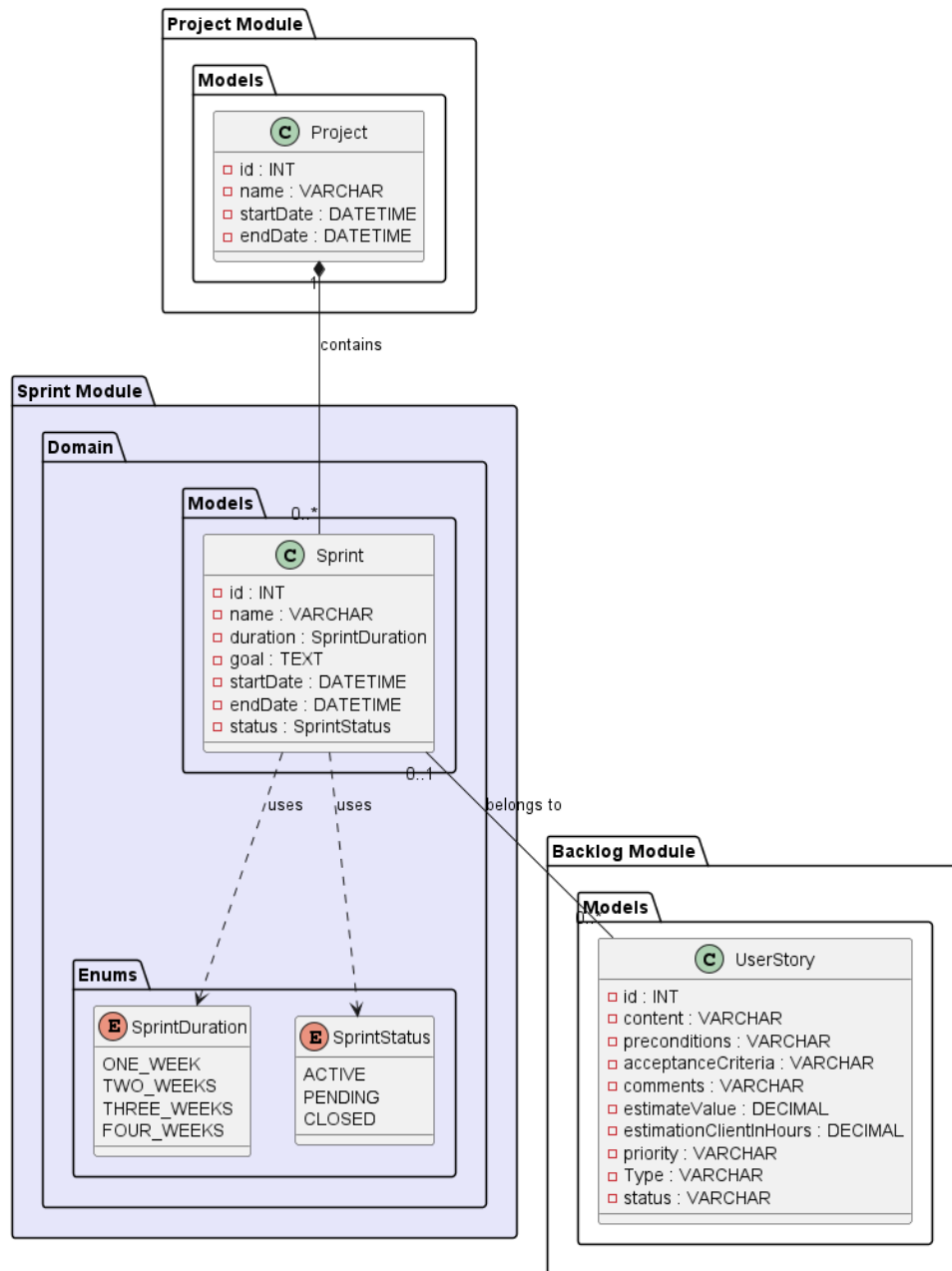


Figure 3.4: Sprint 1 Models package class diagram

The figure 3.4 showcases the relationships between the Sprint module (colored in gray) and other modules within the modular architecture.

- Each Sprint must belong to a Project and uses the enums SprintDuration and SprintStatus.
- Each Project can contain multiple Sprints.

- Each UserStory can belong to a Sprint when it is in the sprint backlog. If it is in the product backlog, it does not belong to any Sprint.

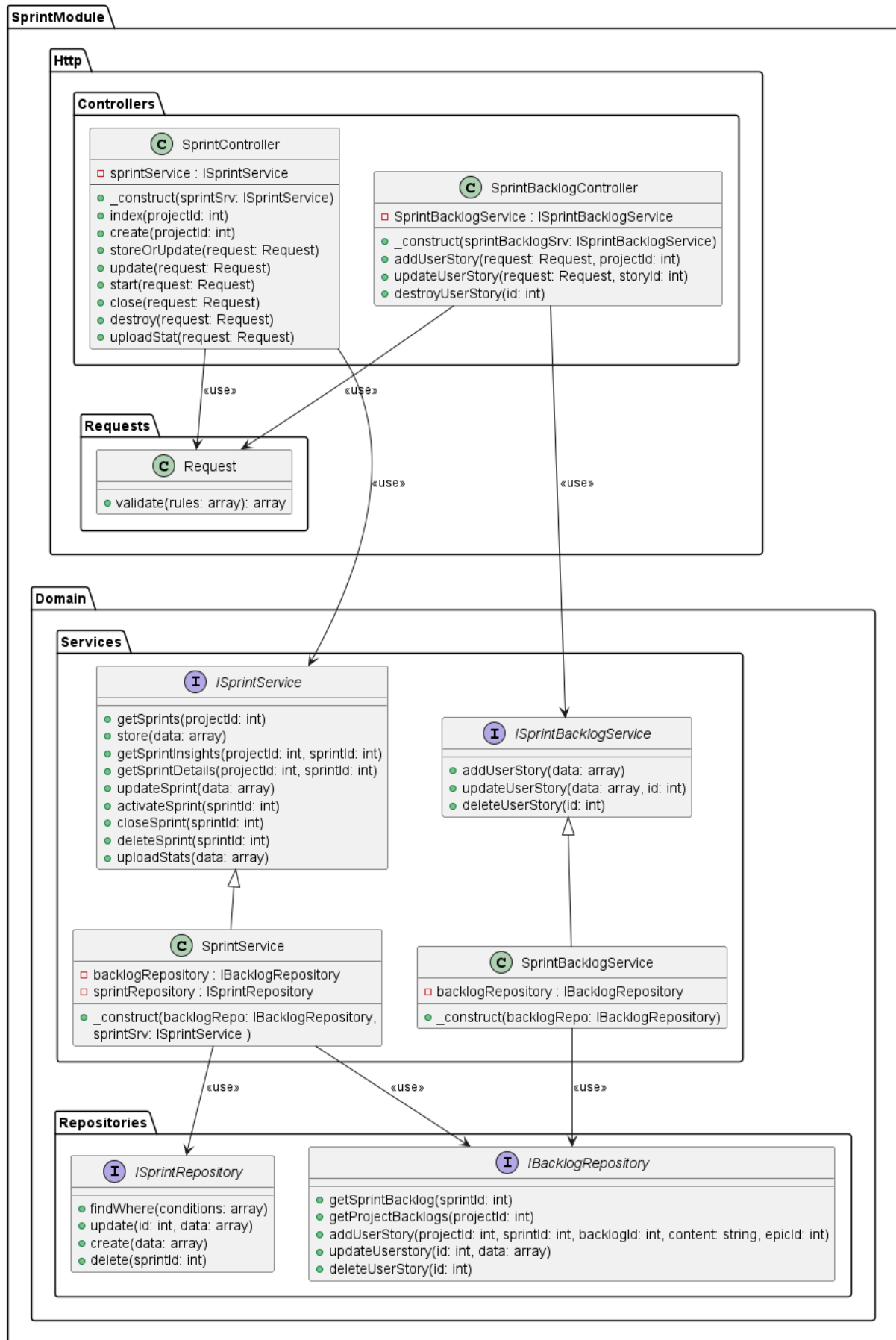


Figure 3.5: Sprint 1 Controllers - Services interaction class diagram

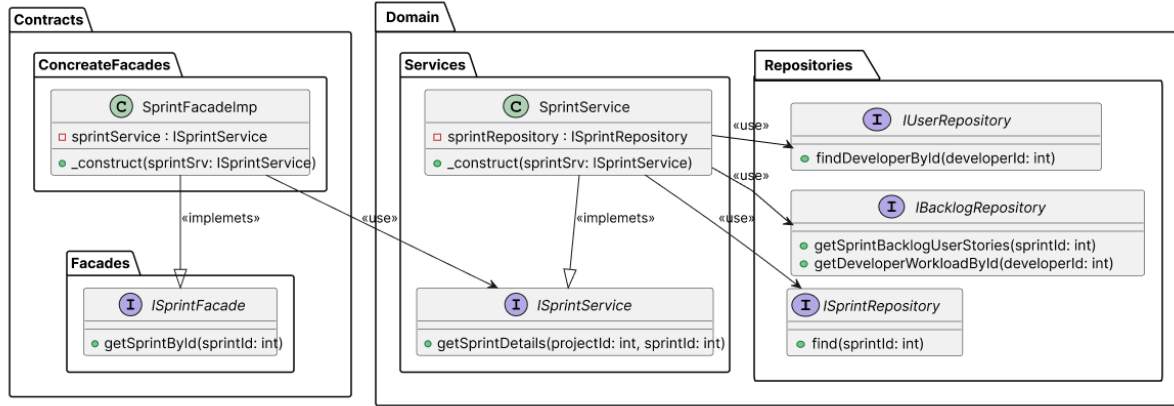


Figure 3.6: Sprint 1 Facades-Services interaction class diagram

The figure 3.6 outlines the structure and relationships between the facade and service classes in the Sprint Manager module through a **Facade pattern** [29].

This pattern ensures that the client module interacts with a simple and consistent interface, while the underlying service handles the complexity of data retrieval and aggregation.

The "getSprintById" function is exposed in the facade to be used by other modules providing a detailed view of the sprint and its associated information.

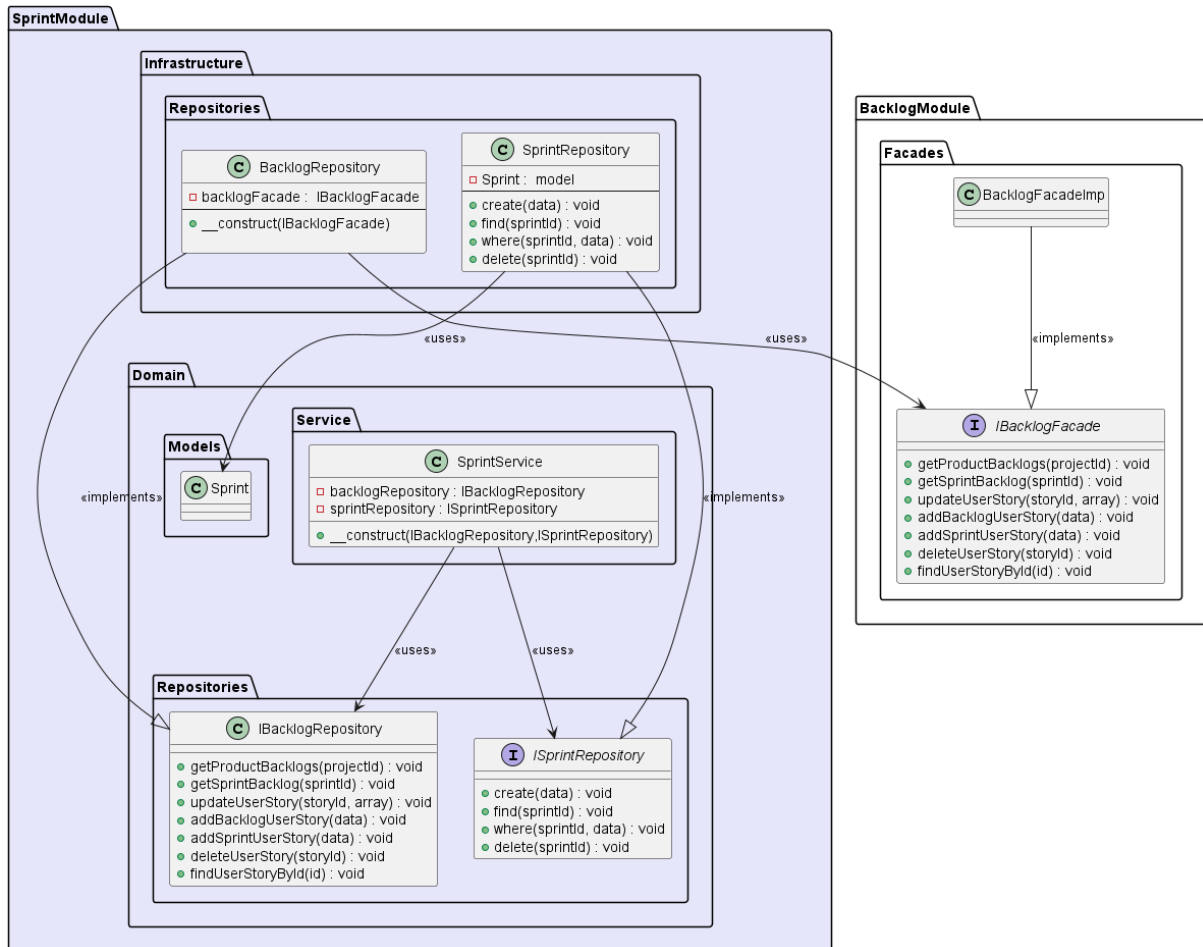


Figure 3.7: Repositories-Services interaction Class Diagram

The figure 3.7 outlines the structure and relationships between the facade, service, and repositories classes in the Sprint and Backlog modules.

It showcases two SOLID principles in action:

- **Single Responsibility Principle (SRP) [26]:** Each class handles a specific part of the system:
 - SprintRepository manages sprint-related data.
 - BacklogRepository handles backlog-related data.
- **Dependency Inversion Principle (DIP) [28]:** High-level modules like "SprintService" depend on abstractions "IBacklogRepository" and "ISprintRepository" rather than concrete implementations, which helps in decoupling the system components.

The repositories in this diagram represent a data access layer for both inside and outside the SprintModule:

- Inside SprintModule: "SprintRepository" directly interacts with the Sprint model to manage sprint data.
- Outside SprintModule: "BacklogRepository" uses the "IBacklogFacade" to access backlog functionalities from Backlog Module using Facade pattern.

3.3.2 Sequence diagrams

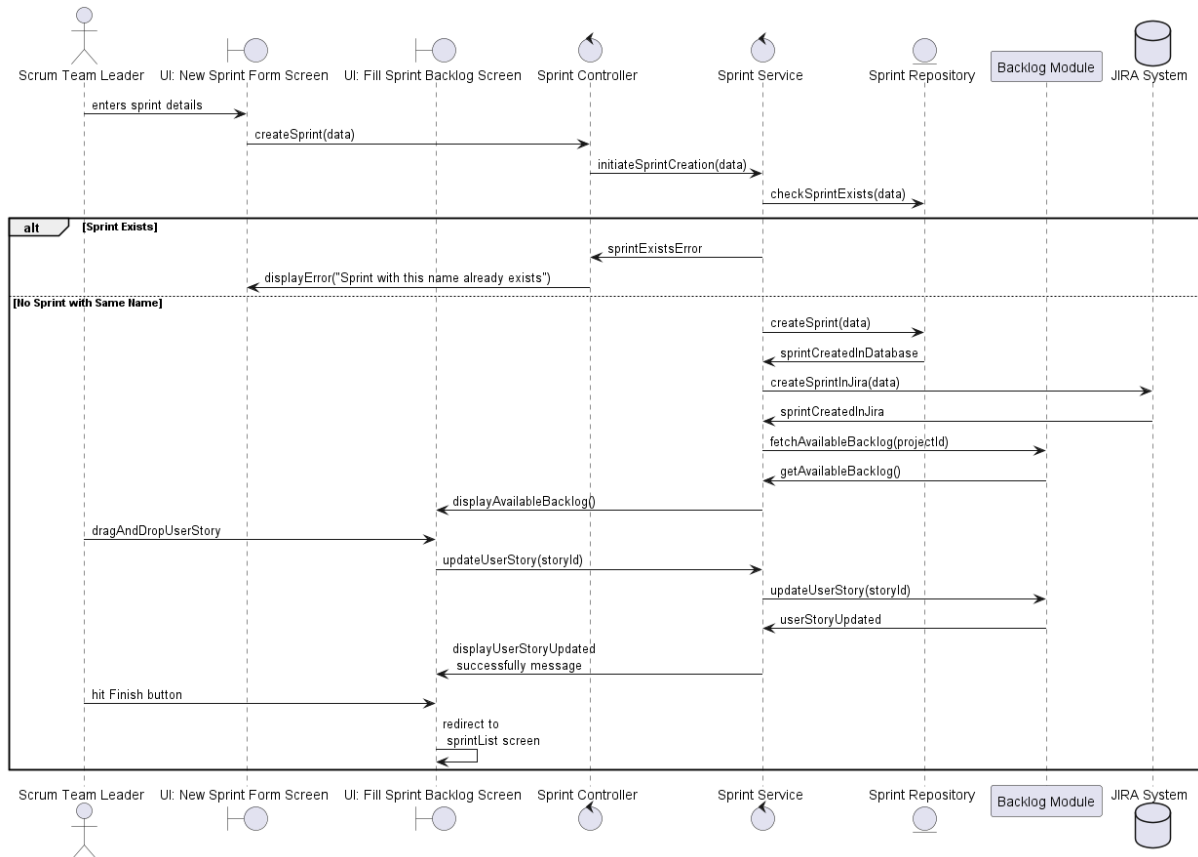


Figure 3.8: Create Sprint Sequence Diagram

The sequence diagram in figure 3.8 describes the process of creating a new sprint and then potentially creating a sprint in JIRA with the provided data.

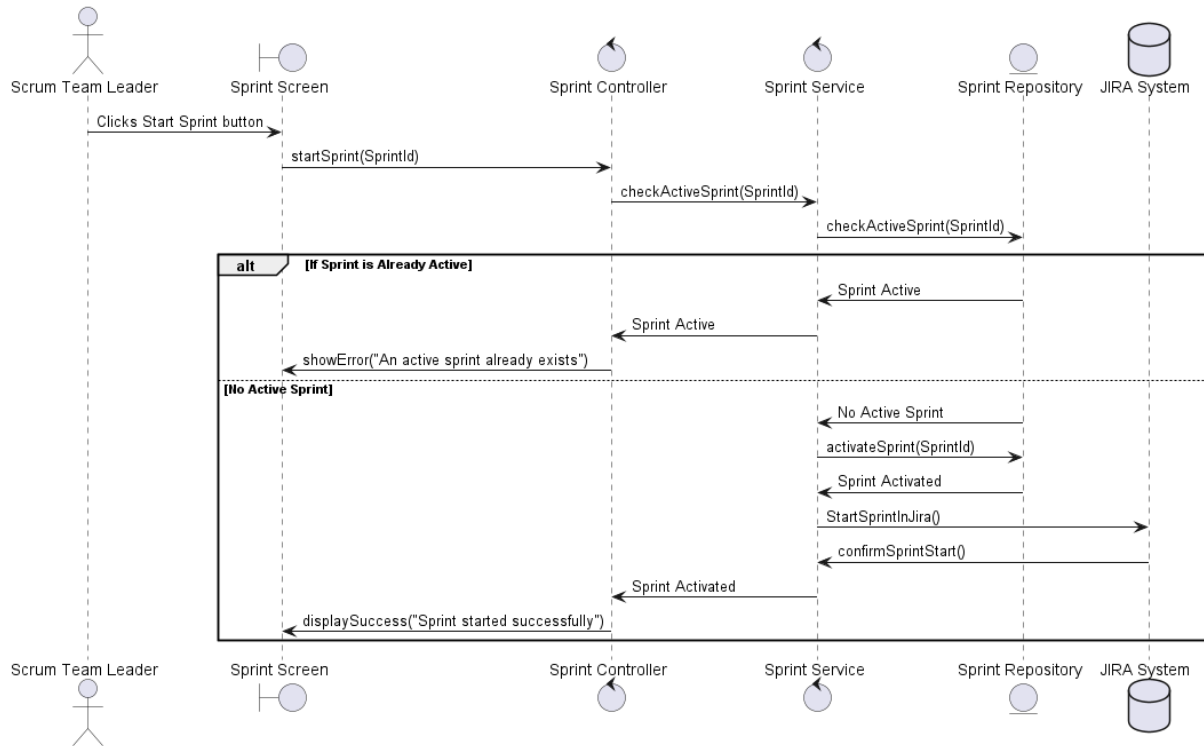


Figure 3.9: Start Sprint Sequence Diagram

Figure 3.9 illustrates the process of starting a sprint, where the Scrum Team Leader initiates the action, the system activates the new sprint and synchronizes it with JIRA, confirming the successful start back to the Scrum Team Leader.

3.4 Implementation

In this section we will present a selection of screenshots showing some interfaces made during this sprint.

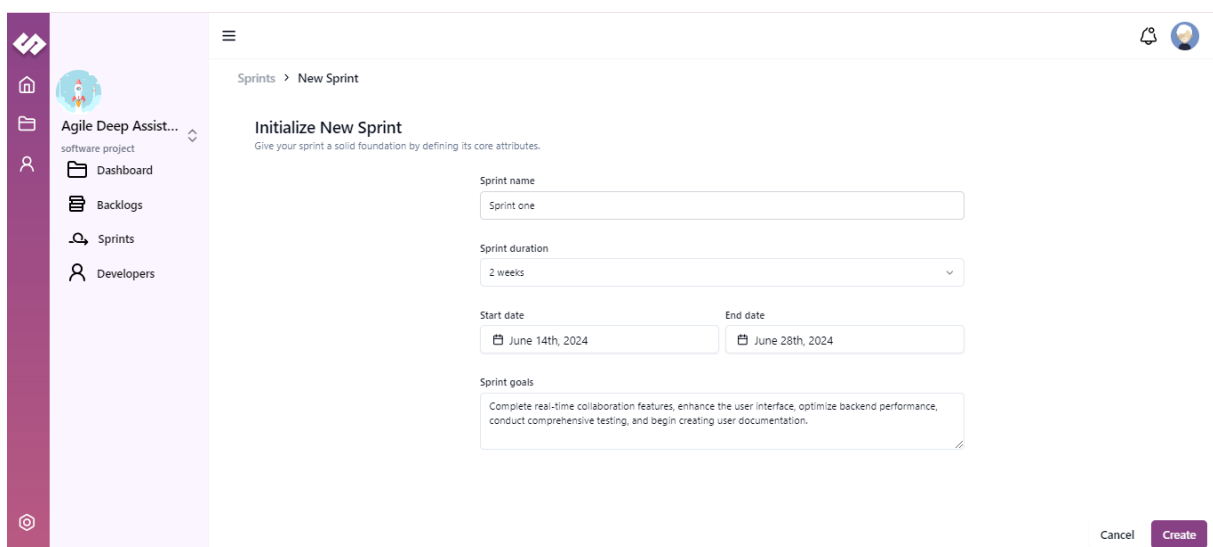


Figure 3.10: Create New Sprint Interface

Figure 3.10 illustrates the interface for creating a new sprint, where the scrum team leader

can enter details such as name, duration, and goals.

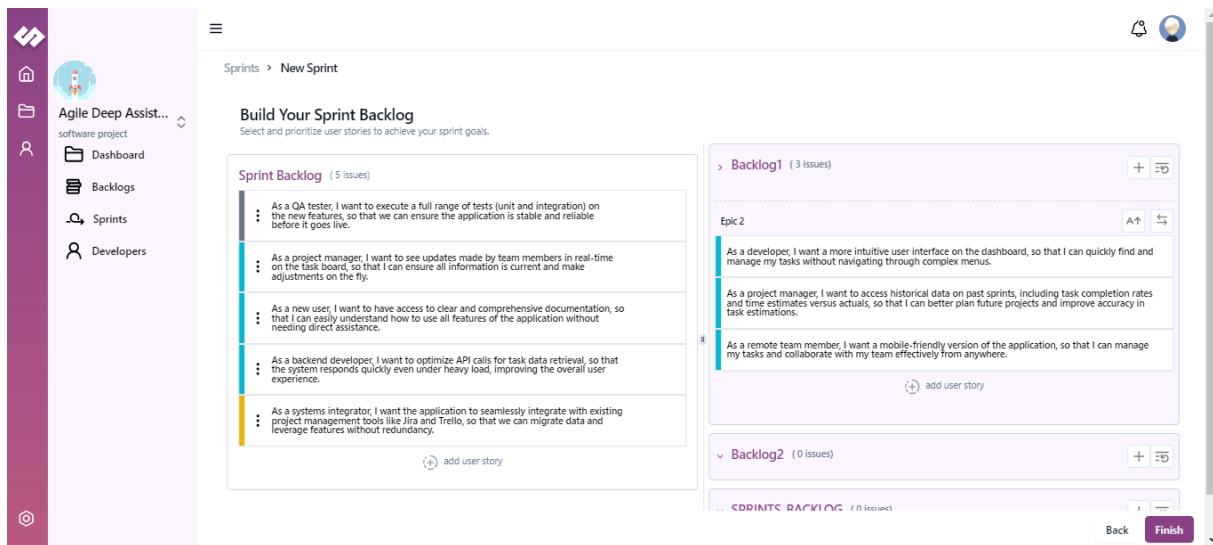


Figure 3.11: Initialize Sprint Backlog Interface

Figure 3.11 outlines the interface for initializing a sprint backlog, enabling the scrum team leader to drag and drop user stories from available project backlogs into the sprint backlog.

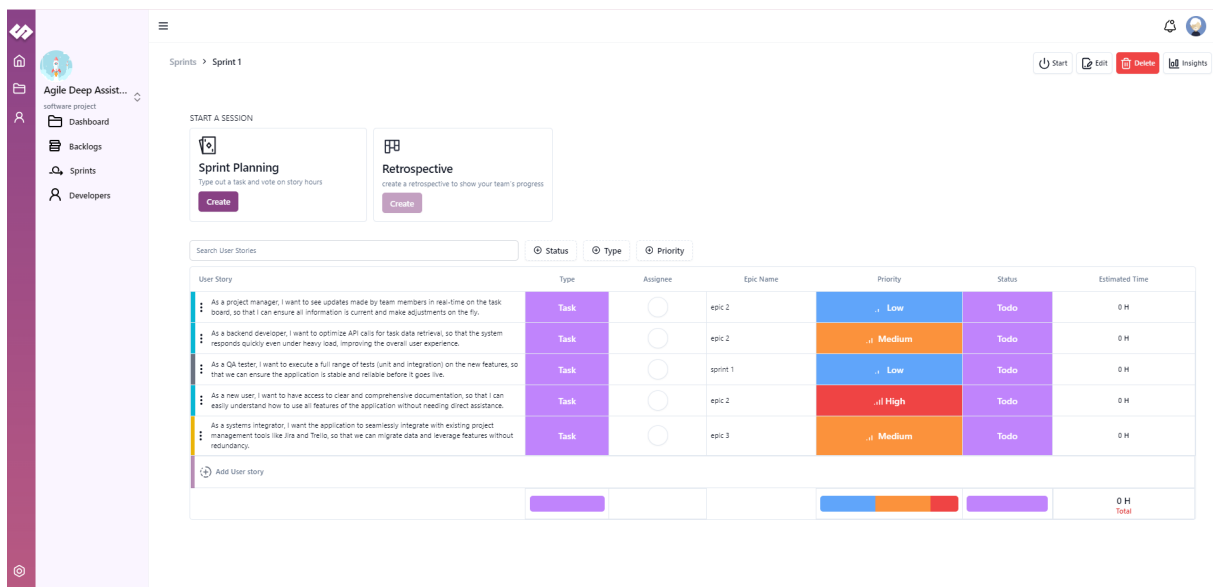


Figure 3.12: Sprint Details Interface

Figure 3.12 describes the interface for viewing sprint details, where the scrum team leader can consult the status, type, and priority of each user story and apply various filters.

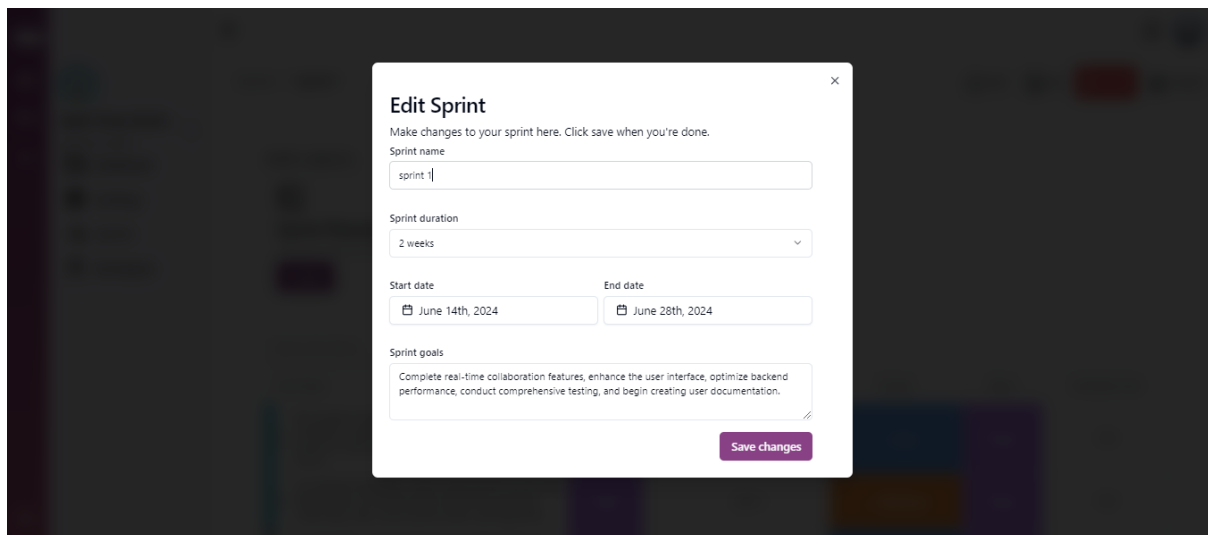


Figure 3.13: Edit Sprint Interface

Figure 3.13 illustrates the interface for editing an existing sprint.

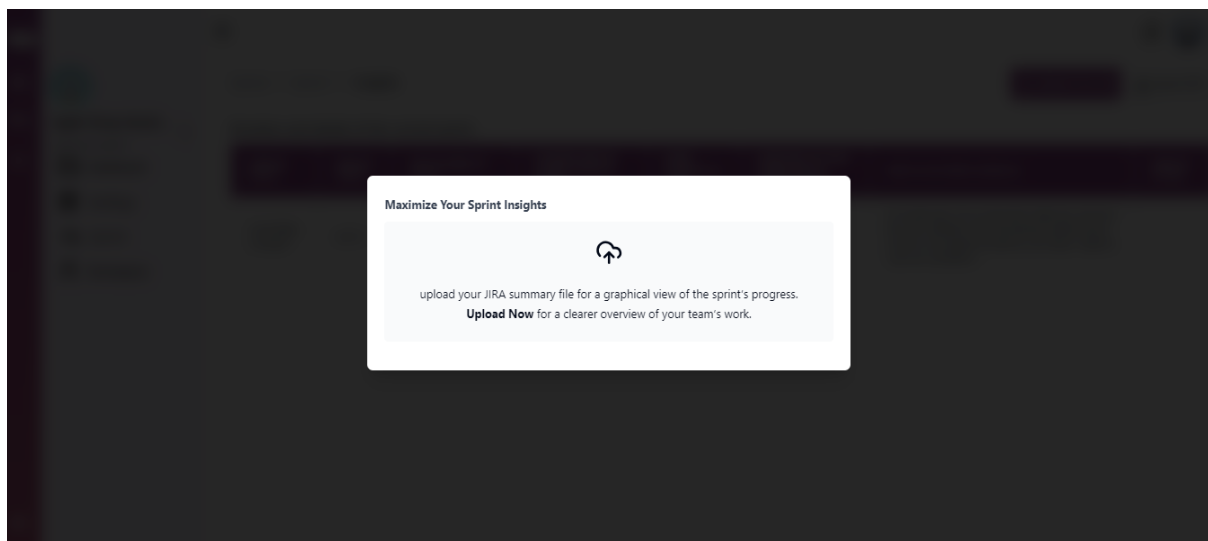


Figure 3.14: Upload Jira Sprint's Statistic Interface

Figure ?? outlines the interface for uploading a Jira sprint statistics file to synchronize it with our system.

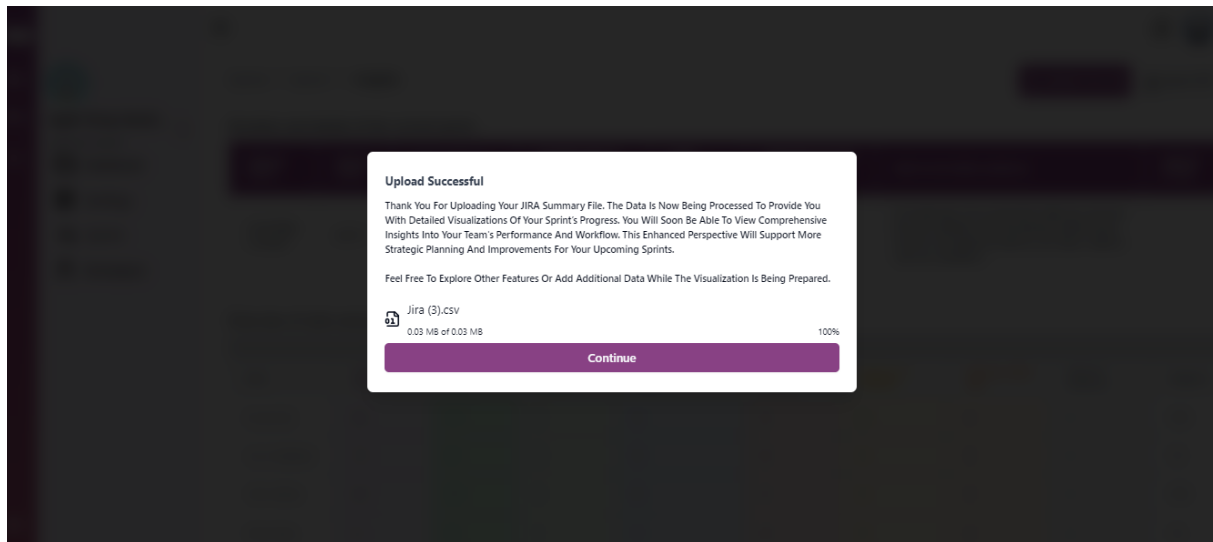


Figure 3.15: Upload Success confirmation

Figure 3.15 describes the interface that confirms successful upload of data.

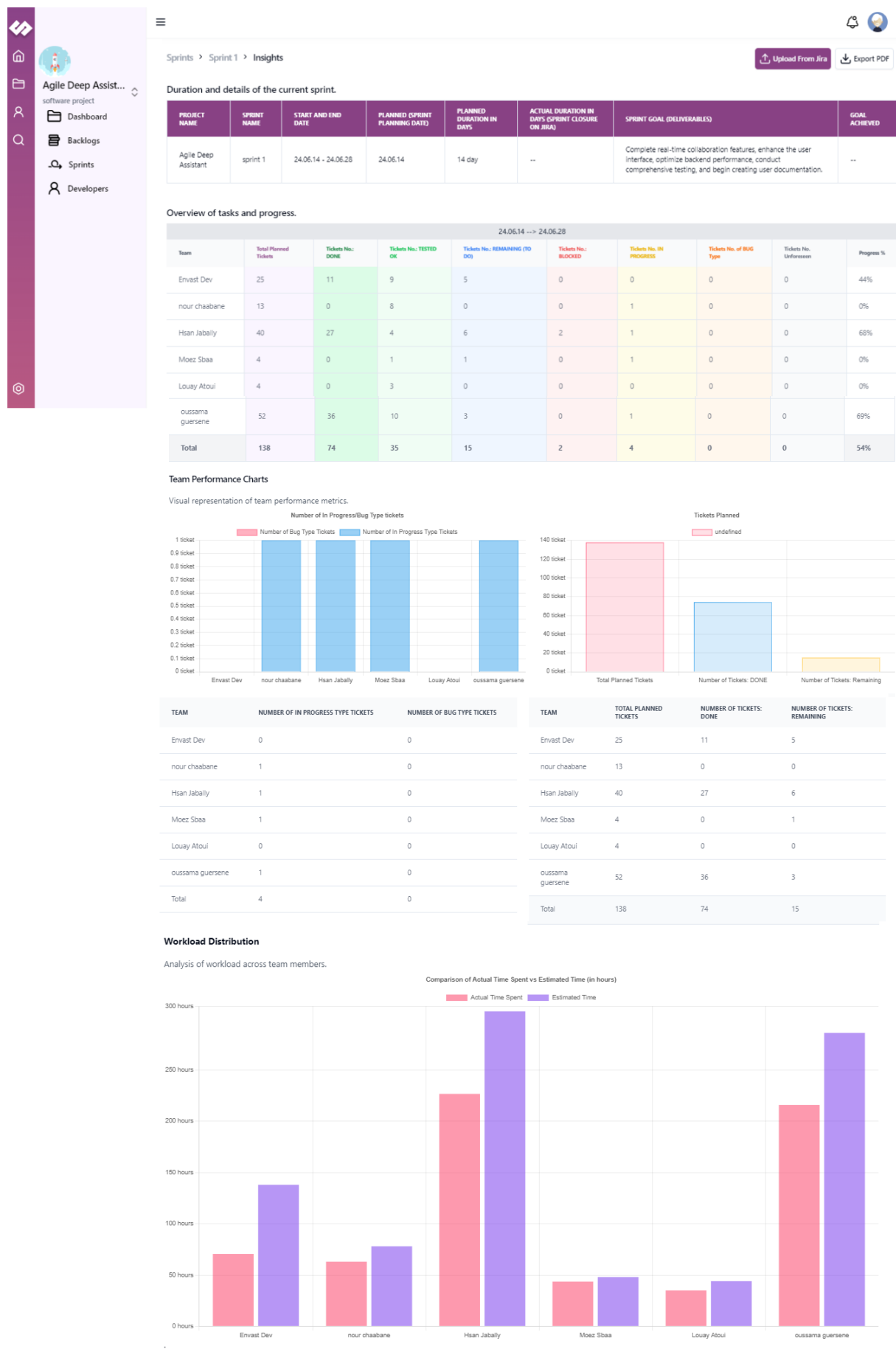


Figure 3.16: Sprint Details Insights

Figure 3.16 illustrates the interface for viewing sprint details insights, showcasing progress in user stories (completed, pending, blocked) and individual developer contributions, displayed in simple tables and charts.

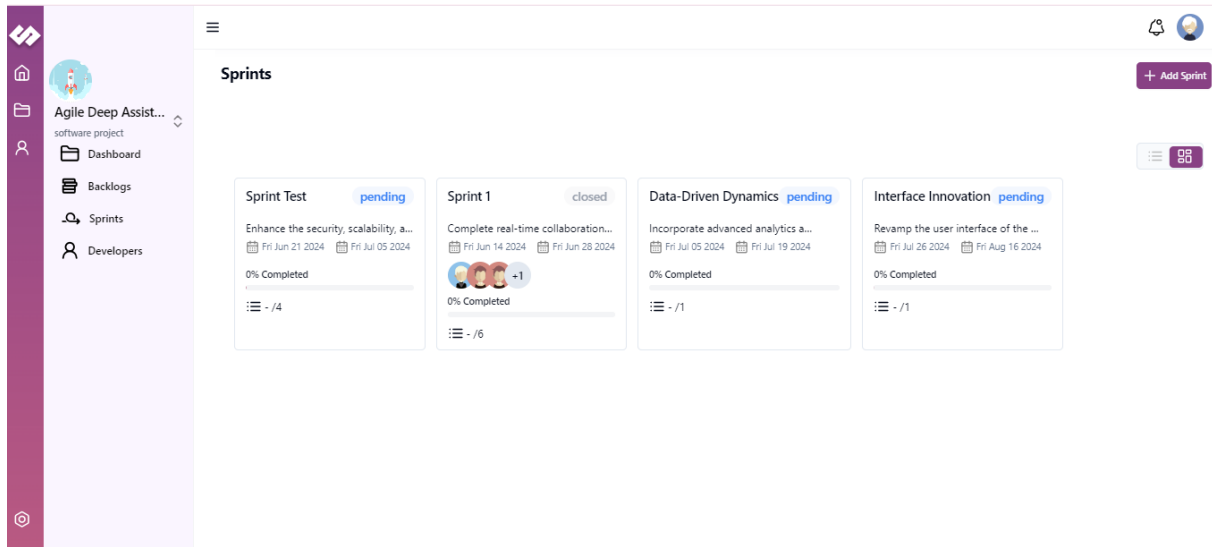


Figure 3.17: Sprints List : Cards View Interface

Figure 3.17 outlines the interface for the cards view of the sprints list.

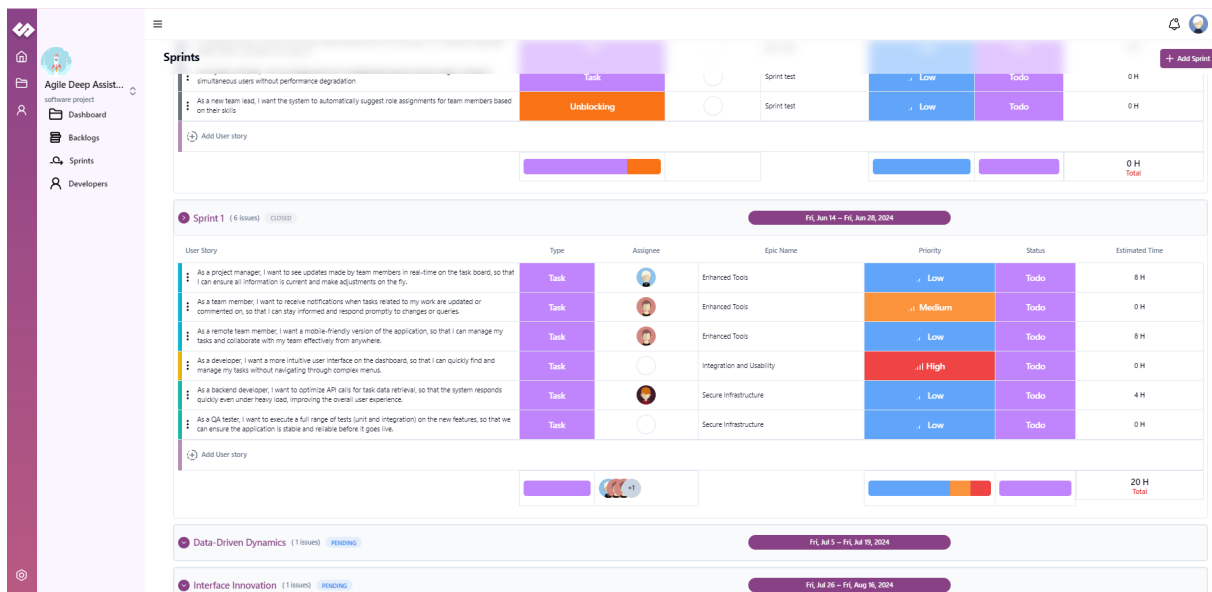


Figure 3.18: Sprints List : List View Interface

Figure 3.18 describes the interface for the list view of the sprints list, where the scrum team leader can view detailed sprint information, including its sprint backlog, and manage user stories by dragging and dropping them between sprints.

Conclusion

During this chapter, we presented the design of our first sprint, utilizing various UML diagrams, and showcased the implementation of different interfaces. In the next chapter, we will introduce the second sprint.

Chapter 4

Implementation of Sprint 2

Introduction

This chapter continues with the implementation details of Sprint 2, focusing on sprint planning management for the Scrum team. First, we will outline the sprint backlog. Then, we will present the phases of analysis, design, and implementation.

4.1 Sprint backlog

Table 4.1 summarizes the backlog for our second sprint:

ID	As a Scrum Team Leader, I can (User story)	Priority	Title
1	Create a room for estimation	High	Sprint Planning Management
2	Add user story from available backlogs	Medium	
3	Import user story	Medium	
4	Consult the details of a story	Low	
5	Invite developer to complete the estimate	High	
6	Start the estimate	High	
7	Moderate the meeting (skip, clear votes, next stories)	High	
8	Consult a developer's estimate	Medium	
9	Consult the history of estimation room	Low	
10	Generate Recommendations for Suitable Developers	High	Developpers Assignment Management
11	Assign a developer for a user story	High	
12	Review the list of assignment requests from developers	Medium	
13	Assign a developer for a user story	High	
14	Modify the assignment of a developer for a user story	Medium	
15	Remove the assignment of a developer for a user story	Low	
ID	As a developer, I can (User story)	Priority	Title
16	Consult the list of user stories planned for an estimate	Medium	User Story Estimation Management
17	Join a sprint planning meet	High	
18	Consult the details of a story	Medium	
19	Estimate a user story	High	
20	Modify the estimate of a user story	Medium	
21	Vote to take charge of a user story	High	
22	Receive a notification to complete estimates	High	Developer Notification Consultation
23	Receive notification of my assignment on a user story	High	

Table 4.1: Sprint Backlog 2

4.2 Functional specifications

4.2.1 Sprint 2 Use Case Diagram

The functionalities to be designed and developed in this sprint are represented by the use case diagram in Figure 4.1.

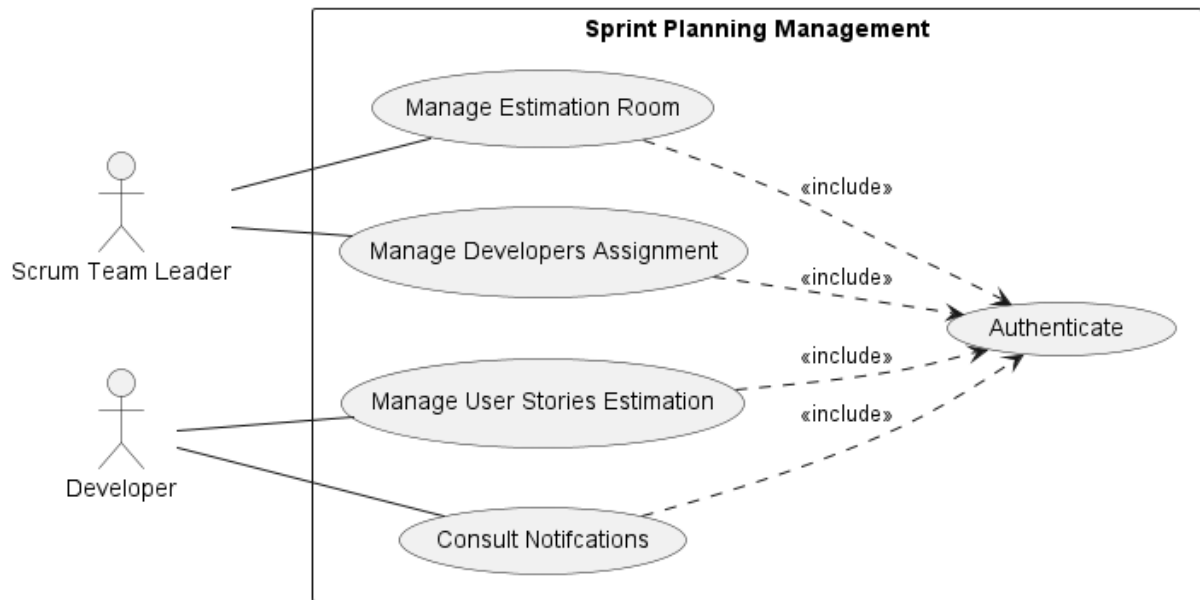


Figure 4.1: Sprint 2 general Use Case Diagram

The diagram in Figure 4.1 showcases the interactions of the Scrum Team Leader and the Developers with the "Sprint Planning Management" system.

The Scrum Team Leader is responsible for managing sprint planning rooms and developer assignments. Meanwhile, Developers handle user story estimations within the room and consult notifications related to their assigned user stories for confirmation.

4.2.2 Refinement of second sprint use cases

Specification of the "Manage Sprint Planning Room" use case

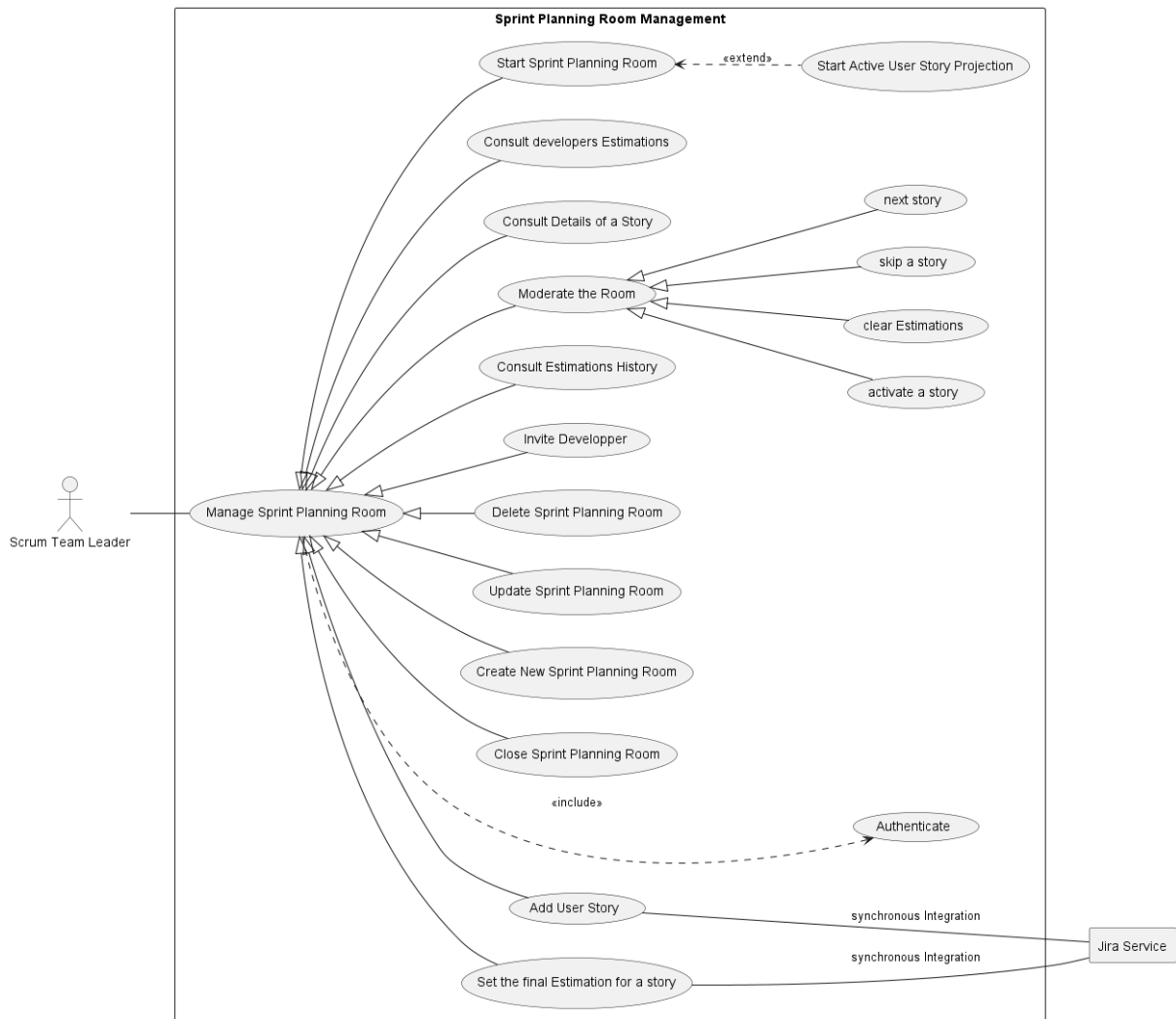


Figure 4.2: Manage Sprint Planning Room Use Case Diagram

The diagram in Figure 4.2 presents the functionalities within the "Sprint Planning Room Management" system, operated by the Scrum Team Leader. The Scrum Team Leader is responsible for managing various aspects of the estimation room, from starting and closing the room to moderating meetings, view estimation histories and developers' estimates. Actions such as setting the final estimation for a user story or adding a user story to the sprint backlog during the meeting are directly integrated with the JIRA Service to ensure synchronization of data.

Title	Create New Sprint Planning Room
Actor	Scrum Team Leader
Precondition	The Scrum Team Leader is authenticated and has selected a project and sprint for planning.
Postcondition	A new sprint planning session is created with user stories loaded for estimation, and developers are invited to participate.
Main Scenario	1. Scrum Team Leader initiates the creation of a new sprint planning estimation session. 2. The system retrieves sprint details and associated user stories. 3. The system retrieves all available backlogs and their associated epics for the project. 4. The Scrum Team Leader moves user stories from various product backlogs into the sprint backlogs via drag-and-drop and enters details for the poker planning session, which includes inviting developers. 5. The system validates the provided information. 6. Upon successful validation, the system schedules the poker planning session and sends notifications to the invited developers.
Exception Scenario	If a sprint planning session already exists for the sprint, the system displays an error message, "A sprint planning session already exists for this sprint." If there is an error in creating the session or sending invites, an error message is displayed and the user is prompted to retry.

Table 4.2: Use Case Specification for "Create New Sprint Planning"

The table 4.2 presents the use case specification for 'Create New Sprint Planning,' detailing the process involved in establishing a new sprint planning session within the system.

Specification of the "Manage Developers Assignment" use case

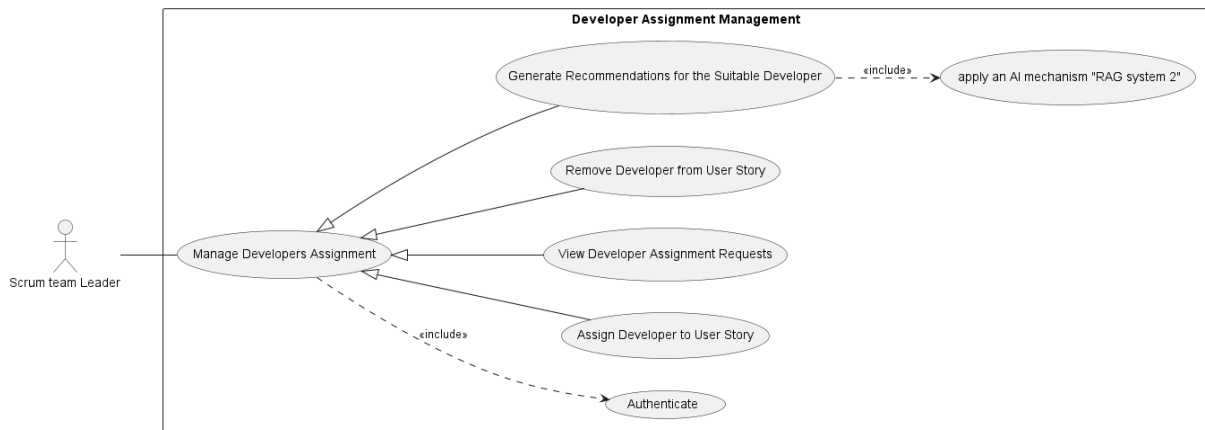


Figure 4.3: Manage Developers Assignment Use Case Diagram

The diagram in Figure 4.3 illustrates the "Developer Assignment Management" system overseen by the Scrum Team Leader. This system handles the assignment of developers to user stories, including the assignment, review of assignment requests, and removal of developers. Also it integrates with the RAG System 2 to provide recommendations on the most relevant developers for a user story based on previous performance, enhancing assignment decisions.

Title	Assign a Developer to a User Story
Actor	Scrum Team Leader
Precondition	The Scrum Team Leader is authenticated, and the estimation room is active. A list of available developers for the sprint is present.
Postcondition	A developer is assigned to the current user story based on their expertise and past experience with similar tasks, and the developer is notified of the assignment.
Main Scenario	1. Scrum Team Leader requests to view a list of developers in the sprint. 2. The system retrieves and displays the list of available developers 3. Scrum Team Leader initiates a search to find the most suitable developer for a specific user story. 4. The system uses ChromaDB to perform a similarity search among developers based on their past user stories within a RAG system 2. 5. The RAG system 2 employs the Llama model to refine the results by understanding context and nuances of user stories and developer skills. 6. The system displays the matched developers for the user story. 7. Scrum Team Leader selects a developer based on the provided recommendations. 8. The system assigns the selected developer to the user story and updates the task assignment records. 9. The system sends a notification to the assigned developer, informing them of the new task assignment.
Exception Scenario	If the similarity search or the Llama model encounters an issue, the system provides an error message and clearly states that it cannot accurately determine the best match and suggests manual assignment.

Table 4.3: Use Case Specification for "Assign a Developer to a User Story"

Table 5.4 describes the Use Case Specification for "Assign a Developer to a User Story."

The effectiveness of identifying suitable developers derives from knowledge sourced from custom sprint statistic files, initially uploaded to our system as observed in the first sprint. Since these have been enhanced in the current sprint to integrate seamlessly with the RAG system 2.

- After successful file upload validation, the system stores the file within the RAG system 2 file storage.
- The system then reads and processes the file, converting each row into a comprehensible text format.
- The processed data is subsequently saved in the vector database to enhance future semantic searches, utilizing embeddings for improved accuracy.

Specification of the "Manage User Stories Estimation" use case

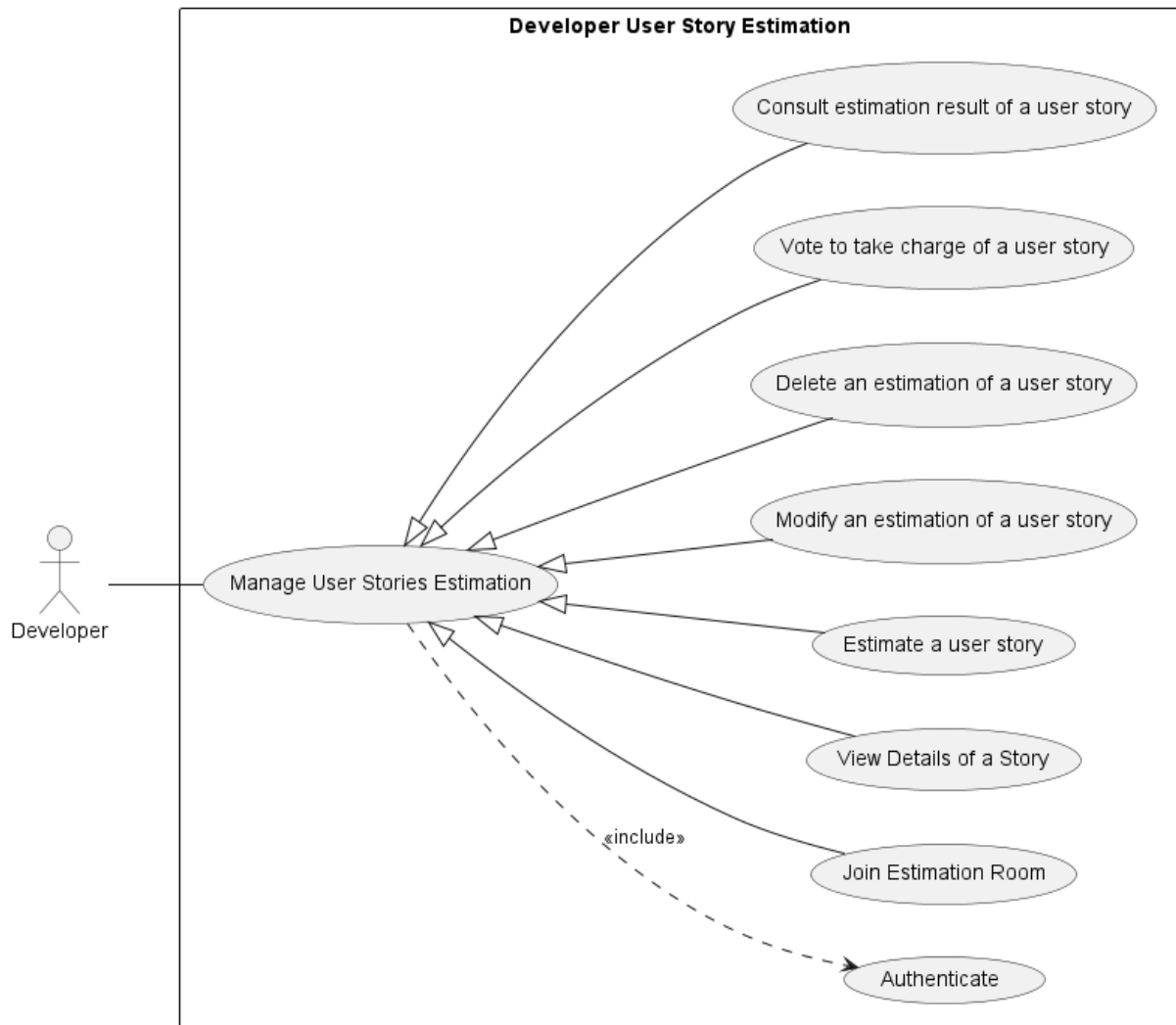


Figure 4.4: Manage User Stories Estimation Use Case Diagram

The diagram in Figure 4.4 outlines the "Developer User Story Estimation" system managed by Developers. This system allows developers to interact with user stories during the sprint planning meeting. The core functionalities include joining the estimation room, viewing story details, estimate and vote to take charge of a specific user story.

Title	Estimate a User Story
Actor	Developer
Precondition	The Developer is authenticated and invited to the sprint planning session.
Postcondition	The Developer's estimation is recorded, and the Scrum Team Leader's dashboard is updated in real-time.
Main Scenario	1. Developer submits an estimation value for a user story through his mobile application. 2. The system receives the estimation and checks for validity. 3. The system verifies if the Developer has already submitted an estimation for the user story. 4. Based on the verification, the system either records a new estimation or updates an existing one. 5. The system then associates the estimation with the Developer's details. 6. The system updates the Scrum Team Leader's dashboard with the new estimation. 7. The system notifies the UI to display a success message to the Developer.
Exception Scenario	If the estimation process fails, the system informs the Developer through the UI, suggesting a resubmission of the estimation.

Table 4.4: Use Case Specification for "Developer Story Estimation Submission"

4.3 System Design

4.3.1 Class Diagrams

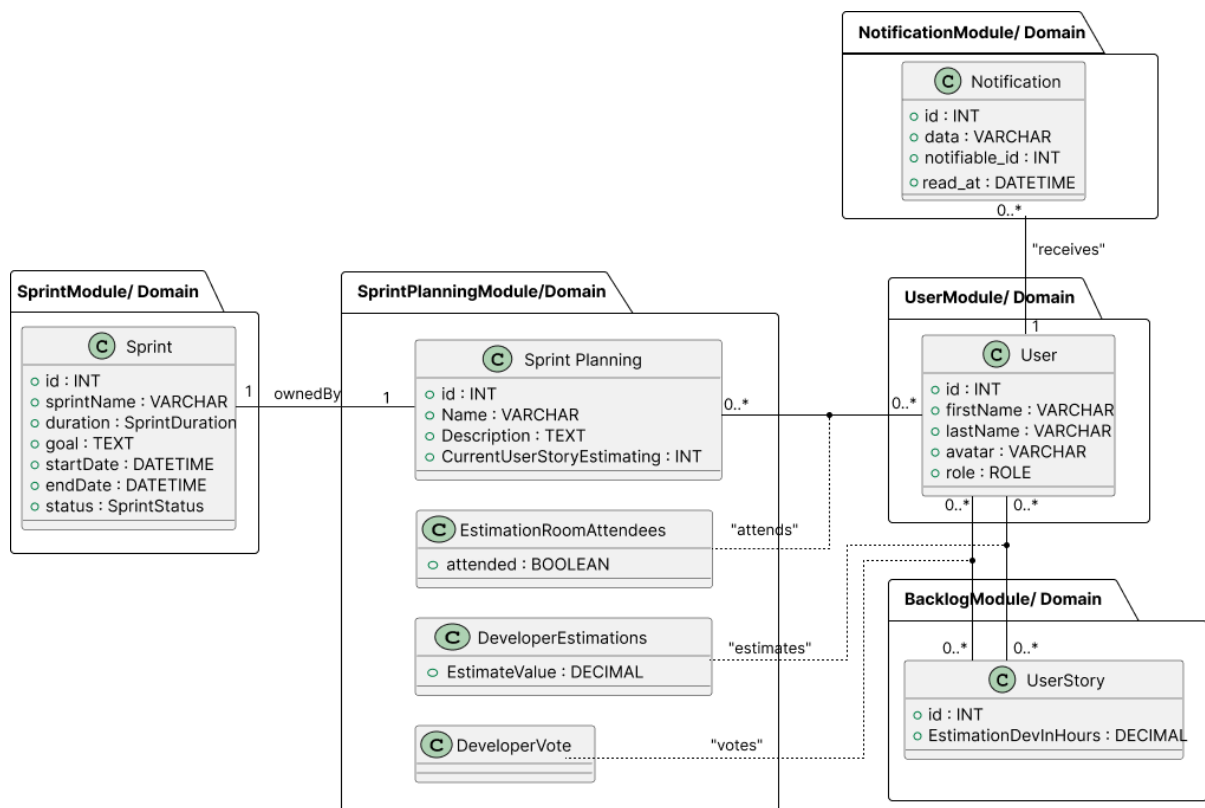


Figure 4.5: Sprint2 Models Class Diagram

Figure 4.5 illustrates the interrelationships between entities within the Sprint Planning Module and other related modules within our agile project management system. Each sprint is linked to a single sprint planning session. Developers from the User Module are deeply engaged, taking part in sprint planning, receiving notifications, attending the estimation rooms and voting and estimating to the user stories from the Backlog Module.

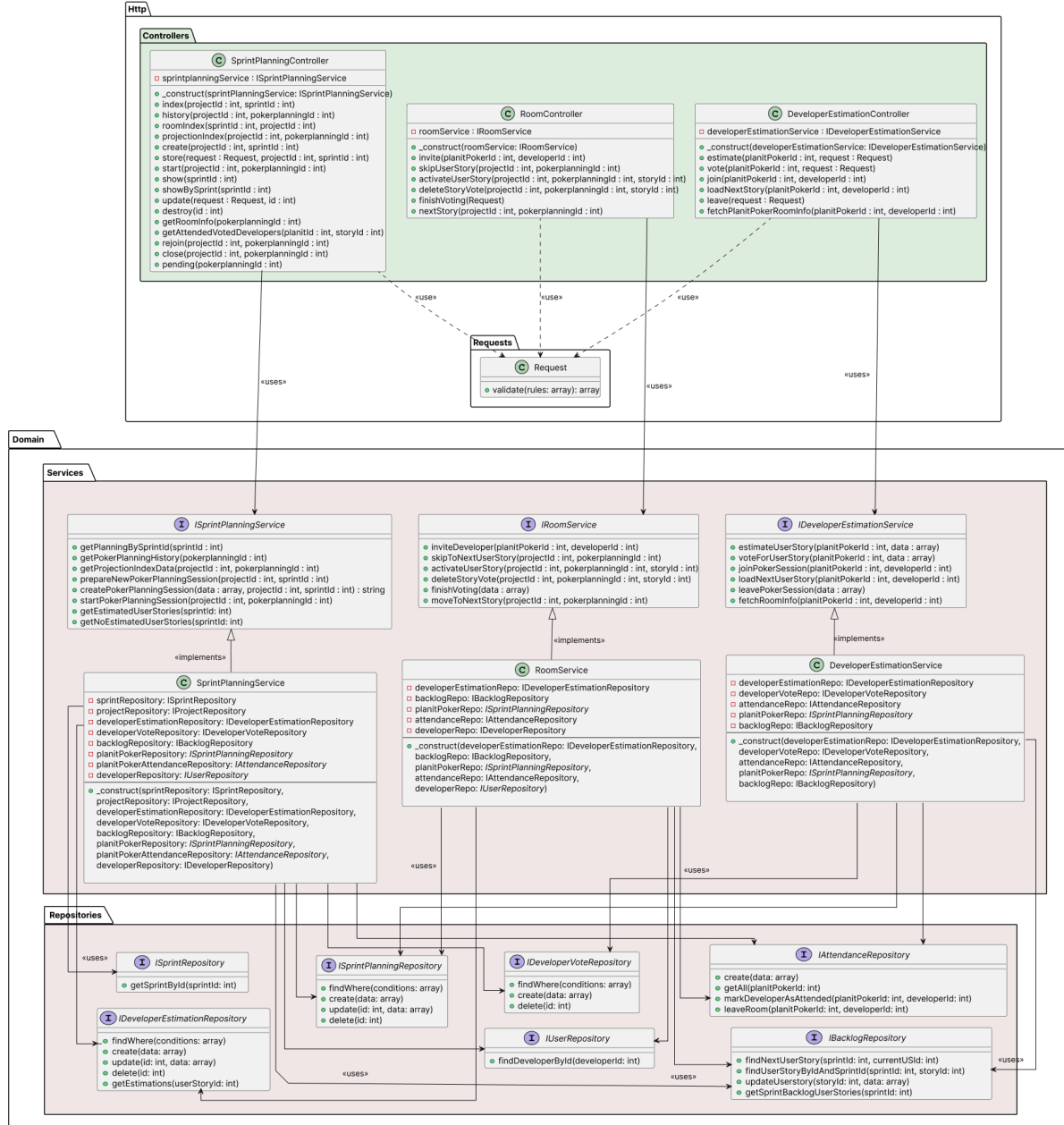


Figure 4.6: Sprint2 Controllers-Services-Repositories interaction Class Diagram

This diagram in figure 4.6 showcases the architecture of the SprintPlanningModule, highlighting how its components interact within a comprehensive system. It effectively demonstrates the use of **dependency injection** via interfaces integration, where services are instantiated with necessary repositories to manage sprint planning, room handling, and developer estimations efficiently.

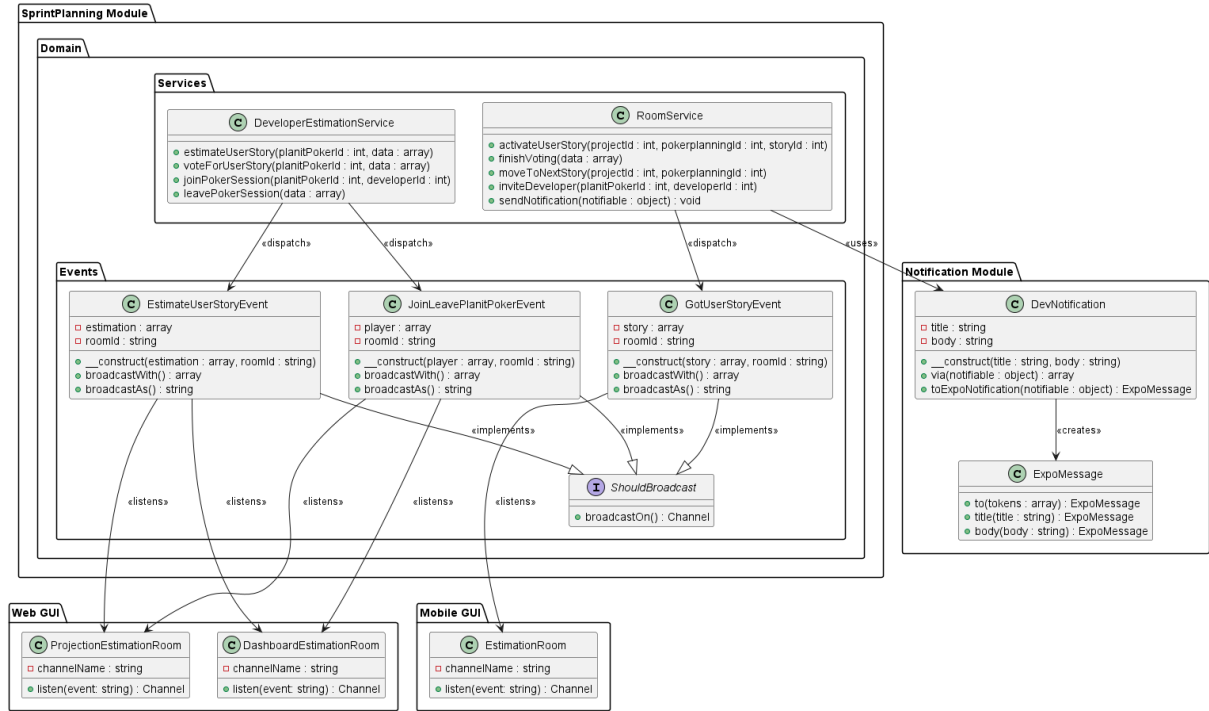


Figure 4.7: Sprint2 Events-Services-Notifications Interaction Class Diagram

Figure 4.7 illustrates the event-driven communication and notification system within a planning poker manager module. In the "Domain" package, events such as `EstimateUserStoryEvent`, `GotUserStoryEvent`, and `JoinLeavePlanitPokerEvent` are defined for broadcasting throughout the system. These events enable listeners to update their states accordingly.

A "Notification Module" has been incorporated to facilitate notifications using the **Expo notification service** [8] which provides an interface for managing and sending push notifications to iOS and Android devices using the Expo framework.

`RoomService` plays a critical role in this module, triggering notifications through methods such as `inviteDeveloper` and `finishVoting`. These methods utilize `DevNotification` to create notifications in the database and create `ExpoMessage` instances, which are sent to developers' devices to keep them informed about the ongoing activities in the estimation room.

The "Web GUI 2" and "Mobile GUI" packages contain UI pages that listen to specific events, ensuring dynamic and real-time responses to changes in the estimation process across both web and mobile interfaces. This real-time communication is crucial during sprint planning meetings, allowing the scrum master to monitor updates such as estimations and votes on their dashboard, while developers receive the current user story for voting, enhancing the interactive and collaborative environment of sprint planning sessions.

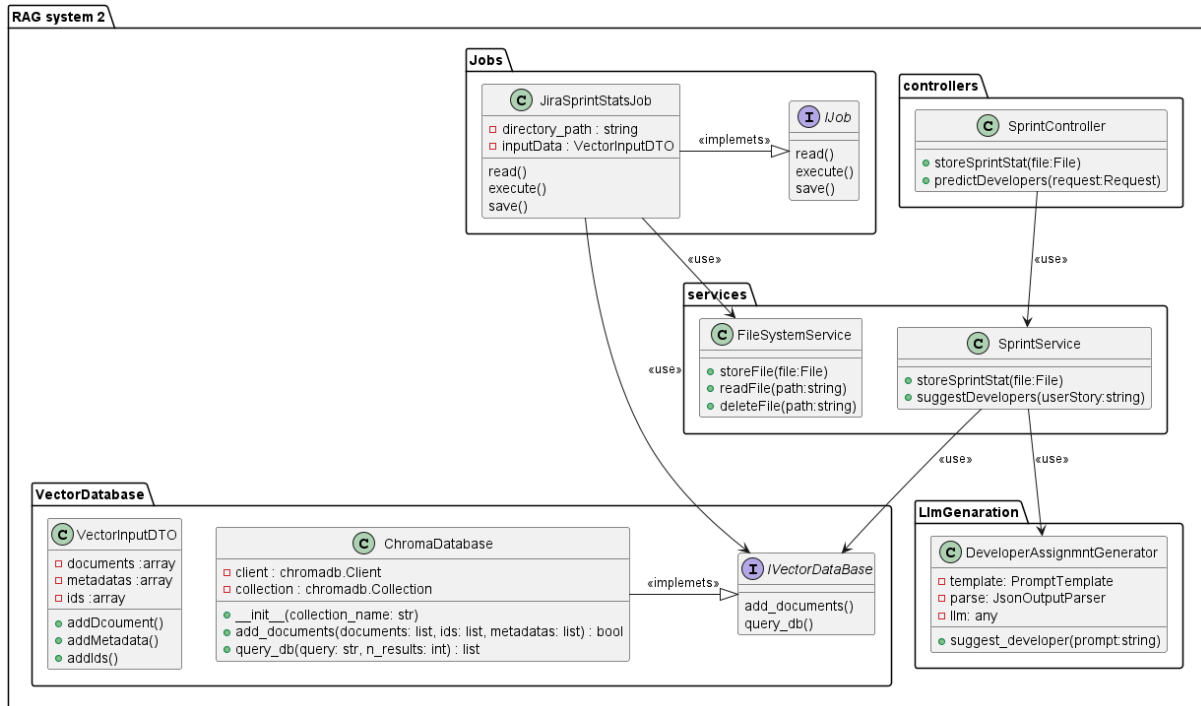


Figure 4.8: RAG System 2 class diagram

The figure 4.8 illustrates the architecture of our RAG system 2, which is designed to automate the assignment of user stories. The system is structured into distinct packages, each responsible for a different aspect of its operation:

- **Controllers Package:** Acts as the primary point for interfacing with users, receiving their inputs and directing them to appropriate services for further processing.
- **Services Package:** Manages the core functionalities of the system. It handles the operations related to the management of sprint data, such as its storage and retrieval, as well as file system operations which include the storing, reading, and deletion of files.
- **Vector Database Package:** This package is focused on the storage and retrieval of vector embeddings, essential for conducting efficient similarity searches. This capability allows the system to quickly pull relevant documents and data, leading to faster and more accurate query responses.
- **LLM Generation Package:** Dedicated to creating data-driven suggestions through the use of advanced language models. It employs predefined templates and parsers to ensure the accuracy and relevance of the responses generated by the models.
- **Jobs Package:** Automates the integration with external documents, ensuring that the system remains synchronized with real-time project data. It includes asynchronous jobs designed to automate the reading and processing of stored files. Each file is transformed into comprehensible phrases, and the transformed data, conforming to the 'VectorInput-DTO' format, is stored in the vector database using the 'addDocument' method of the 'IVectorDataBase'.

4.3.2 Sequence diagrams

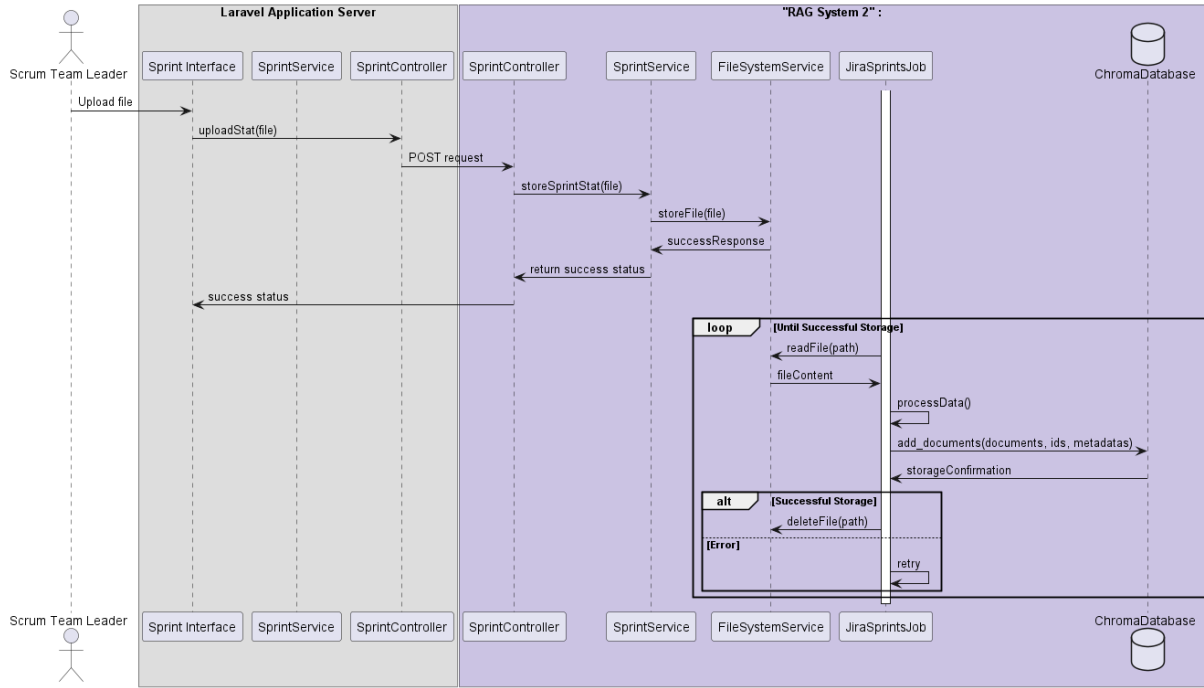


Figure 4.9: Uploading Jira Statistic File Sequence Diagram

The figure 4.9 describes the interactions between the Laravel Application Server and a RAG System 2. The process starts with the Scrum Team Leader uploading a statistics file exported from jira through the Sprint Interface. The Sprint Service receives this file and sends a POST request to a corresponding Sprint Controller in the RAG System 2. This controller then instructs its Sprint Service to store the sprint statistics file, which involves the File System Service saving the file and confirming the action back to the Sprint Service and Sprint Controller.

Once the file is successfully stored, the process within the RAG System 2 shifts to a continuous monitoring and processing operation handled by a Job. This job reads the file, processes its content, and attempts to store the processed data in the Chroma Database. If the database confirms successful storage, the original file is deleted. However, if there's an error in storing the data, the job retries the process from reading the file again. This loop continues until the data is successfully stored, ensuring data integrity and system robustness.

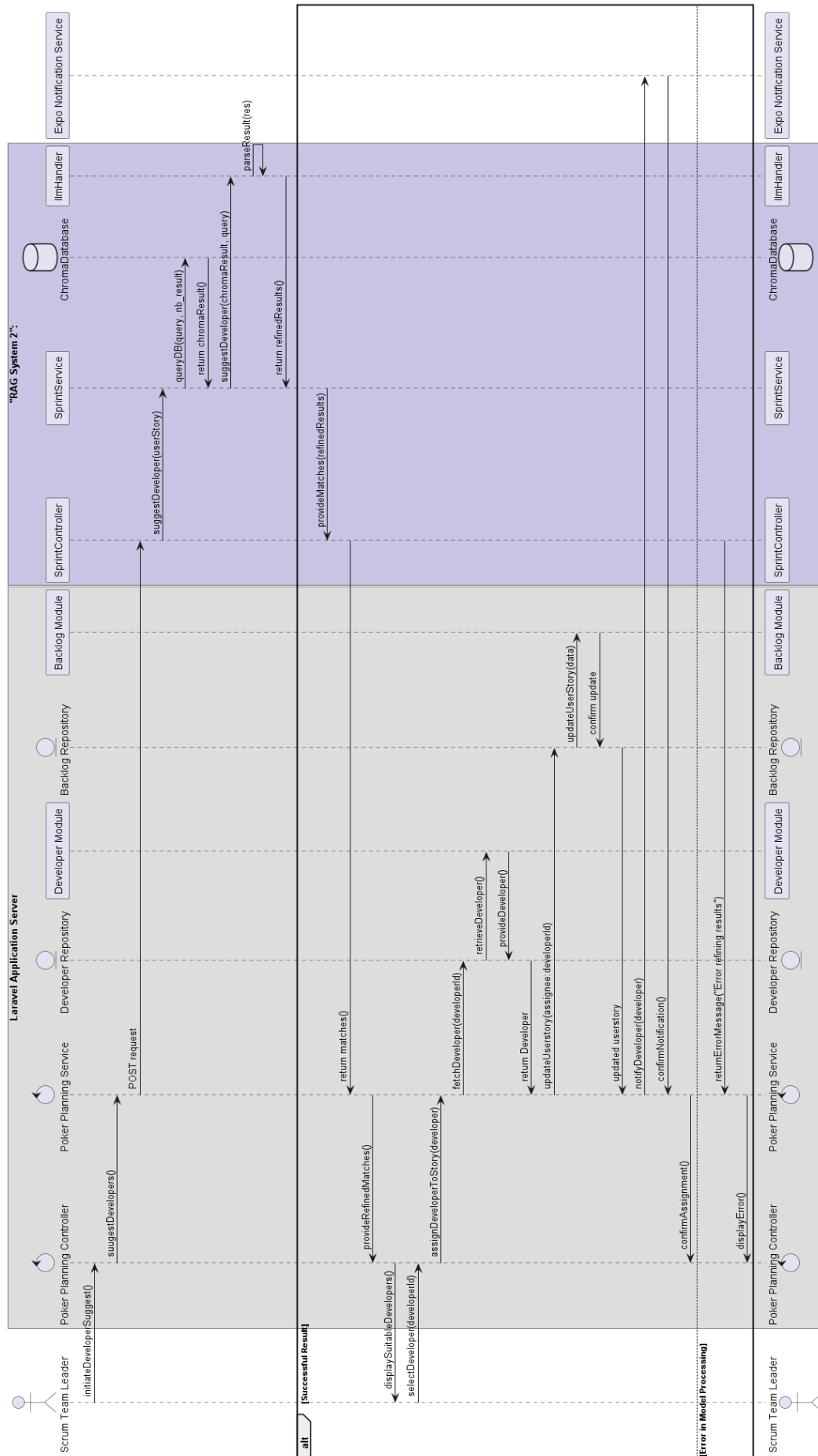


Figure 4.10: Developer Assignment Sequence Diagram

The figure 4.10 demonstrates the workflow to assign developers to specific user stories within a sprint planning, utilizing a Retrieval Augmented Generation (RAG) system 2 to enhance the decision-making process with AI-driven insights.

- The Scrum Team Leader requests the suggestion of developers for a specific user story.
- The Poker Planning Controller receives this request and forwards it to the Service, which interacts with the Controller of RAG system 2.
- The Controller of RAG system 2 communicates with the SprintService, which performs a semantic search in the ChromaDatabase to retrieve potential developer candidates. This search is based on the user story specified in the query.
- The retrieved results are passed to the llmHandler(DeveloperAssignmntGenerator class), which utilizes the relevant data along with the original query within a predefined template to generate refined suggestions for developers. These suggestions are then sent back to the SprintService.
- Based on the results, suitable developers are displayed to the Scrum Team Leader for selection.
- Once a developer is selected, their information is fetched from Developer Module, and they are assigned to the user story and user story from Backlog Module gets updated accordingly.
- The assigned developer receives a notification via the Expo Notification Service, confirming the assignment.

For more detailed scenarios of the SprintPlanning manager system, see the sequence diagrams provided in the annexe A.1.

4.3.3 Activity diagrams

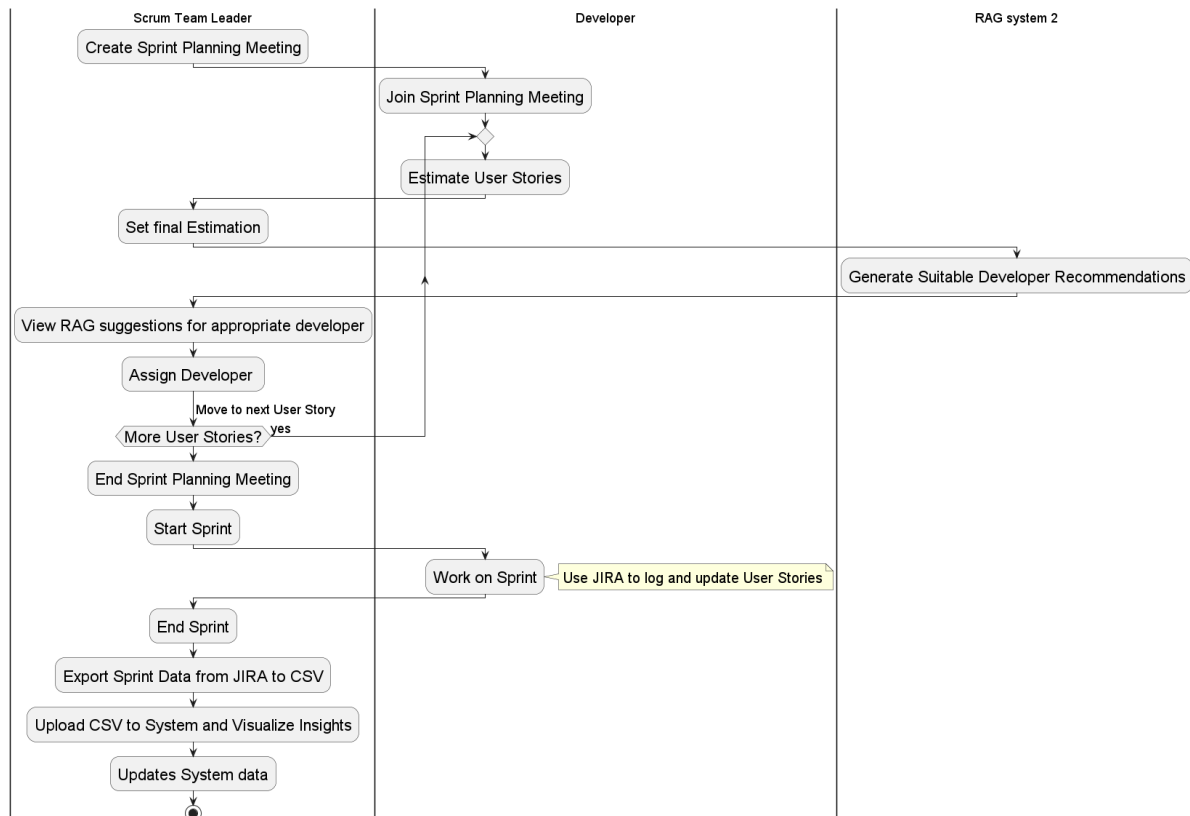


Figure 4.11: Sprint Planning flow activity Diagram

The figure 4.11 illustrates the flow within a sprint planning session, highlighting the enhanced decision-making process facilitated by the RAG system 2.

The sequence begins with the Scrum Team Leader creating a sprint planning meeting. Developers then join this session and start the iterative process of estimating user stories.

During each estimation cycle, the Scrum Team Leader consults the suggestion by the RAG system 2 to receive data-driven recommendations for the most appropriate developer for each user story. This integration ensures that assignments are optimized based on developer historical performance. Following the RAG system 2 recommendations, the Scrum Team Leader sets the final estimation, assigns developers accordingly, and moves to the next story. This loop continues until all user stories are estimated or the leader decides to conclude the meeting.

Once the sprint planning is completed, the sprint officially starts. Developers engage in the sprint, utilizing JIRA to log and update the status of their assigned user stories. The process concludes with the Scrum Team Leader ending the sprint, exporting sprint data from JIRA to a CSV file, and uploading this file to the system for data update and comprehensive insights into the sprint's performance.

4.4 Implementation

In this section we will present a selection of screenshots showing some interfaces made during this sprint.

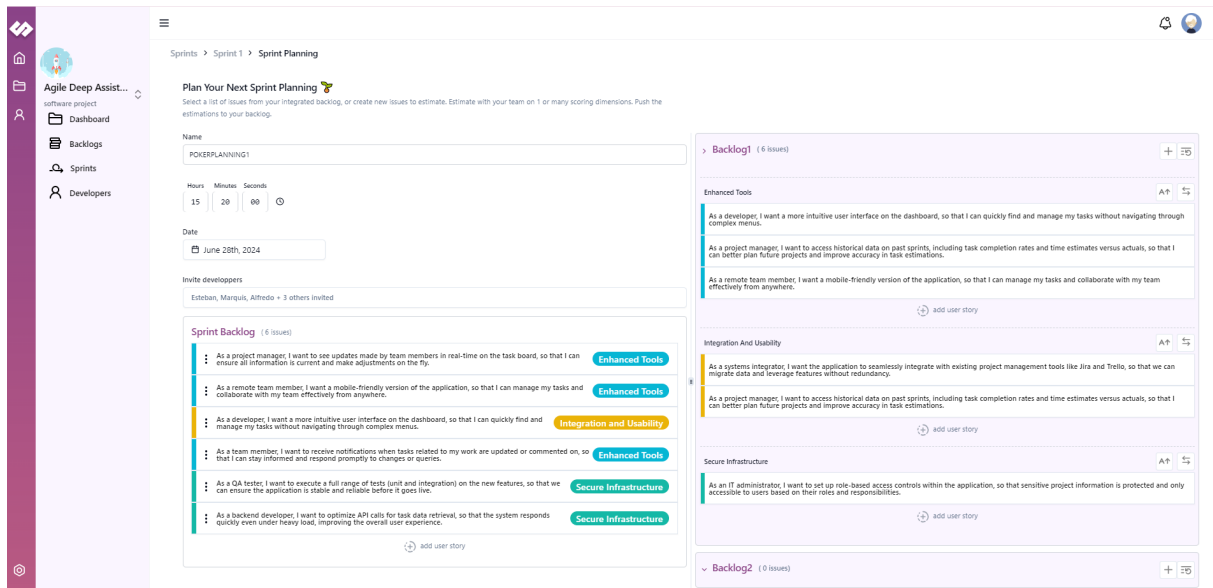


Figure 4.12: Create New Sprint Planning Interface

Figure 4.12 illustrates the interface for creating new sprint planning sessions, where the scrum leader can modify the previous sprint's backlog as needed.

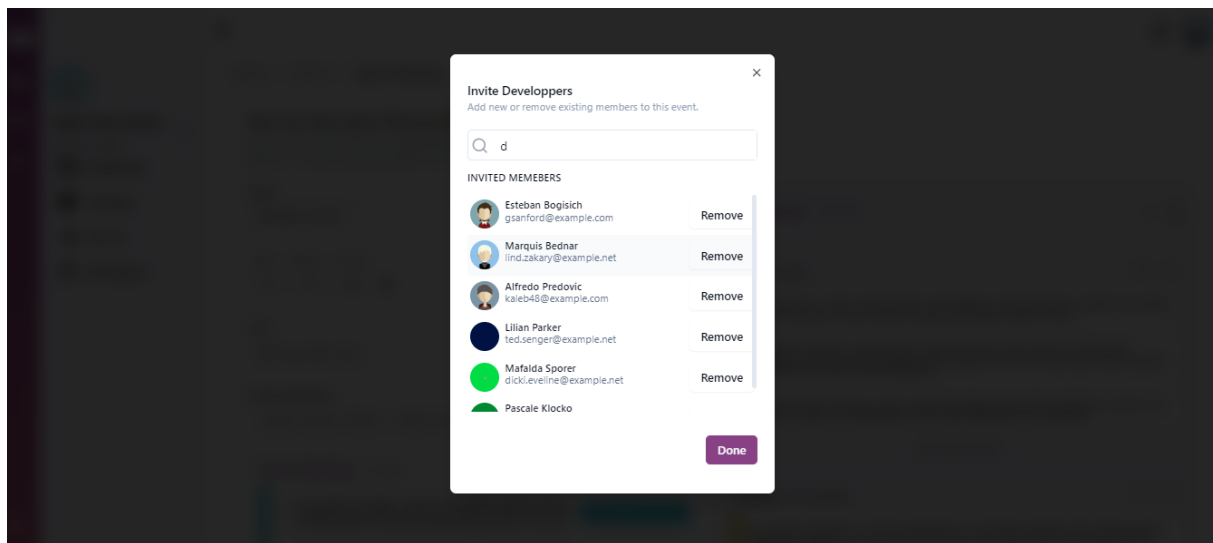


Figure 4.13: Invite developers Interface

Figure 4.13 outlines the interface for inviting developers, allowing the scrum team leader to select project developers for the sprint planning meeting.

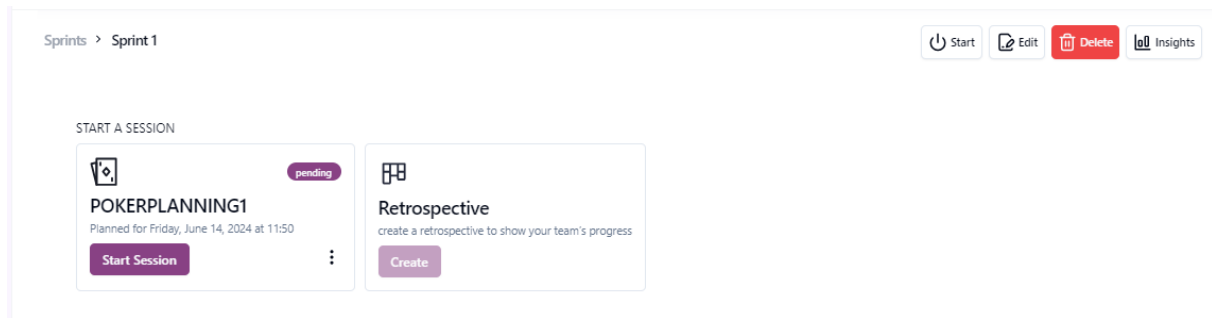


Figure 4.14: New Sprint Planning Created Interface

Figure 4.14 describes the interface displaying a newly created sprint planning card, ready to be initiated.

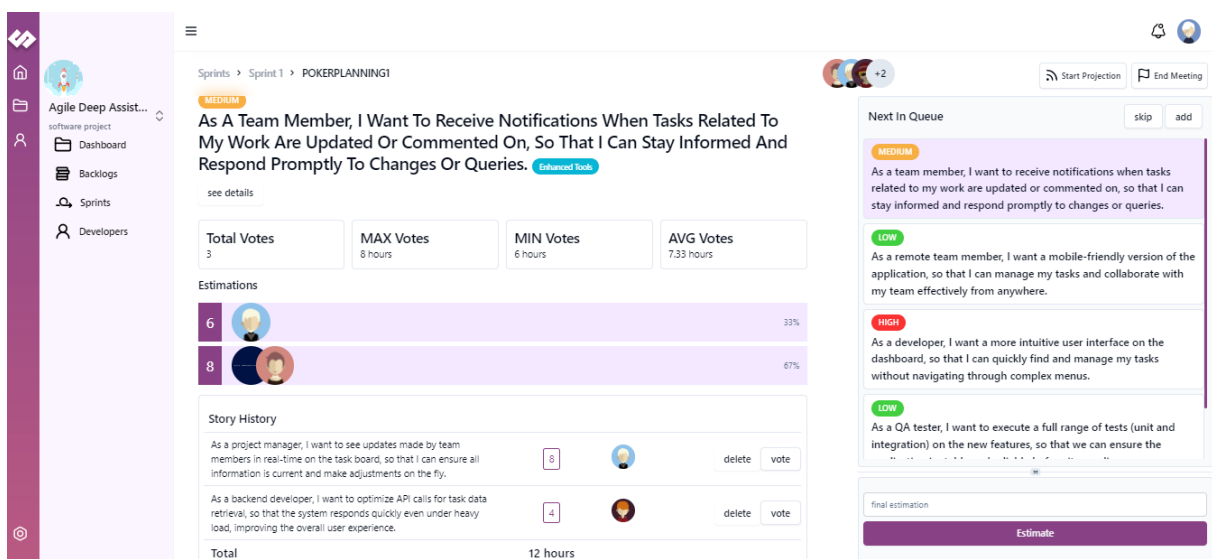


Figure 4.15: Estimation Room Interface

Figure 4.25 illustrates the Estimation Room Interface, where team members submit user story estimates through a mobile app. The scrum master can view these estimates, manage the queue of stories awaiting estimation, check historical data, consult attendee presence, and assign final values for each estimate.

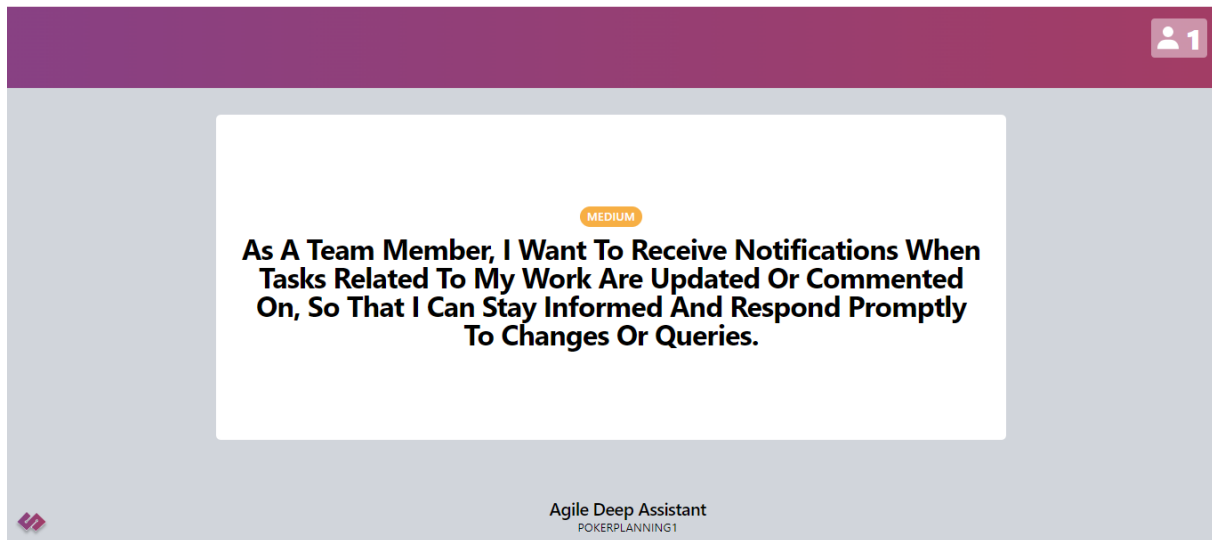


Figure 4.16: Room Projection Interface

Figure 4.16 outlines the Room Projection Interface, showing real-time details like the current user story under estimation and developer attendance, visible to all gathered participants. This interface ensures that individual estimations remain private, allowing participants to focus only on their assessment through their mobile application without being influenced by the estimations of others.

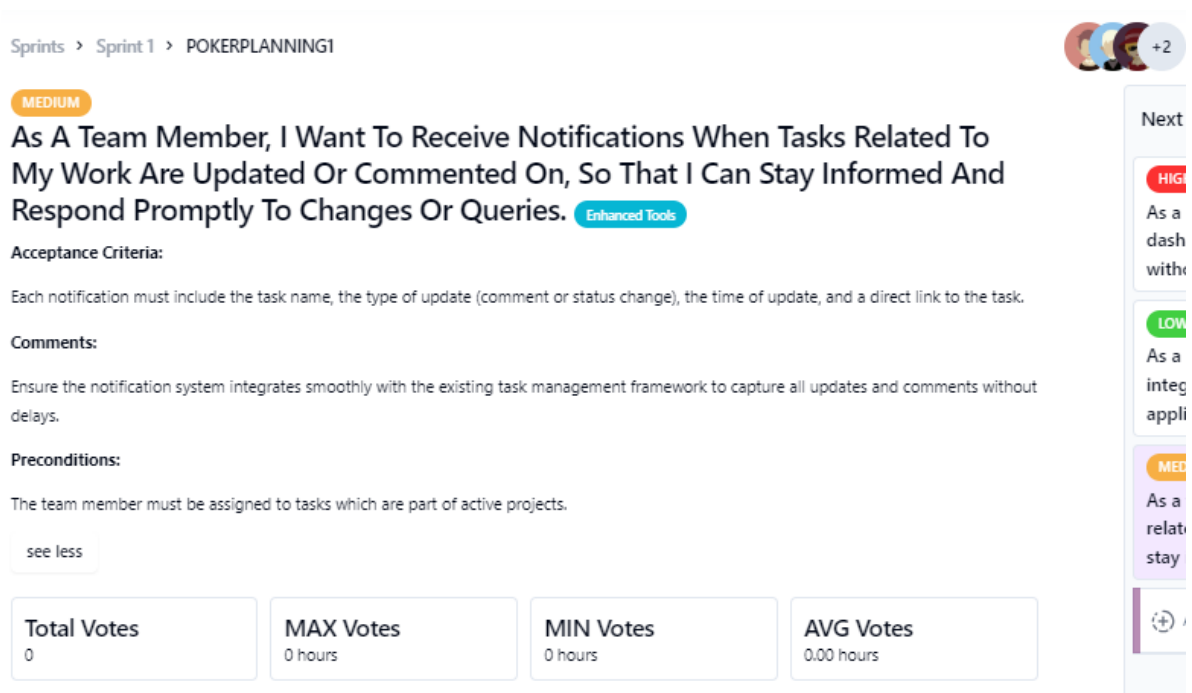


Figure 4.17: User Story Details Consultation View

Figure 4.17 describes the User Story Details Consultation View for a detailed examination of user stories.

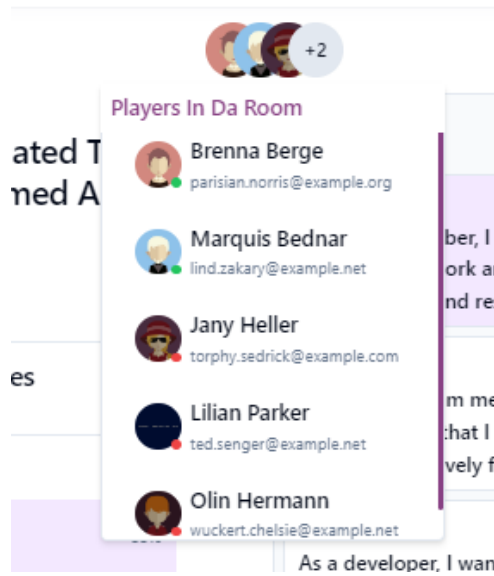


Figure 4.18: Presence Developers View

Figure 4.18 illustrates the Presence List Developers View for Scrum Team Leader, displaying developer attendance and status during a session.

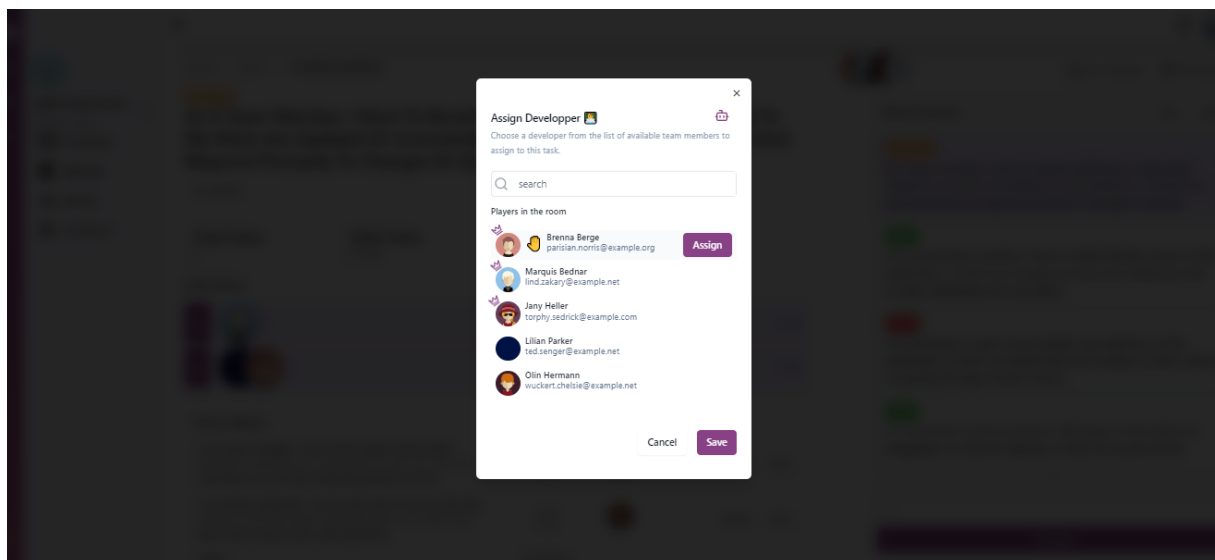


Figure 4.19: Assign Developer Interface

Figure 4.19 outlines the Assign Developer Interface, used for assigning user stories to developers. The interface lists the developers invited to the sprint planning and displays whether any of them have volunteered to take charge of the current user story. Additionally, the Scrum team leader can select the AI generative option to generate the three most suitable developers who have experience with similar tasks.

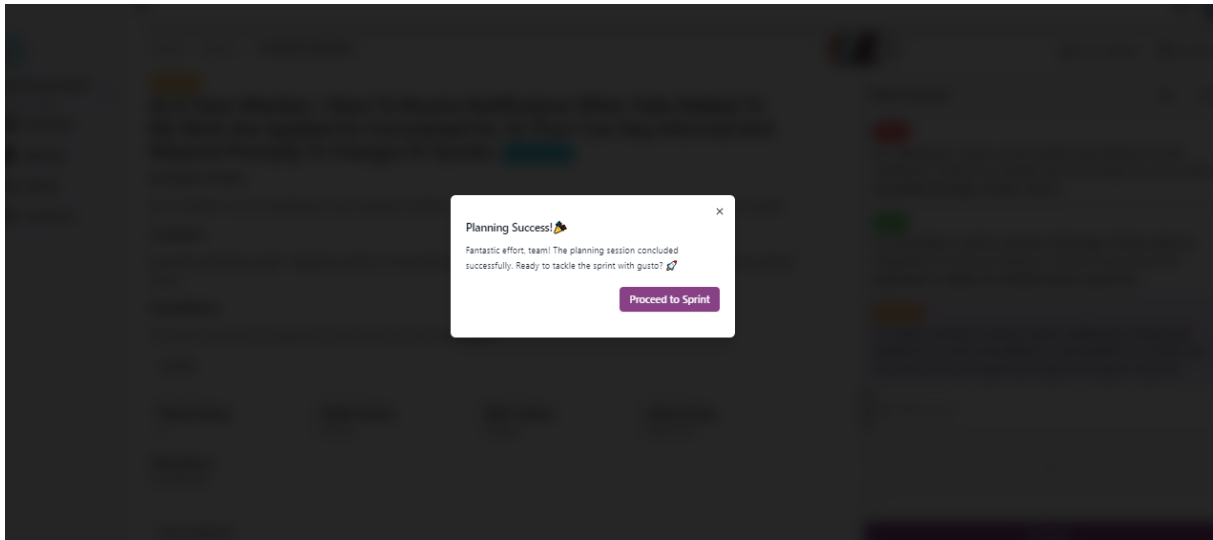


Figure 4.20: Estimation Session Completion Confirmation Interface

Figure 4.20 describes the interface confirming the completion of an estimation session.

Content	Epic Name	Max Estimation	Avg Estimation	Min Estimation	Final Estimation	Assigned Developer
As a user, I want to view my purchase history to keep track of past transactions.	epic 1	5	2.0000	-1	8	
As a user, I want to receive notifications to stay updated with important events or promotions.	Enhanced Tools	8	2.3333	-3	5	
As a user, I want to filter products by categories to find products I'm interested in more easily.	Integration and Usability	8	8.0000	8	6	
As a customer, I want to leave reviews for products to share my experience with other shoppers.	Secure Infrastructure	-	-	-	-	
As a shopper, I want to compare products to decide which one best meets my needs.	epic 6	8	8.0000	8		
As a user with disabilities, I want the user interface to be accessible so that I can use all features of the platform.	epic 5	8	8.0000	8		
story	sprint1	-	-	-	-	

Figure 4.21: Sprint Planning Estimation History Interface

Figure 4.21 illustrates the Sprint Planning Estimation History Interface, which records past estimations.

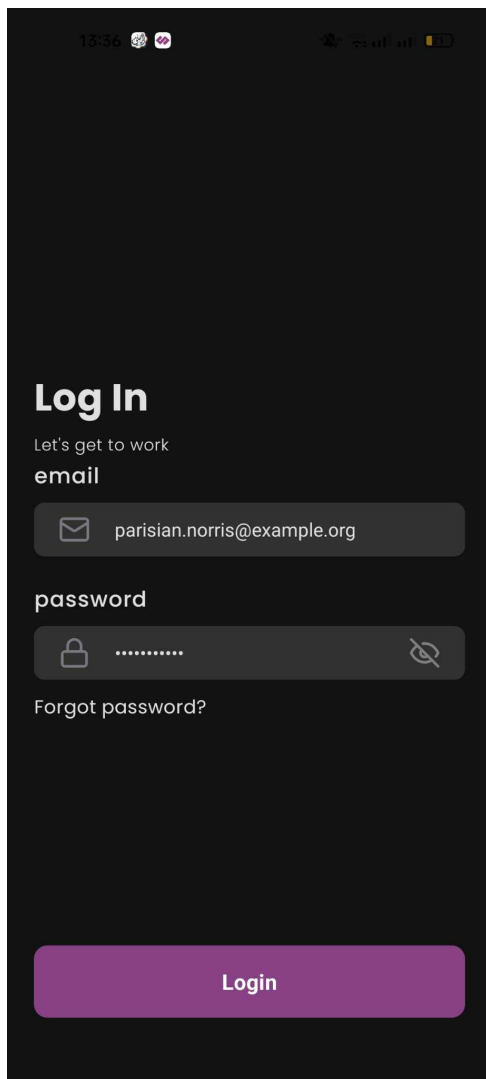


Figure 4.22: Developer Authentication Interface

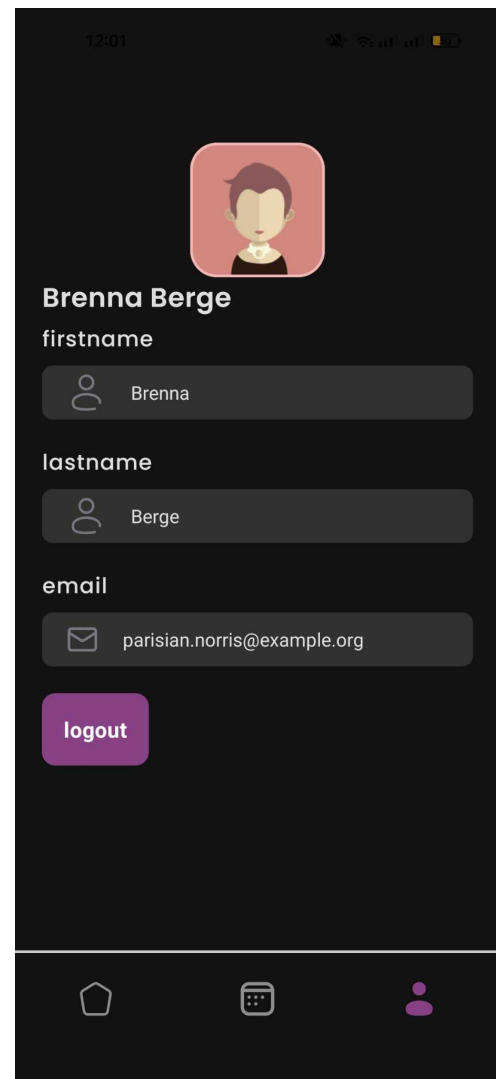


Figure 4.23: Developer Profile Interface

Figure 4.22 outlines the Developer Authentication Interface, securing access to the application and enabling participation in sprint meetings.

Figure 4.23 describes the Developer Profile Interface.

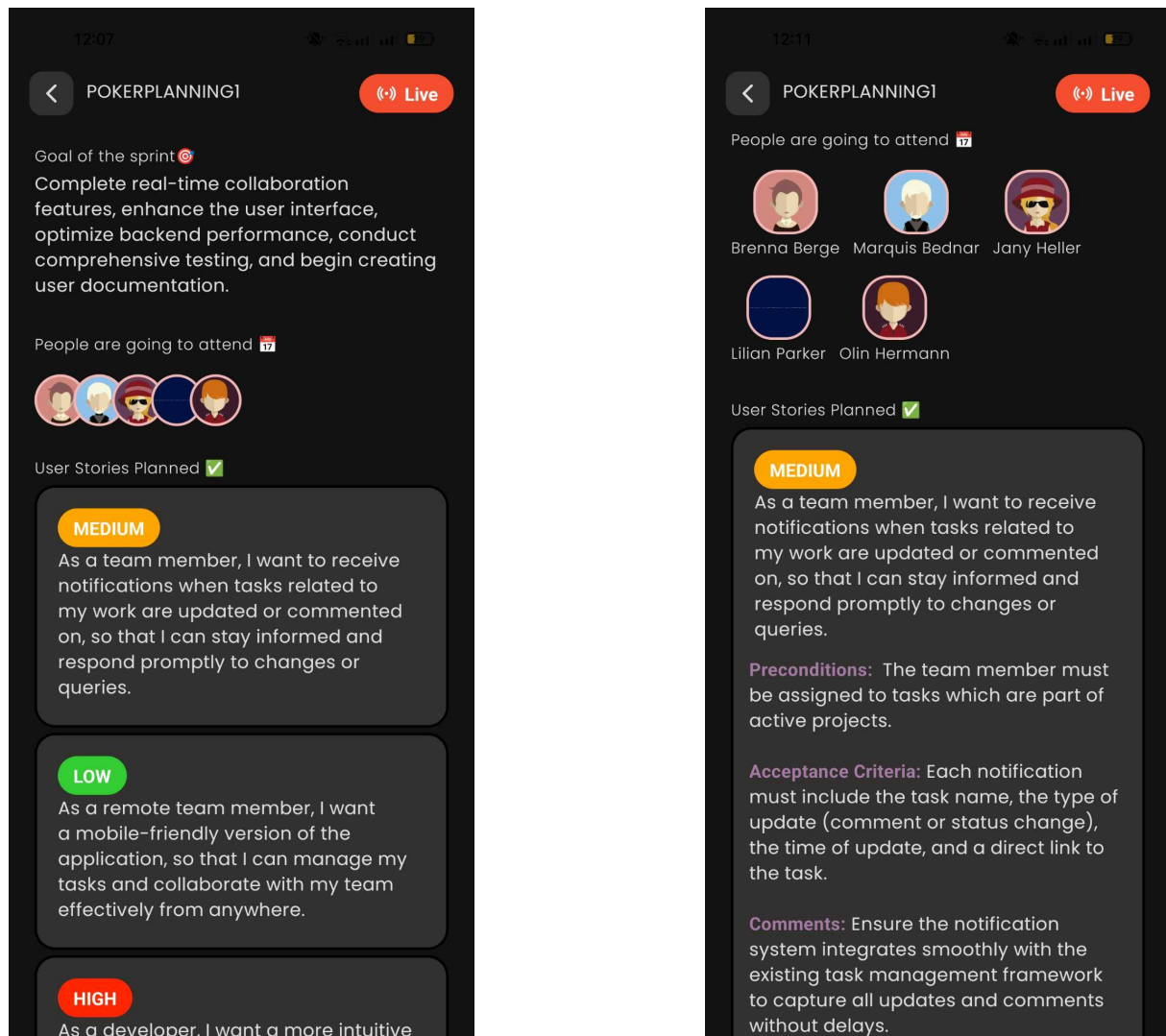


Figure 4.24: Sprint Planning Details Interface

Figure 4.24 illustrates the Sprint Planning Details Interface, detailing session elements like sprint goals, user stories planned, and participating developers.

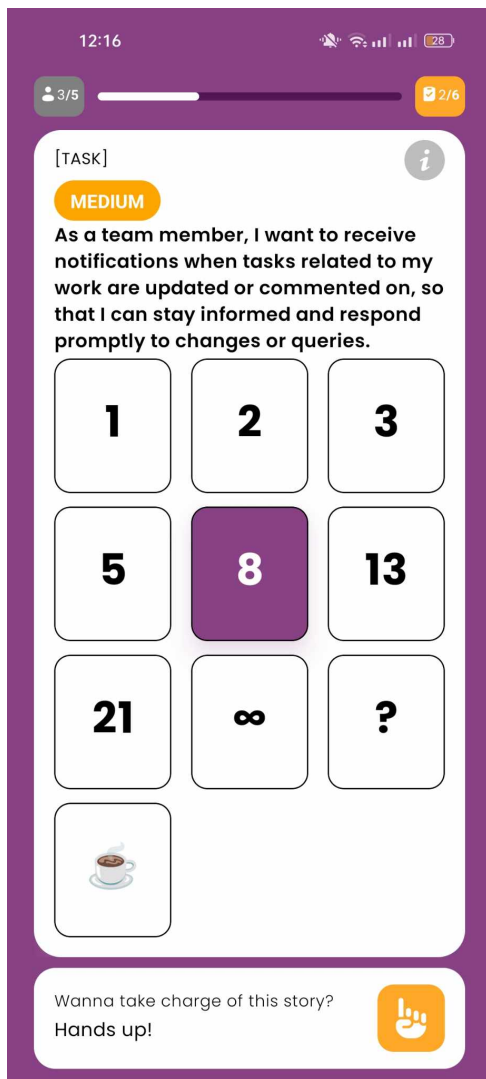


Figure 4.25: Estimation Room Interface

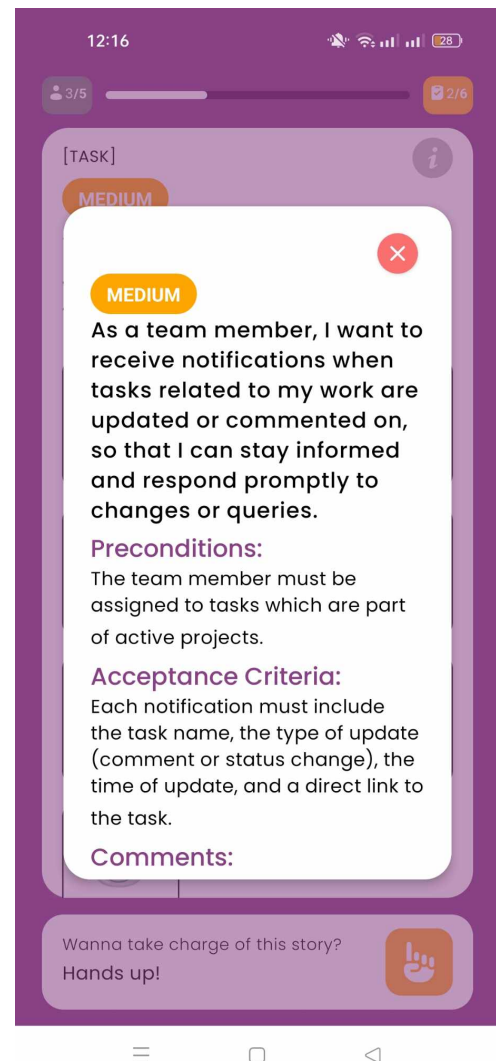


Figure 4.26: User Story Details Interface

Figure 4.25 outlines the Sprint Planning Estimation Room view for developers through their mobile application, enabling them to estimate current user stories using Poker Planning method and vote to take charge of them. The estimation includes special cards for specific scenarios:

- ? Card: Indicates a lack of enough information to make an estimate and clarification from product owner.
- Infinity Card: Used when a user story seems too large or complex to estimate with a standard number.
- Coffee Card: Signals a need for a break, suggesting that the estimator needs some time to refresh or reconsider the user story at hand.

Figure 4.26 describes the User Story Details Interface, allowing developers to thoroughly examine each user story, including preconditions, acceptance criteria, and additional comments.

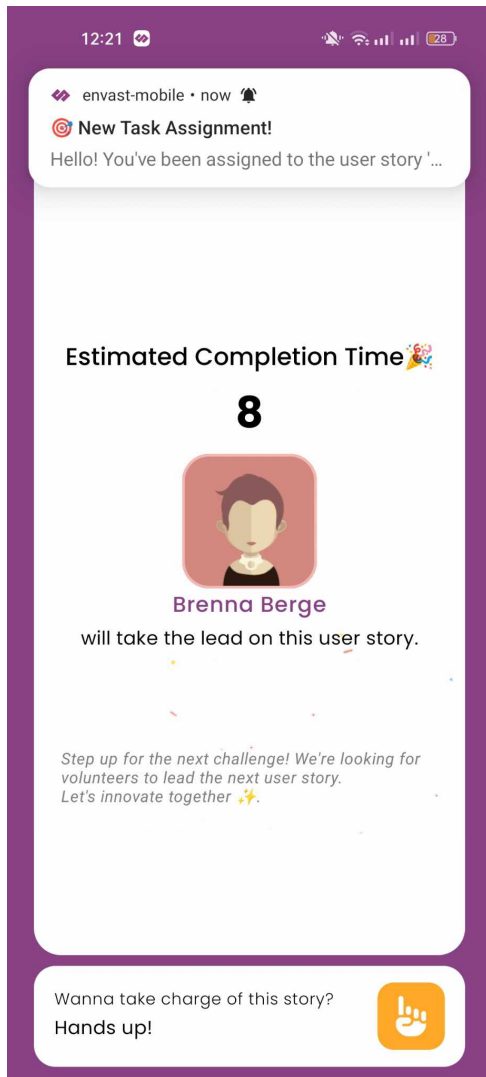


Figure 4.27: User Story Assignment complete Interface

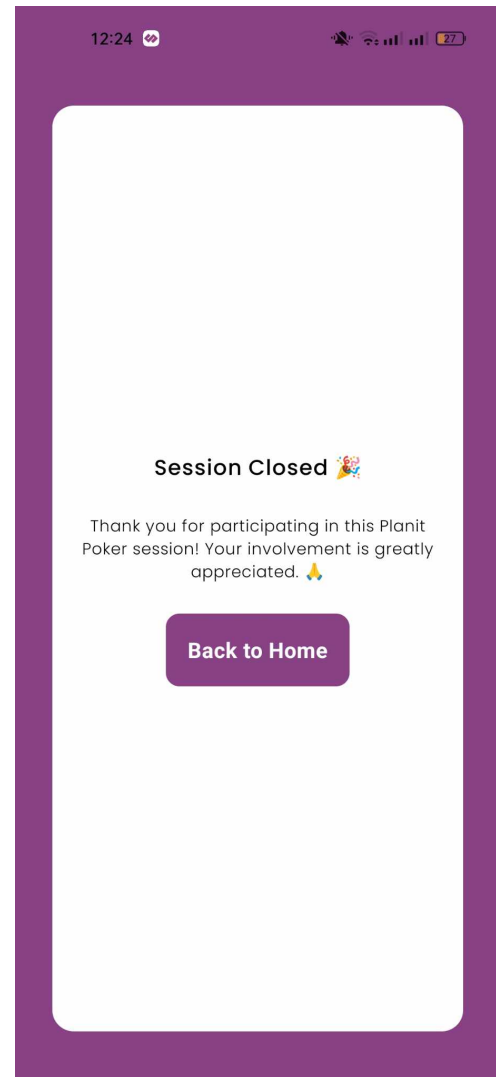


Figure 4.28: Estimation Room Closed Interface

Figure 4.27 illustrates the User Story Assignment Complete Interface, confirming the allocation of user stories to developers. Upon assignment, the designated developer receives a notification on his device.

Figure 4.28 outlines the Estimation Room Closed Interface, signaling the end of the estimation process.

Conclusion

During this chapter, we developed the second sprint by presenting use case diagrams, design, and finally interfaces of the application. The next chapter will continue with the implementation details of Sprint 3.

Chapter 5

Implementation of Sprint 3

Introduction

This chapter delves into the third sprint, which concentrates on the retrospective meetings management, from planning through to execution. First, we will present the sprint backlog. After that we present the analysis, design and implementation phase.

5.1 Sprint backlog

Table 5.1 summarizes the backlog for our third sprint:

ID	As a Scrum Master, I can (User story)	Priority	Title
1	Create a retrospective session	High	Retrospective Management
2	Send invitations to all scrum team members for the retrospective	Medium	
3	Input my ideas during the first step of the retrospective	High	
4	Generate grouped ideas suggestions	High	
5	Approve or Decline all suggested grouped topics collectively	Medium	
6	Approve or Decline grouped topic suggestion individually	High	
7	Create a topic by organize related ideas into groups using a drag-and-drop during the second step of the retrospective	Medium	
8	Vote to determine the priority of discussion topics during the third step of the retrospective	High	
9	Add actionable items during the fourth step of the retrospective	High	
10	Link specific actions to related discussion topics	Low	
11	Generate actions suggestion for discussed topics	High	
12	Approve or Decline all suggested actions collectively	Medium	
13	Approve or Decline each suggested action individually	High	
14	Export the retrospective outcomes for record keeping	Low	
ID	As a developer, I can (User story)	Priority	Title
15	Participate in a retrospective session	Medium	Developer Ideas Management in Retrospectives
16	Contribute my ideas at the start of the retrospective	High	
17	Collaboratively group ideas into topics using a drag-and-drop tool during the retrospective	Medium	

18	Vote on the prioritization of topics during the retrospective	High	
19	Access a summary of the retrospective outcomes	Low	
20	Receive an invitations notification to participate in retrospectives	Low	Notification Management
21	Consult notifications related to retrospective sessions	Low	Calendar Consultation
22	Access the calendar to check schedules for upcoming retrospectives and sprint planning meetings	Low	

Table 5.1: Sprint Backlog 3

5.2 Functional specifications

5.2.1 Sprint 3 Use Case Diagram

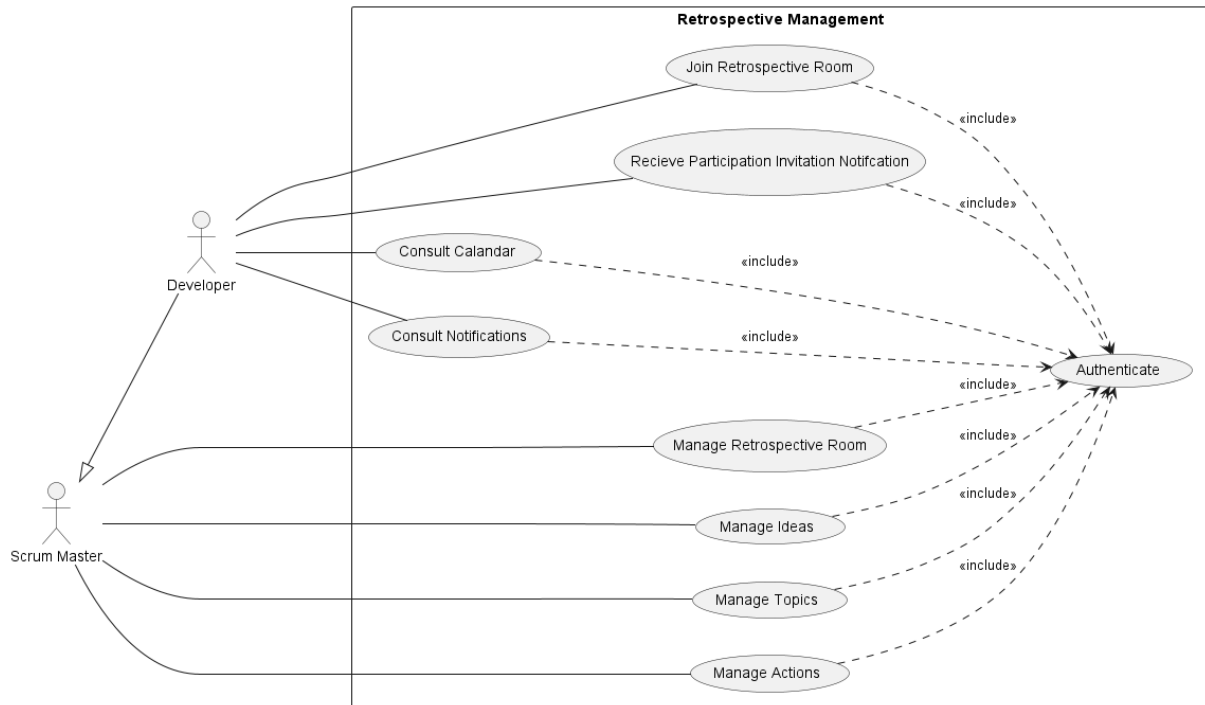


Figure 5.1: Use Case Diagram for sprin3

Figure 5.1 outlines the roles and interactions within the "Retrospective Management" system through a use case diagram, highlighting functionalities performed by the Scrum Master and Developers. The Scrum Master is responsible for managing the retrospective room, organizing ideas, topics, and actions.

Developers participate by joining the retrospective room after receiving participation invitations, sharing and managing their ideas, voting on topics, and staying informed through calendar updates and notifications.

5.2.2 Refinement of third sprint use cases

Specification of the "Manage Retrospective Room" use case

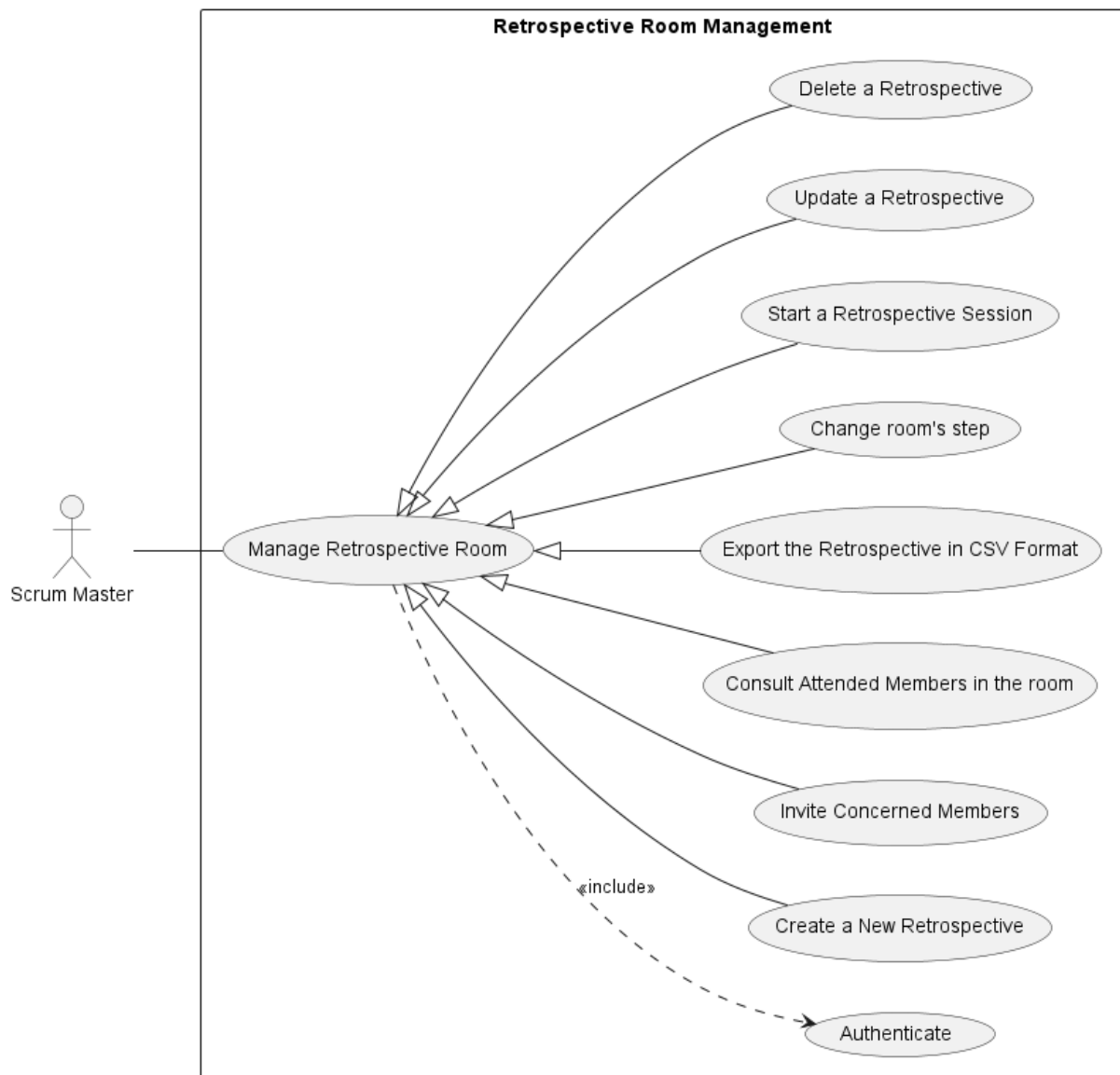


Figure 5.2: Use Case Diagram for Manage Retrospective Room

The figure 5.2 depicts the "Retrospective Room Management" system, managed by the Scrum Master, including creating, inviting members, consulting attendance and exporting retrospectives.

Title	Create New Retrospective Room
Actor	Scrum Master
Precondition	The Scrum Master is authenticated and sprint planning has been completed.
Postcondition	A new retrospective session is created.
Main Scenario	1. Scrum Master initiates retrospective creation through the UI. 2. The system retrieves sprint details. 3. The system gathers templates and developer attendance from planning. 4. Scrum Master submits detailed retrospective data. 5. The system validates and checks for existing retrospective sessions. 6. If no existing session, the system stores the new retrospective details and updates attendance records. 7. The system confirms the creation of the retrospective to the Scrum Master and sends notifications to the invited members.
Exception Scenario	- If poker planning is not completed, "Poker planning not completed" is shown. - If a retrospective already exists, "Retrospective session already exists" is displayed.

Table 5.2: Use Case Specification for "Create New Retrospective room"

Specification of the "Manage Ideas" use case

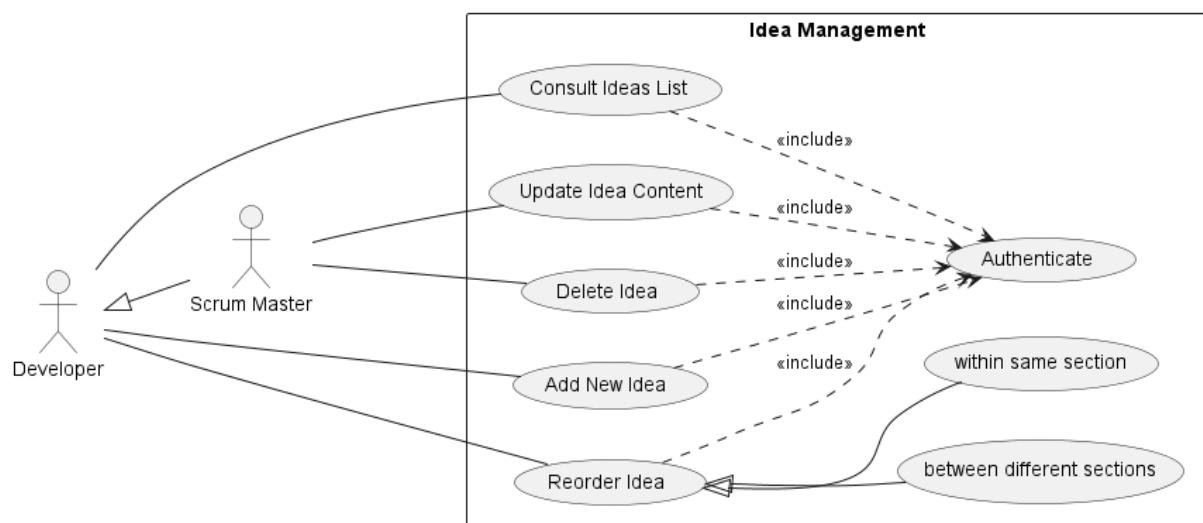


Figure 5.3: Use Case Diagram for Manage Ideas

The figure 5.3 illustrates the "Idea Management" system, showcasing interactions between the Scrum Master and Developers. Developers are primarily responsible for adding new ideas, reordering them within or between different sections, and consulting the ideas list. The Scrum Master has the ability to update or delete ideas.

Title	Create Idea
Actor	Developer / Scrum Master
Precondition	The Developer/Scrum Master is authenticated.
Postcondition	An idea is created and linked with a new topic.
Main Scenario	1. Developer or Scrum Master submits an idea. 2. The system validates the inputs provided by the Developer or Scrum Master for the idea. 3. The system creates the idea, fetches developer data from the User Module, and associates it with the new created Idea. 4. The system creates a new topic and attaches the new idea to it. 5. The updated topic details, including the idea and its author, are displayed by the system. 6. Finally, the system updates the Scrum Master's dashboard with the new idea.
Exception Scenario	- If the inputs are invalid, the system displays an error message, "Invalid inputs for idea creation." - If the idea cannot be created, an error message, "Idea handling error," is displayed.

Table 5.3: Use Case Specification for "Create Idea"

Specification of the "Manage Topics" use case

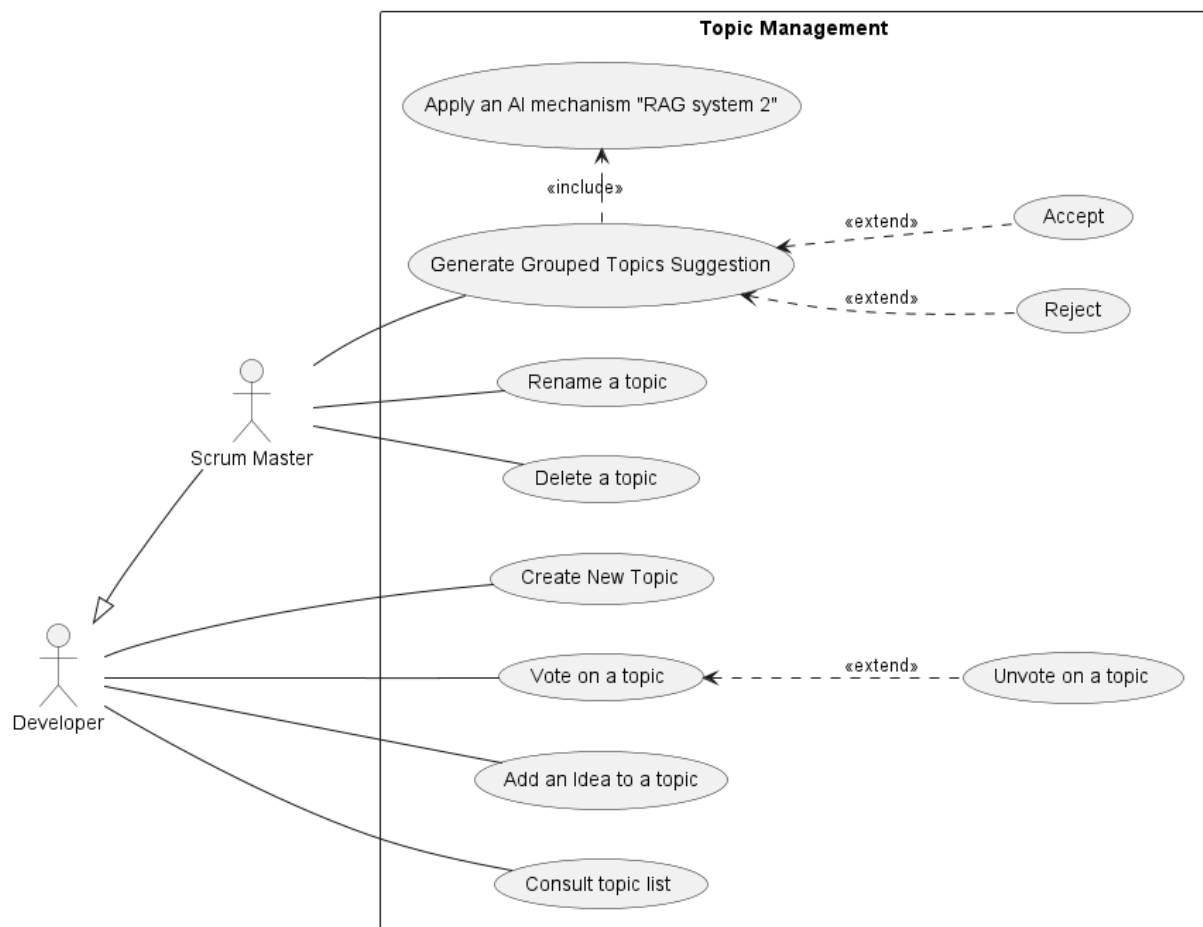


Figure 5.4: Use Case Diagram for Manage Topics

The figure 5.4 outlines the "Topic Management" system, where both Scrum Masters and Developers are engaged in managing and interacting with topics. In this system, both roles can create new topics, vote on topics, add ideas to topics, and consult the topic list. Exclusive to the Scrum Master's responsibilities are the abilities to rename and delete topics, as well as to generate and act on suggested grouped ideas(topics) from the RAG system. This integration allows the Scrum Master to accept or reject suggested groupings.

Title	Vote on Topic
Actor	Developer
Precondition	Developer is authenticated and has access to vote on the topic.
Postcondition	Developer's vote is registered or updated on the topic.
Main Scenario	1. Developer initiates the vote on a topic through the UI. 2. The system validates the developer's vote based on existing votes and limits. 3. If not voted previously, the system creates a new vote record. 4. If already voted, the system updates the existing vote based on limits. 5. The system updates the total votes for the topic. 6. The system confirms the successful voting or updating of the vote to the developer.
Exception Scenario	- If the voting limits are reached, the system displays "Vote limit reached".

Table 5.4: Use Case Specification for "Vote on Topic"

Specification of the "Manage Actions" use case

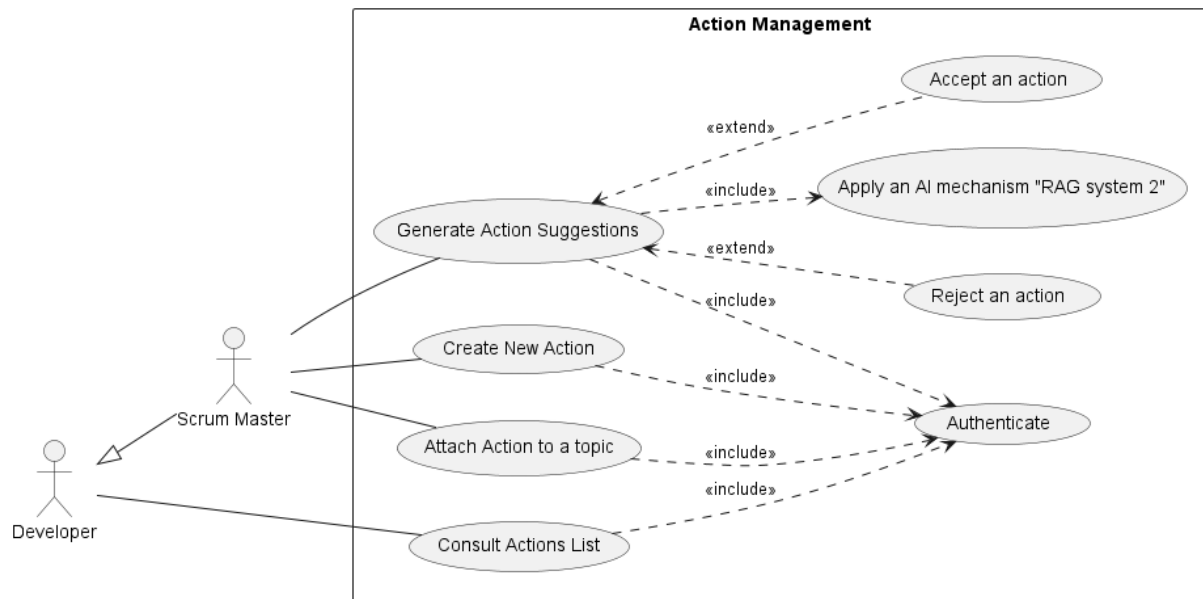


Figure 5.5: Use Case Diagram for Manage Actions

The figure 5.5 illustrates the interaction of Scrum Masters and Developers with "Action Management" system, facilitated by the RAG system 2. Both roles require authentication for their activities, which enhances security.

Scrum Masters have the capabilities to create new actions, attach actions to specific topics, and consult a list of suggested actions generated by the LLM within our RAG system 2. Developers primarily engage in consulting the final actions list.

The interaction with the RAG system 2 includes the ability to accept or reject these suggested actions, allowing the Scrum Master to refine and finalize the action items associated with topics based on system recommendations.

Title	Generate actions suggestions in Retrospective
Actor	Scrum Master
Precondition	The Scrum Master is authenticated and has access to the retrospective room.
Postcondition	Topics are displayed sorted by vote count, ideas and suggested actions are linked to topics.
Main Scenario	1. Scrum Master navigates step four of the retrospective screen. 2. The system loads all topics from the repository. 3. The system sorts the topics by their vote count in descending order. 4. For each sorted topic, the system loads associated ideas from the Idea Repository and display the data back to scrum master. 5. The Scrum Master can choose the option of automatically generating actions through the "AI Assistant" button. 6. The system uses the Llama model from RAG system 2 to analyze discussion topics ideas and generate actionable suggestions for each topic. 7. The system displays the sorted topics, their ideas, and suggested actions to the Scrum Master. 8. Scrum Master reviews each actionable suggestion: 8.1 If accepted, the system creates a new action and adds it to the topic. 8.2 If rejected, the system removes the suggestion from the UI.
Exception Scenario	- If no topics are available, the system displays "No topics available." - If there is a failure in loading topics or ideas, the system shows "Error in loading data." - If the Llama model fails to generate suggestions, the system alerts "Error processing discussion points."

Table 5.5: Use Case Specification for "Generate actions suggestions in Retrospective"

5.3 System Design

5.3.1 Class Diagrams

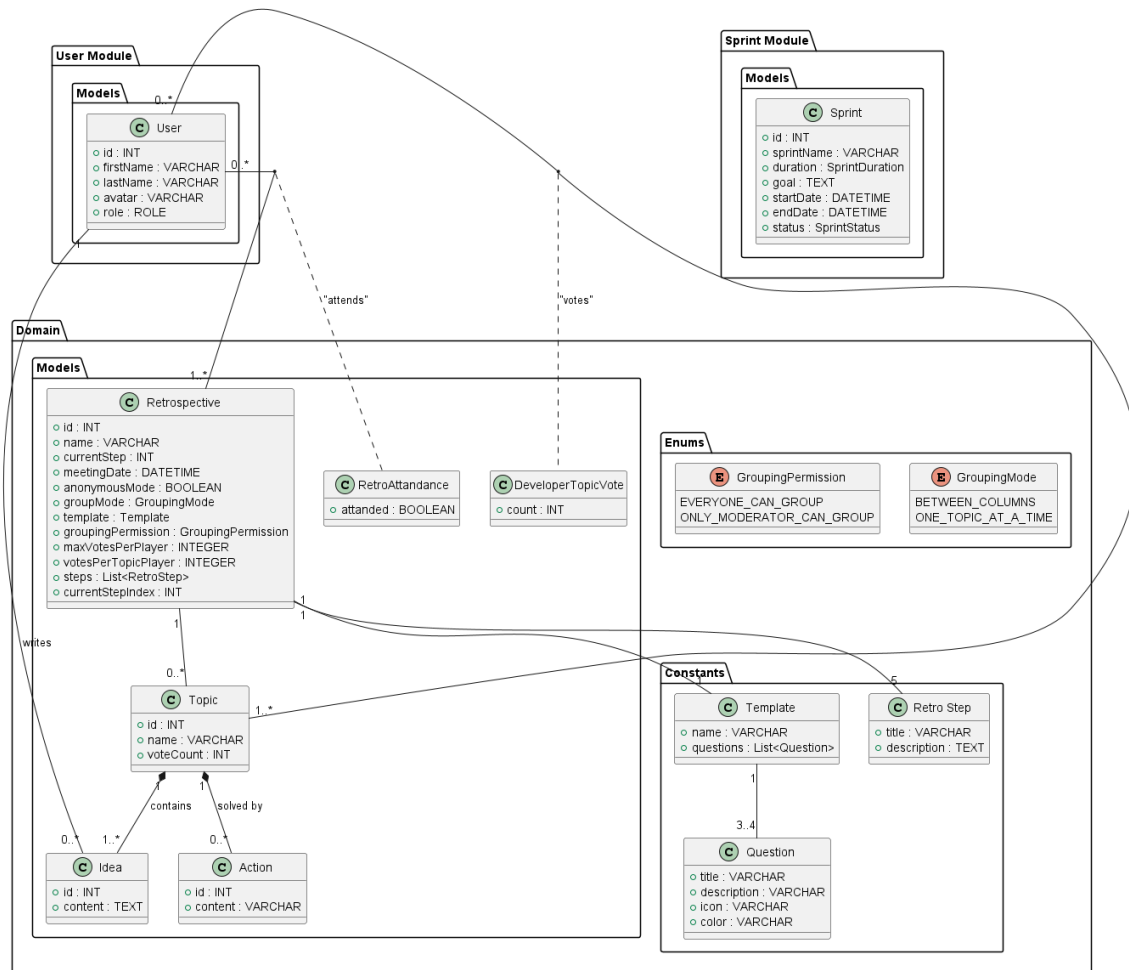


Figure 5.6: Sprint 3 Models Packages Interaction Class Diagram

The diagram 5.6 illustrates the interactions of the Retrospective Sprint models with other models from system modules.

Users with the DEVELOPER role participate in retrospective sessions, where they express their thoughts by writing ideas, grouping them to form topics, and voting to prioritize those topics, with the Scrum Master serving as the meeting moderator.

During the setup, the Scrum Master selects from predefined templates which guide the retrospective through five structured steps: BRAINSTORM, GROUP, VOTE, ACTION, and CLOSE.

Each template contains between three to five questions, each focusing on specific aspects of the sprint. Templates, questions and steps are static objects, and this design adheres to DDD principles, as value objects are identified by attribute values rather than by IDs.

Ideas generated in the sessions are linked to topics that highlight specific sprint challenges.

Topics can expand as more ideas are added, facilitating a comprehensive review within the retrospective session. Actions are then proposed as solutions to these identified problem topics.

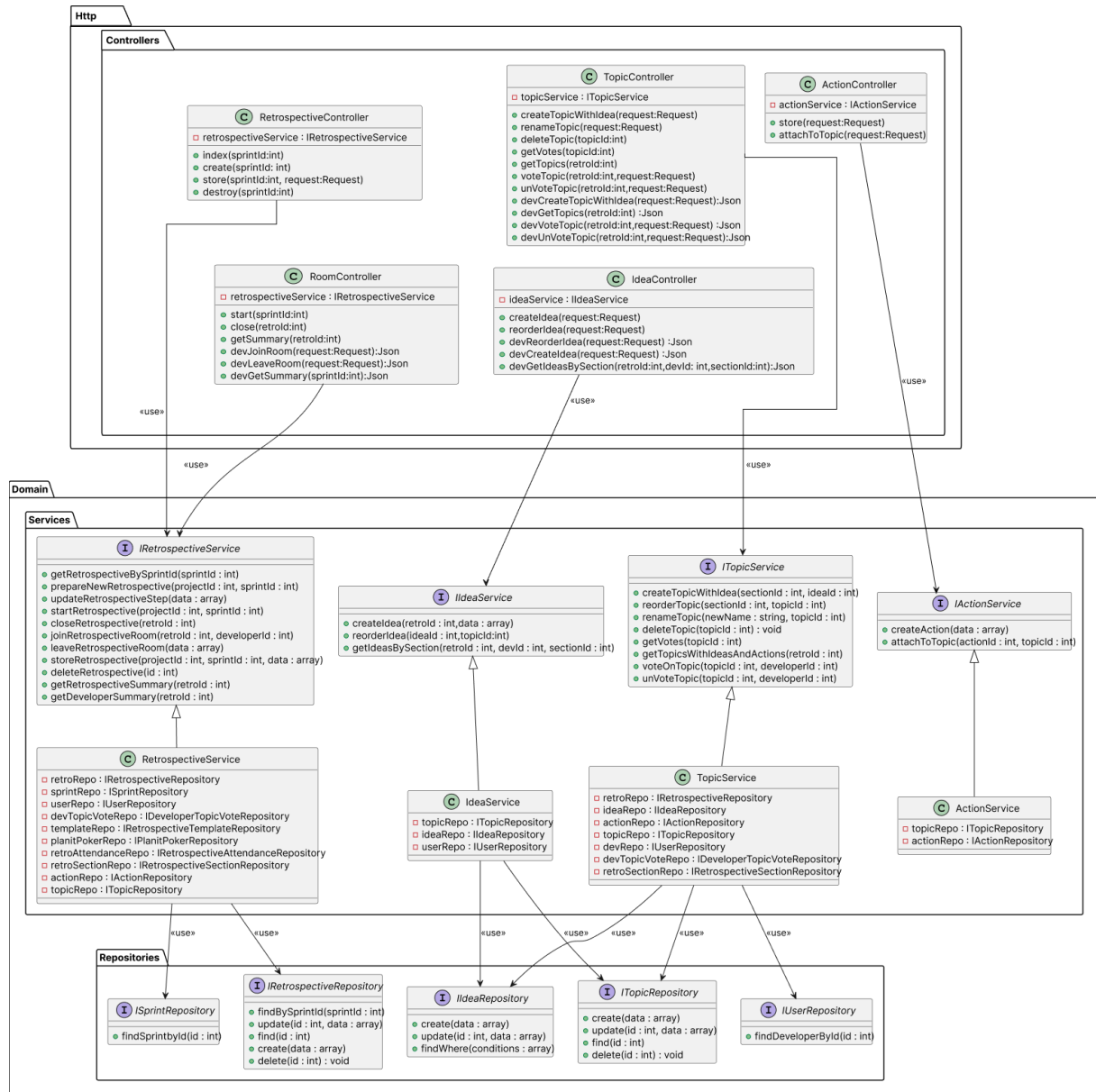


Figure 5.7: Sprint 3 :Controllers-Services-Repositories Interaction Class Diagram

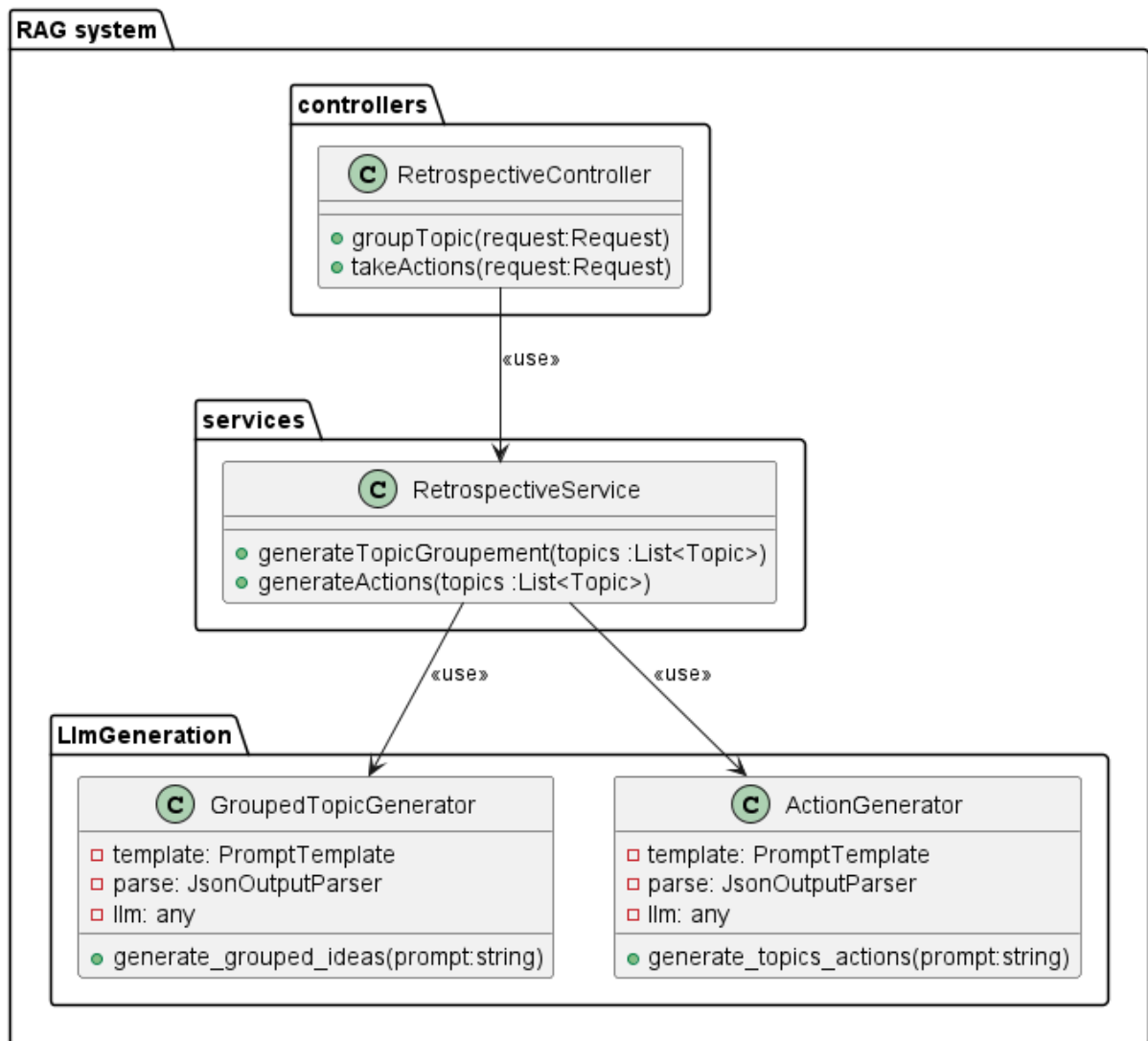


Figure 5.8: Retrospective RAG system Class Diagram

Figure 5.8 illustrates the architecture of the "RAG" system 2, designed to generate retrospective suggestions. This system features a main controller, the `RetrospectiveController`, which processes requests for creating grouped topics and actionable items, submitted directly from the Scrum Master's dashboard.

Consequently, there is no need to perform similarity searches within a vector database, as the necessary data is already provided by the Scrum Master. These requests are handled by the `RetrospectiveService`, which leverages specialized generators equipped with unique `PromptTemplate` to format the prompt with the submitted topics and `JsonOutputParser` to verify and interpret the output from the language model provided by the `LlmGeneration` package.

5.3.2 Sequence diagrams

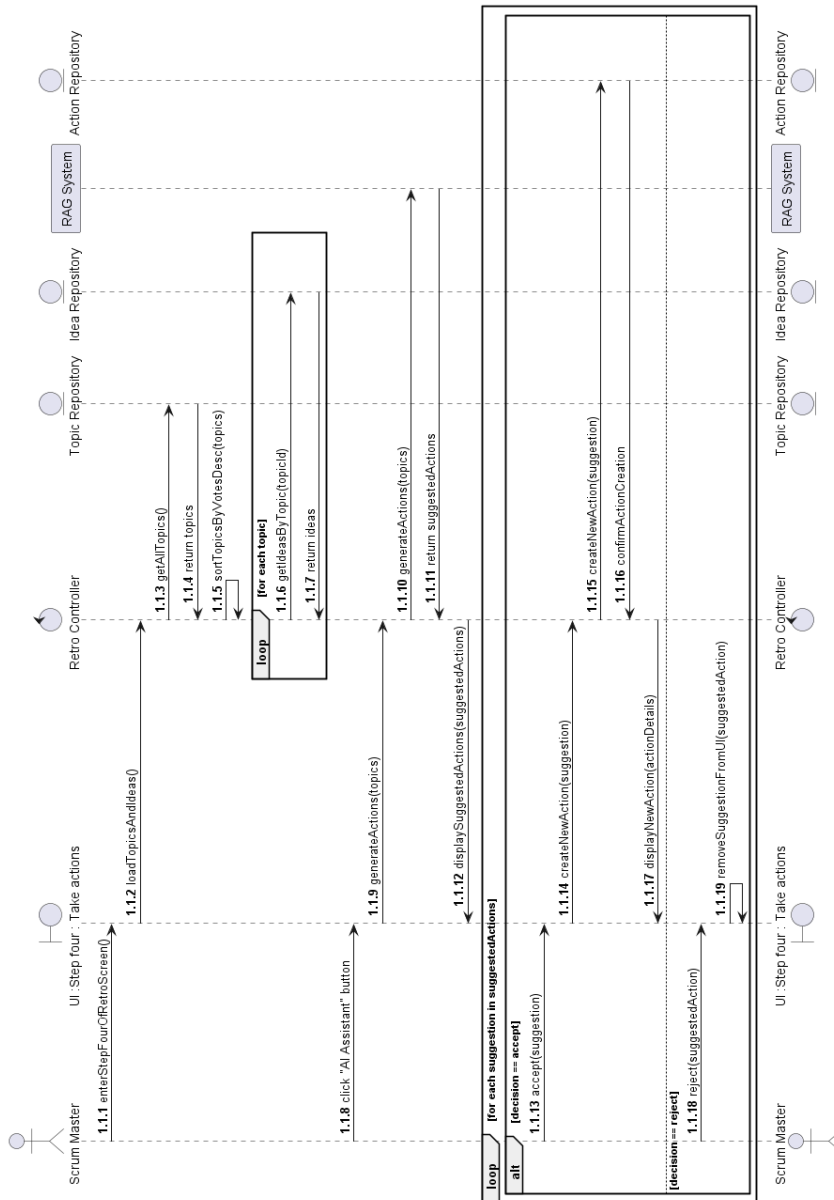


Figure 5.9: Generate actions suggestions Sequence Diagram

Figure 5.9 outlines the interaction process for generating suggestions in the retrospective session step four, integrating external analysis via an LLM within our RAG system 2 that provides actionable suggestions, each suggestion is presented to the Scrum Master through the UI. the Scrum Master has the ability later on to accept or reject actionable suggestions. If a suggestion is accepted, it creates a new action; if rejected, it disappears from the UI.

For more detailed scenarios of the retrospective manager system, see the sequence diagrams provided in the annexe A.3.

5.3.3 Activity diagrams

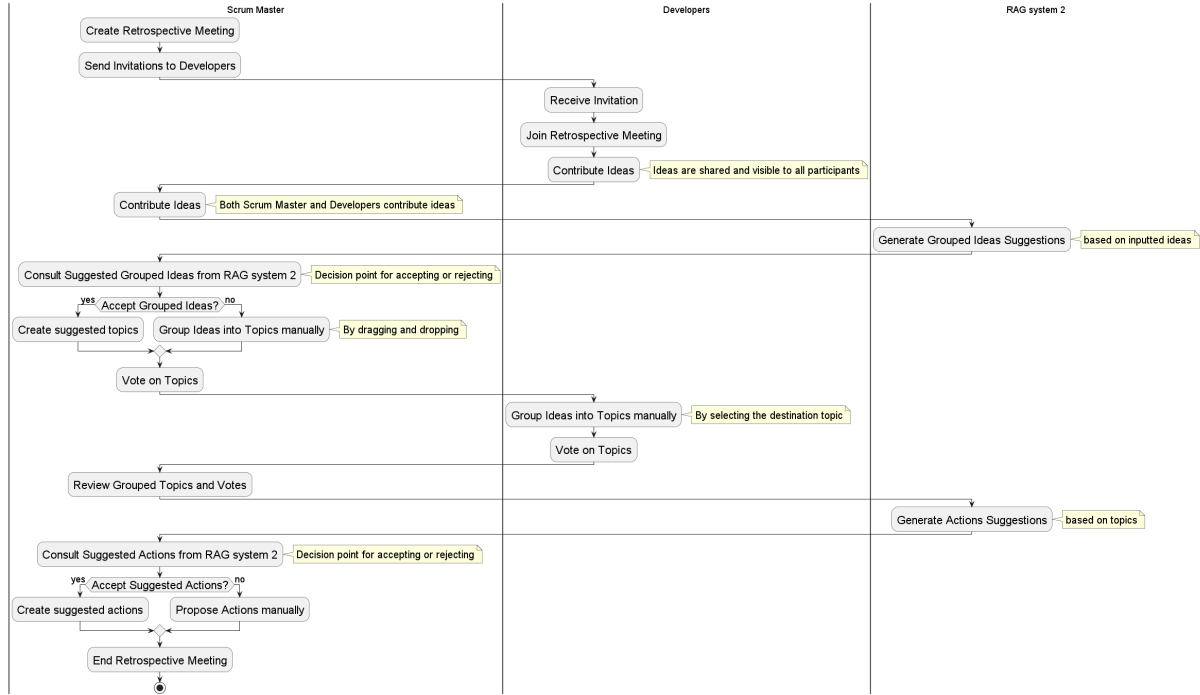


Figure 5.10: Retrospective flow activity Diagram

The Figure 5.10 outlines the retrospective meeting flow managed by the Scrum Master, involving both the Scrum Master and Developers.

The sequence begins with the Scrum Master creating and sending invitations for a retrospective meeting, which Developers join upon receipt. Both actors contribute ideas, which are visible to all participants, fostering a collaborative environment.

The Scrum Master then consults the suggested groupings of ideas generated by LLM model within RAG system 2. Depending on whether these suggestions are accepted or not, topics are either created from these suggestions or manually grouped by dragging and dropping. Subsequently, both the Scrum Master and Developers vote on these topics.

Further, the Scrum Master consults the suggested actions based on the voted topics, facing another decision point. Actions are either created directly from these suggestions or proposed manually if the suggestions are rejected.

The meeting concludes with a review of the grouped topics and votes, finalizing the actions to be taken, and then officially ending the retrospective session.

5.4 Implementation

In this section, we will present a series of screenshots that illustrate various interfaces developed during this sprint.

Sprints > Sprint 1 > Retrospective

Plan Your Next Retrospective

Create an online retrospective board that will get your team talking

General

Name:

Hours: Minutes: Seconds:

Date:

Template:

Tags: ContinuousImprovement TeamDynamics TechDebtResolution

Invited Developers

- ☒ Brenna Berge [parisan.norris@example.org](#)
- ☒ Marquis Bednar [lind.zakary@example.net](#)
- ☒ Jany Heller [torphy.sedrick@example.com](#)
- ☒ Lilian Parker [ted.senger@example.net](#)
- ☒ Olin Hermann [wuckert.chesie@example.net](#)

Preferences

Grouping permissions: ☐ Everyone can group ☒ Only the Scrum Team Leader can group

Max votes per player: ☒ Unlimited

Max votes per topic: ☒ Unlimited

Figure 5.11: Create New Retrospective Interface

Figure 5.11 illustrates the Create New Retrospective Interface, where the scrum master can enter the session name, set the time, choose a template (example: 4L, Sailboat, Sad, Glad, Mad), configure session preferences like grouping permissions and voting limits, and confirm attendance of all team members for this sprint.

Sprints > Sprint 1 > RETRO#SPRINT1

+1 Anonymous Ideas

Brainstorm Session
Generate a wide range of ideas and solutions

Liked
What did we like about the last sprint?

Learned
What did we learn from the last sprint?

Lacked
What do we think was missing from the last sprint?

Longed For
What could we have done better in the last sprint?

Liked Feedback:

- Appreciation for the clear and consistent daily stand-up meetings
- The team's willingness to help each other when blockers arise.
- The integration of new project management software that helped in task tracking.
- The effectiveness of new automation framework

Learned Feedback:

- Understanding the benefits of a defined sprint review process
- New communication tools that facilitated better remote collaboration.
- New communication skills
- Comprehensive technique in writing test cases

Lacked Feedback:

- Missing a formal method for mid-sprint check-ins to adjust priorities
- Enough structured time for brainstorming and creative thinking
- Trained on advanced feature

Longed For Feedback:

- A more robust system for tracking and visualizing sprint progress

Figure 5.12: Retrospective Room Step1 : Brainstorm Interface

Figure 5.12 outlines the Retrospective Room Step1: Brainstorm Interface, where the scrum master can manage and consult team members' ideas and feedback according to the chosen template.

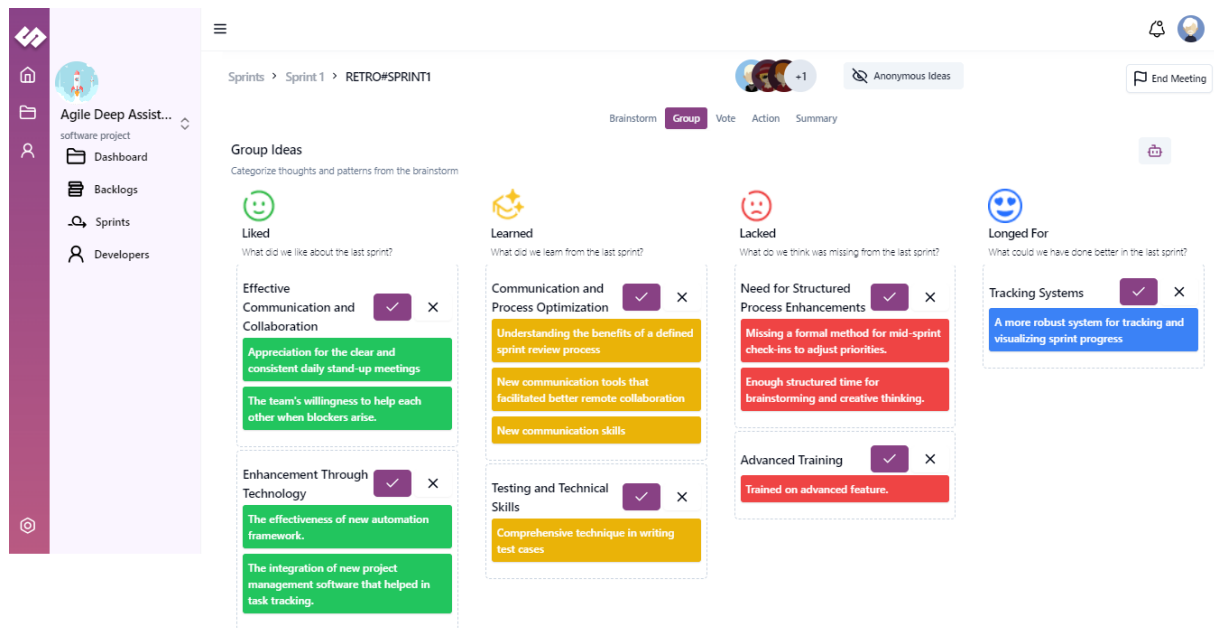


Figure 5.13: Retrospective Room Step2 : Group Interface

Figure 5.13 illustrates the Retrospective Room Step2: Group Interface, where feedback and ideas can be manually grouped or grouped using the AI suggestion option from our RAG system.

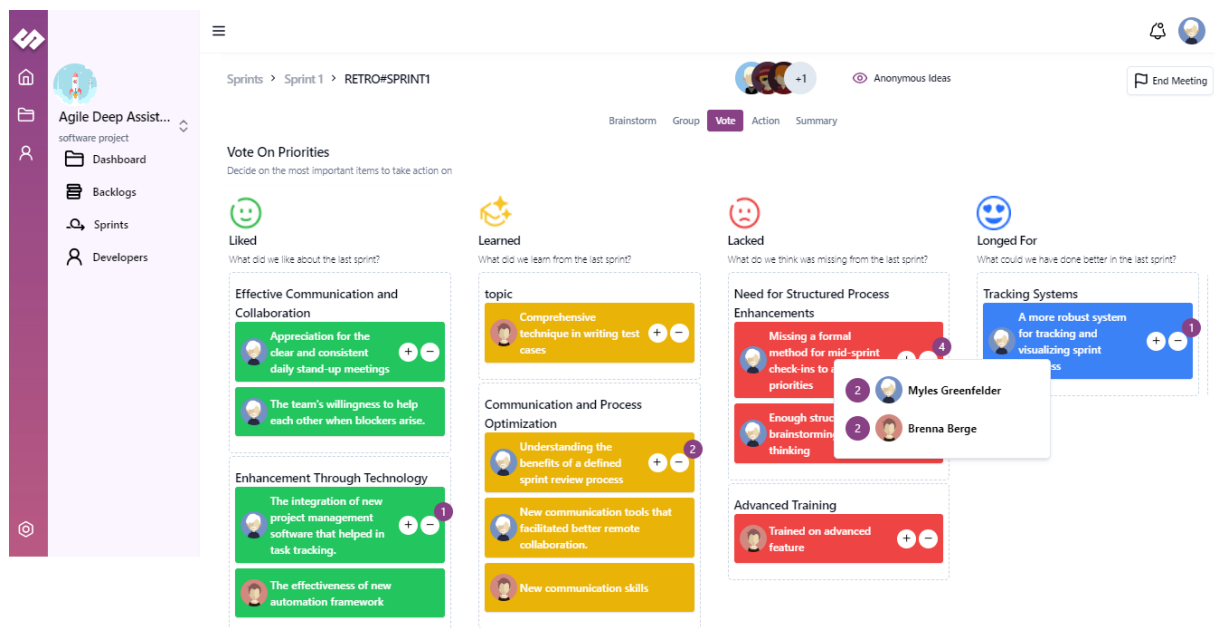


Figure 5.14: Retrospective Room Step3 : Vote Interface

Figure 5.14 outlines the Retrospective Room Step3: Vote Interface, enabling team members to vote on key issues, with details on votes visible to the scrum master.

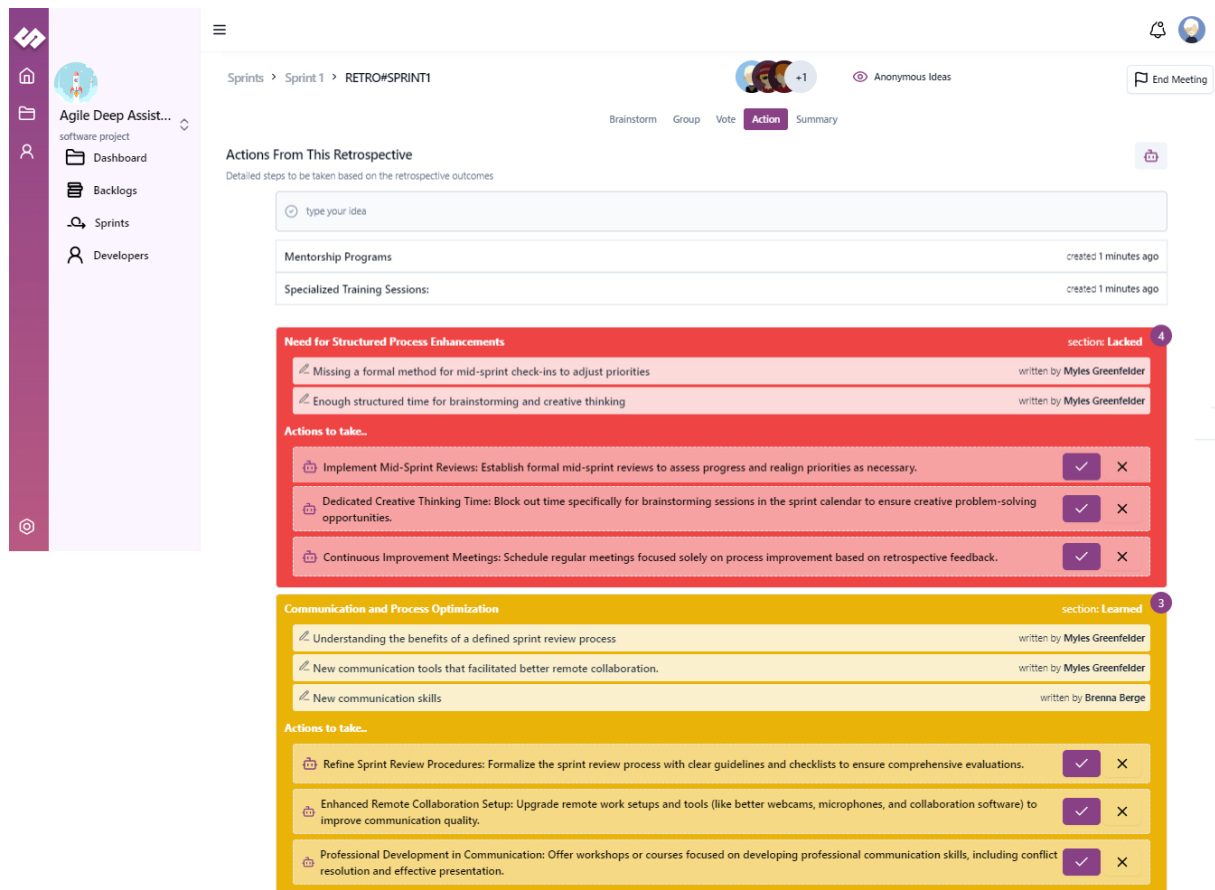


Figure 5.15: Retrospective Room Step4 : Action Interface

Figure 5.15 describes the Retrospective Room Step4: Action Interface, where actionable plans are created based on the voting results and discussed topics. Here, the scrum master can manually add actions and attach them to specific relevant topics or automatically generate appropriate actions for each discussed topic through the RAG system 2. The scrum master then decides whether to accept or reject these actions.

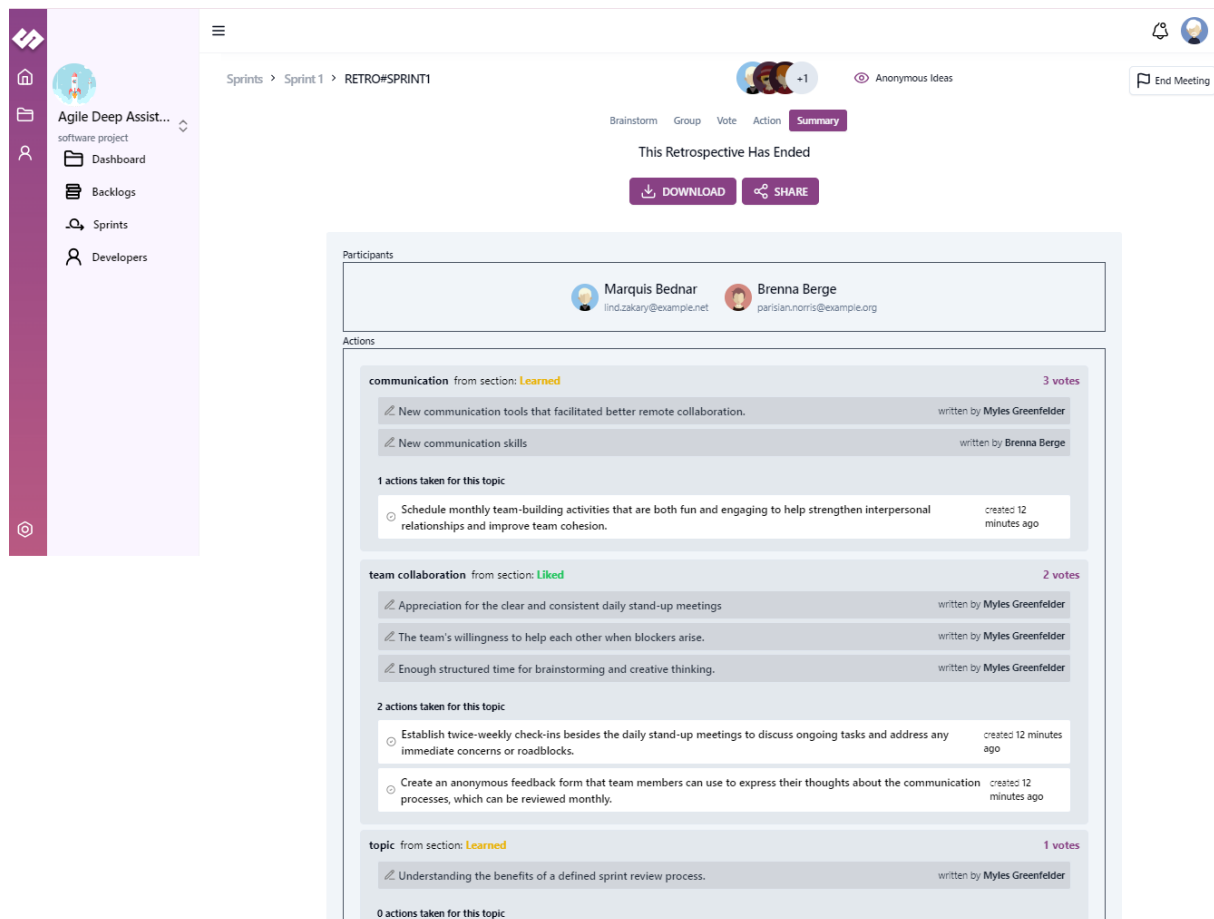


Figure 5.16: Retrospective Closure and Summary Interface

Figure 5.16 illustrates the Retrospective Closure and Summary Interface, summarizing the outcomes, discussed topics, presence participants and actions decided during the session.

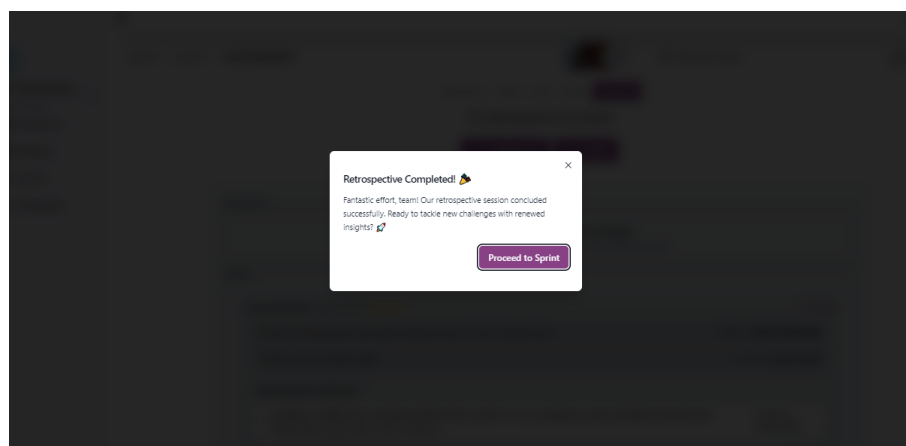


Figure 5.17: End Retrospective Meeting Confirmation Interface

Figure 5.17 outlines the End Retrospective Meeting Interface, confirming the closure of the retrospective session.

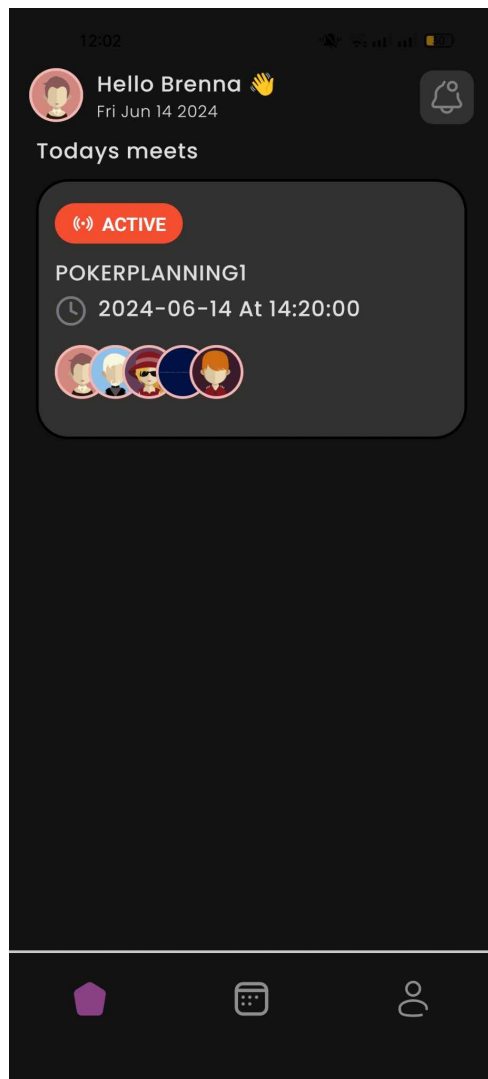


Figure 5.18: Developer Todays Meeting Interface

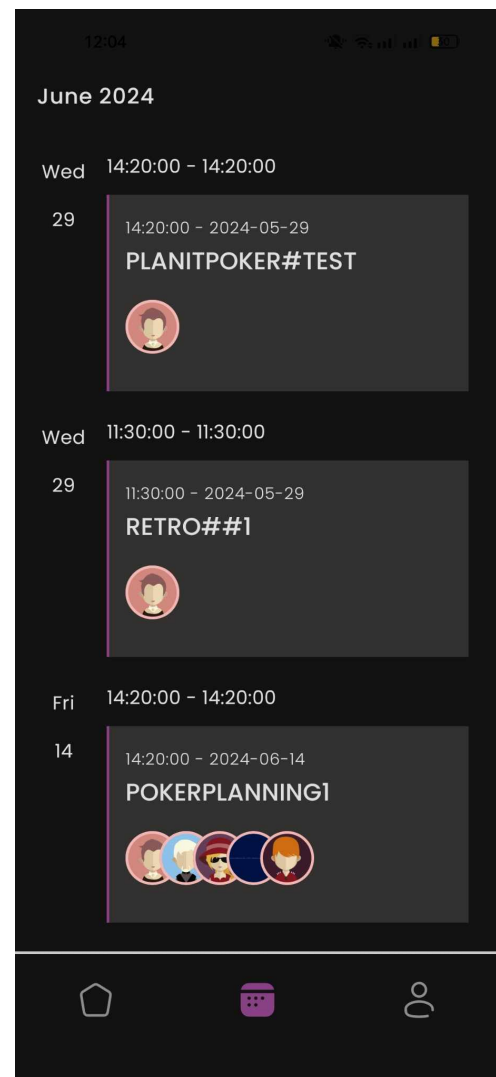


Figure 5.19: Developer Calendar Interface

Figure 5.18 describes the Developer Today's Meeting Interface, detailing the schedule and agenda for developers' daily meetings.

Figure 5.19 illustrates the Developer Calendar Interface, providing an overview of scheduled meetings and events for developers.

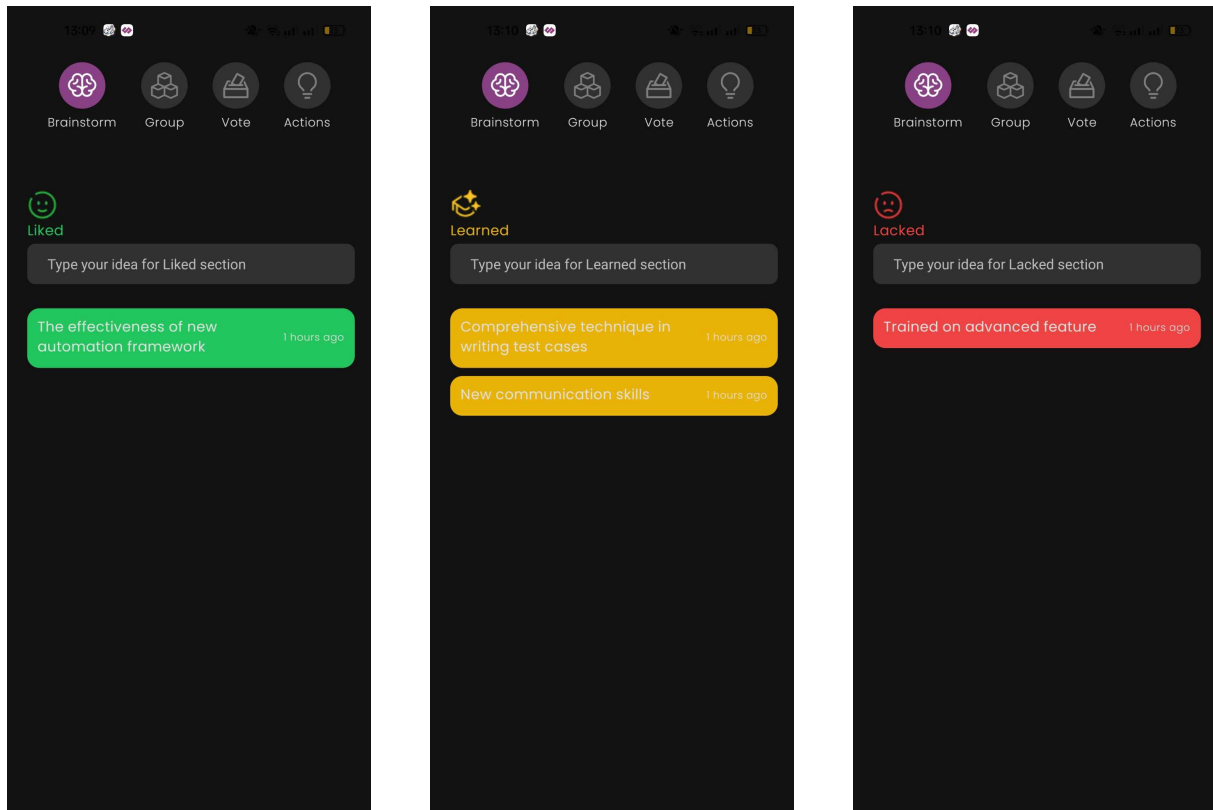


Figure 5.20: Retrospective Room Step1 : Brainstorm Developer Interfaces

Figure 5.20 outlines the Retrospective Room Step1: Brainstorm Developer Interfaces, enabling developers to add their thoughts and ideas through their mobile app during the brainstorming phase.

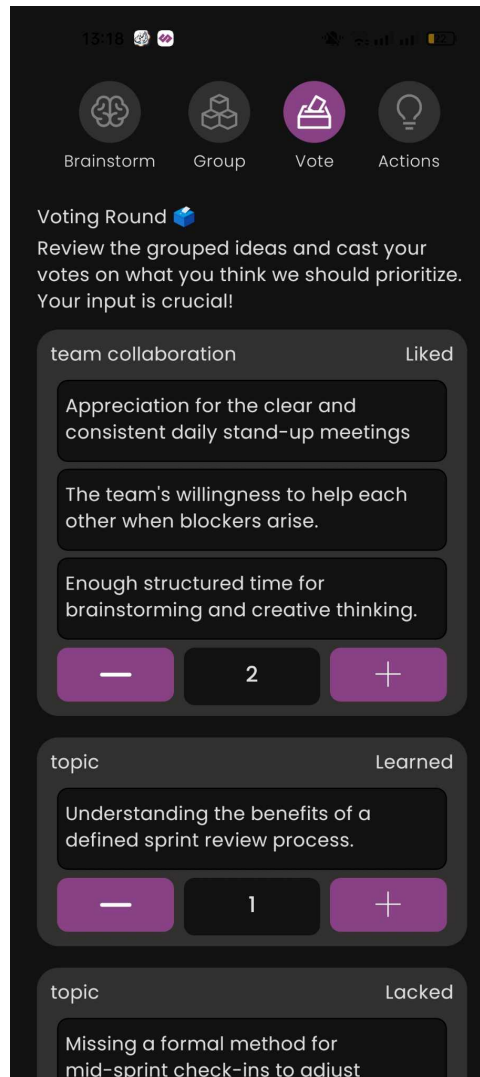


Figure 5.21: Retrospective Room Step3 : Vote Developer Interface

Figure 5.21 describes the Retrospective Room Step3: Vote Developer Interface, focusing on the voting mechanism for developers, displaying a list of topics for developers to consult, review, and vote on as important.

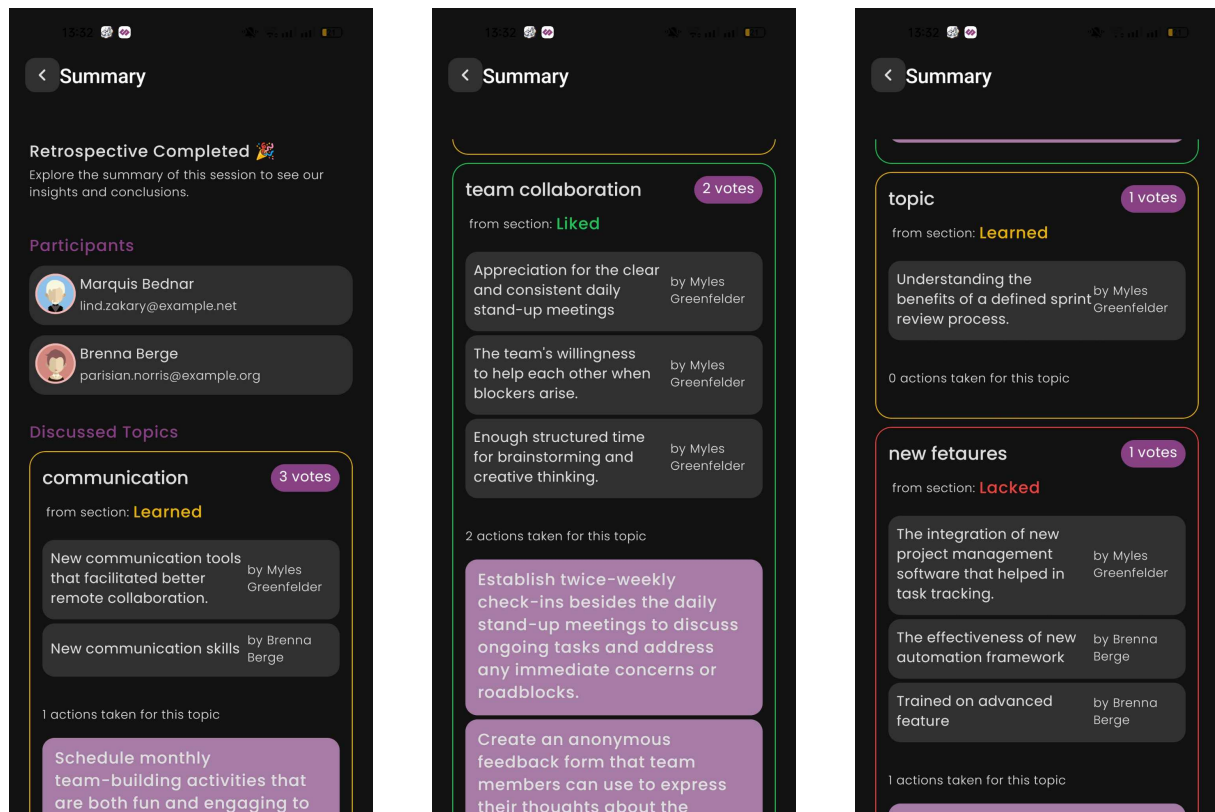


Figure 5.22: Retrospective Summary Developer Interface

Figure 5.22 illustrates the Retrospective Summary Interface for developer, summarizing key outcomes and actions for topics.

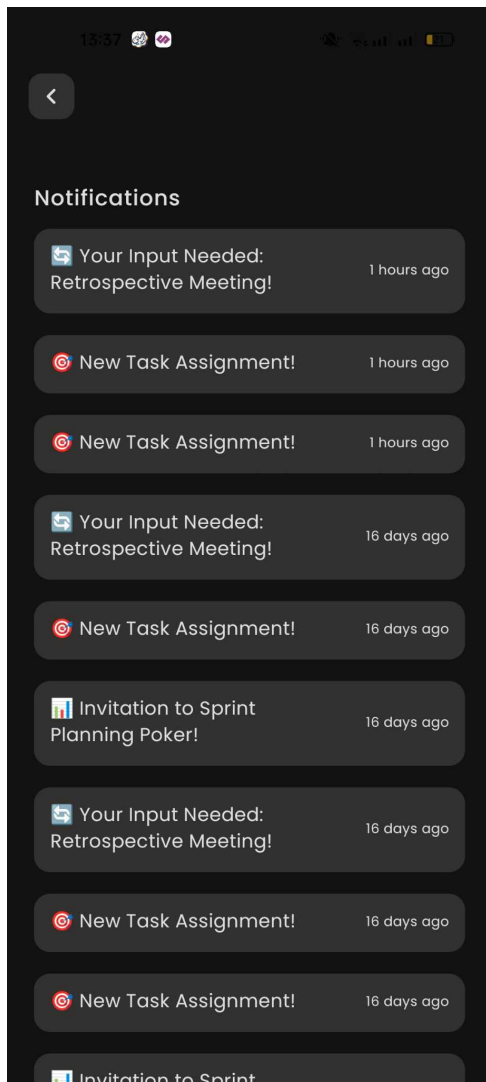


Figure 5.23: Notification Interface

Figure 5.23 outlines the Notification Interface, managing alerts and notifications for developers.

Figure 5.24 describes the Notification Consultation on Device Interface, where developers can review notifications received on their devices.

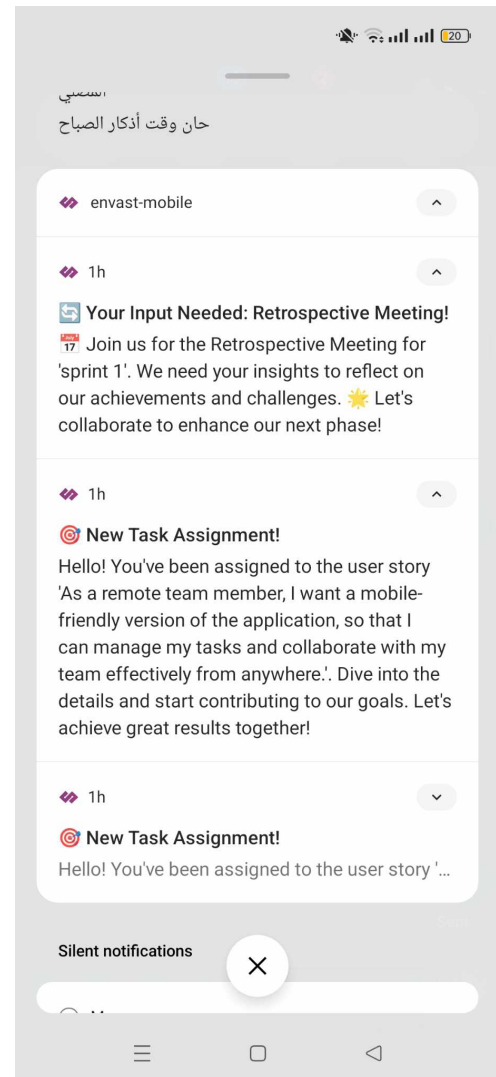


Figure 5.24: Notification Consultation on Device

Conclusion

In this chapter, we conclude the development of our Agile Deep Assistant project, highlighting the implementation of the third and final sprint during this internship. This phase focused on managing retrospectives and integrating a large language model (LLM) within the Retrieval Augmented Generation (RAG) system 2 to generate effective suggestions. These enhancements are designed to guide the Scrum team effectively during improvement meetings.

General Conclusion and Perspectives

This final year project marked a significant milestone in our educational journey, providing a practical opportunity to apply the theoretical knowledge we have acquired. This report summarizes our end-of-study project at ENVAST, which involved designing and developing a Deep Agile Assistant. This tool automates the estimation of sprint user stories and conducting team evaluations, with a focus on improving sprint planning and retrospectives.

Our project uses a Retrieval-Augmented Generation (RAG) setup and a powerful pretrained large language model to improve decision-making and automate various sprint management operations. This approach enables accurate user story estimations and recommends the most suitable developers for each story, crucial for effective sprint planning. Additionally, it enhances decision-making and team evaluations during retrospective meetings by suggesting actionable solutions to the discussed topics and identified issues.

Our project directly integrates with Jira to create and prepare the project environment, configuring it from our solution accordingly. This setup involves creating projects, sprints, user stories, and initiating sprints. During the sprint process, developers can log in and use the planning board (Todo, In Progress, Done) [30] to track the status and progress of their assigned user stories. Subsequently, this data is utilized to update our system.

Looking ahead, we plan to enhance our application by:

- Integrating fully with Jira through webhooks, our application will stay up-to-date with any changes in Jira without manual synchronization through file uploads.
- Incorporating more AI across various aspects to facilitate the work of different Scrum role members. For example, we will explore:
 - **Automating Next Sprint Planning** this will include analyzing the previous sprint and selecting the appropriate next user stories from the product backlog to work on during the next sprint. Additionally, we plan to automate the scheduling of sprint planning meetings by considering the availability of all team members to ensure optimal participation.
 - **AI-Powered Problem Resolution Suggestions:** If a developer came across an issue, the system can automatically suggest potential solutions based on past successful interventions archived in the database. This feature minimizes the time developers spend and provides immediate support. If no existing solution is found, the system will automatically escalate the issue to a senior team member.
 - **Implementing a Daily Scrum Chatbot:** To further streamline the daily scrum process, we plan to develop a chatbot that interacts with developers each morning. This chatbot will automate the traditional daily stand-up meetings by asking developers key questions:

- * **What did you accomplish yesterday?** This helps track progress against the sprint goals.
- * **What will you work on today?** This ensures that the developer is focusing on the right tasks and aligns their daily goals with the sprint objectives.
- * **What challenges are you facing?** Identifying blockers early helps in taking proactive steps to address them.

The chatbot will be integrated into our system, with access to the tasks assigned to each developer, enabling it to focus on the right tasks and use the AI Problem Resolution Suggestions tool we discussed earlier to make informed suggestions for overcoming obstacles.

References

- [1] Advanced rag architecture. <https://ugurozker.medium.com/advanced-rag-architecture-b9f8a26e2608>. Accessed: 2024-05-12.
- [2] Atlassian developer - jira platform rest apis. <https://developer.atlassian.com/server/jira/platform/rest-apis/>. Accessed: 2024-03-01.
- [3] Chromadb - homepage. <https://www.trychroma.com/>. Accessed: 2024-05-10.
- [4] Domain-driven design (ddd). <https://www.geeksforgeeks.org/domain-driven-design-ddd/>. Accessed: 2024-03-18.
- [5] Embedding models - ollama. <https://ollama.com/blog/embedding-models/>. Accessed: 2024-05-19.
- [6] Envast - home page. <https://envast.tn/fr>. Accessed: 2024-02-02.
- [7] Expo. <https://expo.dev/>. Accessed: 2024-04-03.
- [8] Expo documentation - push notifications overview. <https://docs.expo.dev/push-notifications/overview/>. Accessed: 2024-04-09.
- [9] Fastapi. <https://fastapi.tiangolo.com/>. Accessed: 2024-04-30.
- [10] Figma. <https://www.figma.com/>. Accessed: 2024-02-02.
- [11] Gitlab. <https://about.gitlab.com/>. Accessed: 2024-04-03.
- [12] Jupyter notebook. <https://jupyter.org/>. Accessed: 2024-05-15.
- [13] Langchain. <https://www.langchain.com/>. Accessed: 2024-05-03.
- [14] Laravel - the php framework for web artisans. <https://laravel.com/>. Accessed: 2024-02-10.
- [15] Learn latex. <https://www.overleaf.com/learn>. Accessed: 2024-03-03.
- [16] Llama. <https://www.openai.com/>. Accessed: 2024-05-03.
- [17] Mysql. <https://www.mysql.com/>. Accessed: 2024-02-01.
- [18] Plantuml. <https://plantuml.com/>. Accessed: 2024-03-10.
- [19] Postman. <https://www.postman.com/>. Accessed: 2024-03-03.
- [20] React native. <https://reactnative.dev/>. Accessed: 2024-04-04.
- [21] Reactjs. <https://reactjs.org/>. Accessed: 2024-02-20.
- [22] Retrieval-augmented generation. <https://www.willowtreeapps.com/craft/retrieval-augmented-generation>. Accessed: 2024-05-10.

- [23] Visual studio code. <https://code.visualstudio.com/>. Accessed: 2024-02-03.
- [24] What are embeddings in machine learning? <https://www.cloudflare.com/learning/ai/what-are-embeddings/>. Accessed: 2024-06-22.
- [25] What is scrum? <https://www.scrum.org/resources/what-scrum-module>. Accessed: 2024-02-18.
- [26] The single responsibility principle (srp) of solid. <https://giovannamoeller.medium.com/the-single-responsibility-principle-srp-of-solid-eb2feed0c64b>, Accessed: 2024-02-15.
- [27] Separation of concerns (soc). <https://www.geeksforgeeks.org/separation-of-concerns-soc/#what-does-soc-stand-for>, Accessed: 2024-03-09.
- [28] Solid: Dependency inversion principle. <https://medium.com/@inzuael/solid-dependency-inversion-principle-part-5-f5bec43ab22e>, Accessed: 2024-03-13.
- [29] Facade design pattern in software development. <https://psid23.medium.com/facade-design-pattern-8db51f37cc44>, Accessed: 2024-04-08.
- [30] Jira: Tracking status and progress of issues. <https://confluence.atlassian.com/jiraportfoliocloud/tracking-progress-and-status-828785247.html>, Accessed: 2024-06-08.
- [31] Dr. Selma Belgacem Ben Mansour. Génie logiciel :processus de développement : Méthodologies agiles. *.GL courses, ISSAT of Sousse, 2020*, pages 12–18, 2023.

ANNEXE

A.1 Sprint Management System

Title	Start a Sprint
Actor	Scrum Team Leader (primary actor), JIRA (secondary actor)
Precondition	The Scrum Team Leader is authenticated and has access to the sprint within the project management system.
Postcondition	The selected sprint is started both within the internal system and in JIRA, and ready for task execution and tracking.
Main Scenario	1. Scrum Team Leader clicks the 'Start Sprint' button on the Sprint Screen. 2. The system receives the request and checks if the sprint is already active. 3. If no active sprint is found, the system activates the sprint. 4. The system uses JIRA's REST API to start the sprint in JIRA. 5. JIRA confirms the sprint start, and the system receives this confirmation. 6. The system displays a success message indicating that the sprint has started successfully.
Exception Scenario	- If the system finds that the sprint is already active, it displays an error message stating, "An active sprint already exists".

Table A.1: Use Case Specification for "Start a Sprint"

A.2 Sprint Planning Management System

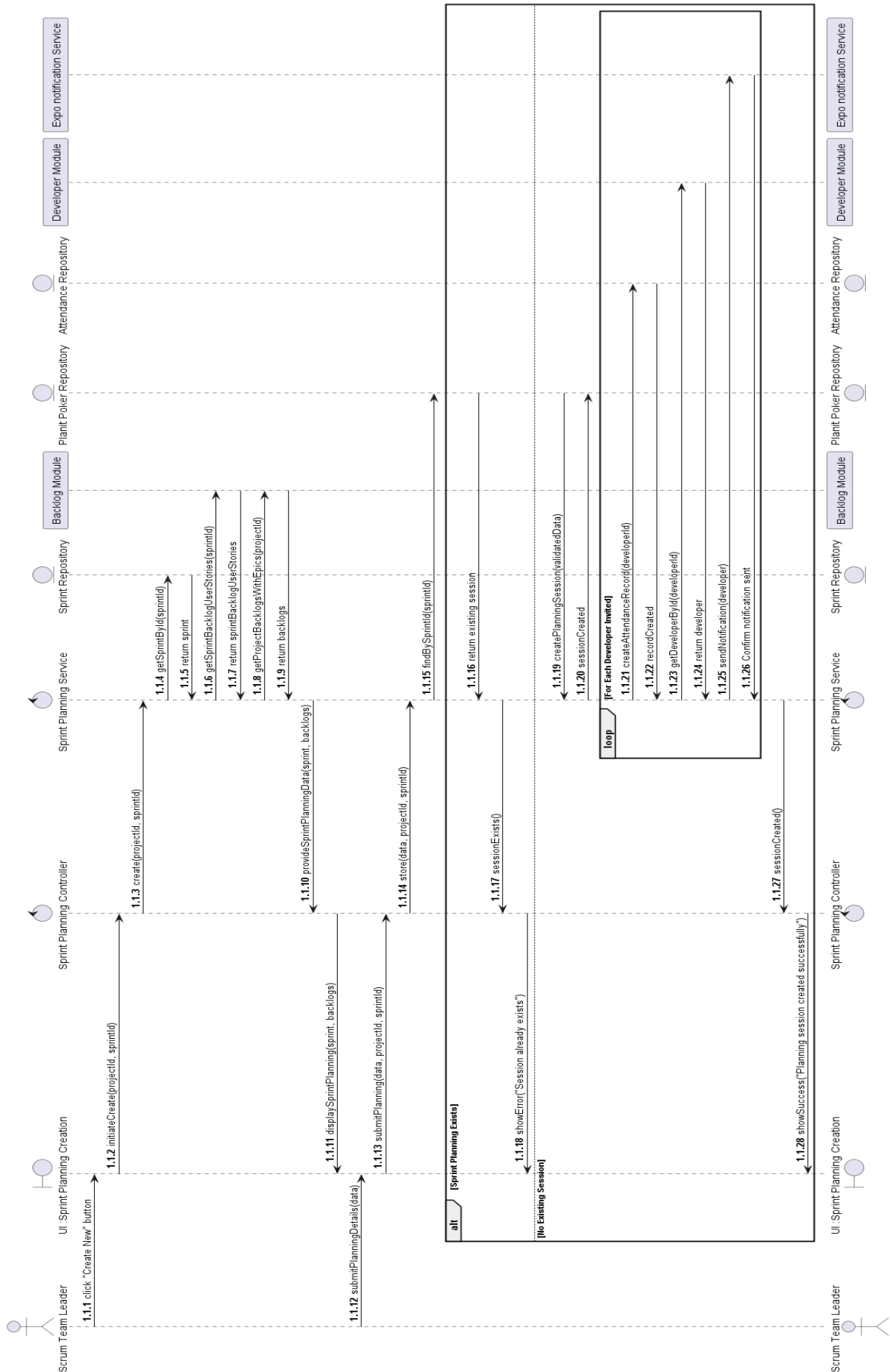


Figure A.1: Create and Store Poker Planning Sequence Diagram



Figure A.2: Developer User Story Estimation Sequence Diagram

A.3 Retrospective Management System

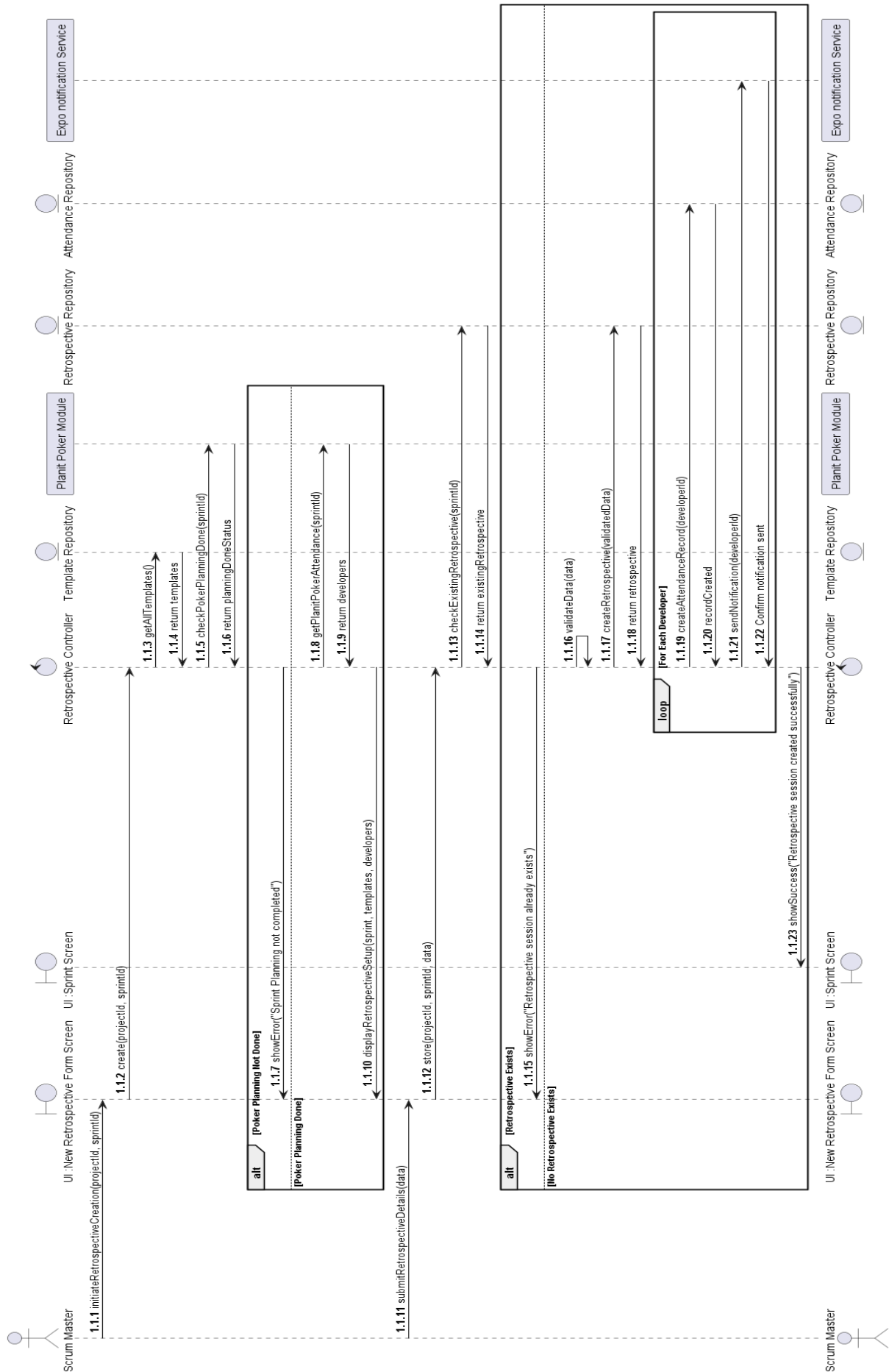


Figure A.3: Create and Store Retrospective Sequence Diagram

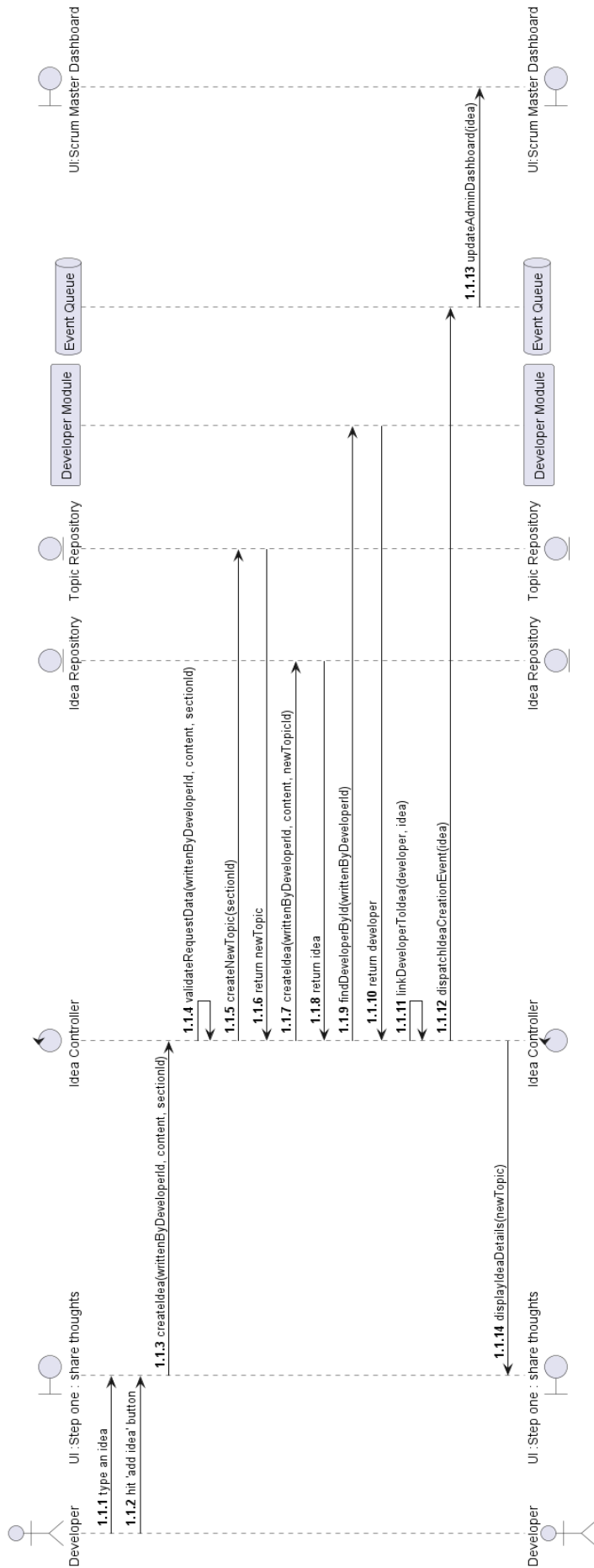


Figure A.4: Create an idea Sequence Diagram

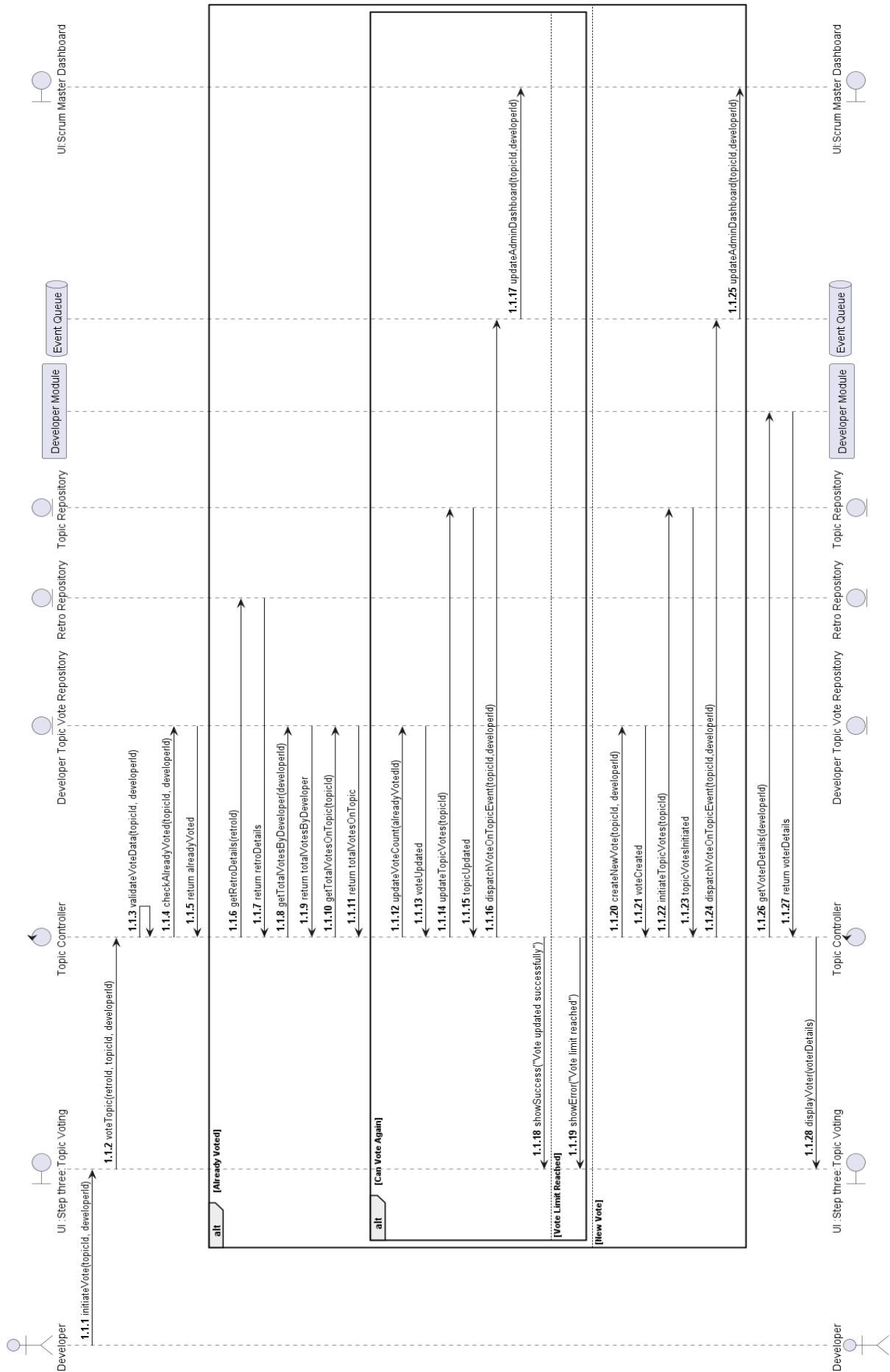


Figure A.5: Vote on a Topic Sequence Diagram